uc3m | Universidad Carlos III de Madrid

Grado Universitario en Gestión de la información y contenidos digitales
2024-2025

Trabajo Fin de Grado

"El uso de la IA en clasificación automática de documentos textuales"

Pablo Blas Hernández

Tutor

Ricardo Eito Brun

Getafe, 2025



[Incluir en el caso del interés de su publicación en el archivo abierto]

Esta obra se encuentra sujeta a la licencia Creative Commons **Reconocimiento – No Comercial – Sin Obra Derivada**

RESUMEN

En la era digital, el número de documentos que generan a diario las empresas e instituciones ha crecido de manera exponencial, presentando diferentes desafíos para su organización y clasificación. Numerosas organizaciones todavía recurren a procedimientos manuales para la gestión de esta información, lo que supone un alto consumo de tiempo y recursos humanos. La falta de automatización en este proceso dificulta la búsqueda eficiente de información, retrasa la toma de decisiones y puede provocar problemas en la gestión de grandes cantidades de datos.

Este Trabajo de Fin de Grado busca demostrar que la Inteligencia Artificial, a través de algoritmos de *Machine Learning* y *Deep Learning*, puede resolver de manera eficiente el problema de la clasificación de documentos de manera automática. La idea principal es probar que estos métodos de IA son capaces de organizar grandes cantidades de textos de forma precisa. Para esto, se va a desarrollar un modelo de IA que analizará los procedimientos a seguir y las principales características que debe cumplir un modelo, para entender qué condiciones optimizan la clasificación, y así, obtener los mejores resultados.

Palabras clave
Inteligencia Artificial, clasificación de documentos, procesamiento de lenguaje natural, documentos, aprendizaje automático.

AGRADECIMIENTOS

Al profesorado y a mis compañeros de la uc3m, en especial a Ricardo, mi tutor del TFG, por su guía y apoyo en esta aventura llamada universidad.

A mis amigos de siempre, de la universidad y del erasmus, por sacarme una sonrisa y estar ahí cuando más los necesito.

A mi familia, a quienes están y a quienes ya no están, pero me siguen acompañando en cada momento de mi vida. Gracias por ser siempre mi apoyo incondicional y mi mayor refugio. En especial, a mis abuelas, a mi madre, a mi padre y a mi hermano.

ÍNDICE DE CONTENIDOS

CAPÍTULO 1: INTRODUCCIÓN 1

1.1. Contexto y relevancia del problema de clasificación de documentos 1

1.2. Objetivos 2

1.3. Metodología 3

1.4. Motivación y justificación 4

CAPÍTULO 2: MARCO TEÓRICO 6

2.1. Inteligencia Artificial 6

2.2. *Machine Learning* en la Clasificación de Documentos 8

2.3. *Deep Learning* y Redes Neuronales en Clasificación de Documentos 8

2.4. Procesamiento de Lenguaje Natural (NLP) 10

2.4.1. Técnicas Preliminares de Procesamiento de Texto 11

2.4.2. Métodos de Representación de Texto 12

2.4.3. Modelos de lenguaje basados en Transformer 15

2.5. Algoritmos de Clasificación de documentos 17

2.5.1. Árboles de decisión y *Random Forest* 17

2.5.2. K-Nearest Neighbors (k-NN) 18

2.5.3. Naïve Bayes 18

2.5.4. Máquinas de Vectores de Soporte (SVM) 18

2.5.5. Otras técnicas de *clustering* 19

2.6. Evaluación de Modelos en Clasificación de Documentos 20

2.6.1. Matriz de confusión 20

2.6.2. Exactitud ("*Accuracy"*) 21

2.6.3. Precisión ("*Precision"*) 22

2.6.4. Exhaustividad ("*Recall"*) 22

2.6.5. *F1-score* 23 ……23

CAPÍTULO 3: REVISIÓN DE LA LITERATURA 24

3.1. Relación entre clasificadores textuales y técnicas de clasificación basados en indicadores bibliométricos. 24

3.2. Casos prácticos de clasificación automática 25

3.3. *Clustering* y UX mediante recomendaciones de libros. 28

3.4. *Clustering* y otras técnicas relacionadas (LDA, *embeddings*, etc.) 29

3.5. *Clustering*, clasificación y técnicas bibliométricas. 34

CAPÍTULO 4: CASO PRÁCTICO DE ALGORITMO DE CLASIFICACIÓN DE DOCUMENTOS TEXTUALES 36

4.1. Obtención de los datos 36

4.2. Clasificación de Documentos con MATLAB 37

4.2.1. Funcionamiento y Aplicaciones de MATLAB 38

4.2.2. Ventajas y Limitaciones de MATLAB 39

4.2.3. Algoritmos en MATLAB para la Clasificación Automática de Documentos ……..……41

4.3. Evaluación de resultados de clasificación 45

4.4. Sistema de recomendación de documentos para el usuario 65

4.4.1. Planteamiento: recomendación basada en contenido 65

4.4.2. Funcionamiento del sistema 65

4.4.3. Limitaciones y posibles mejoras 66

4.5. Conclusiones y líneas futuras de investigación 68

BIBLIOGRAFÍA 70

ÍNDICE DE FIGURAS

Figura 2.1: Estructura de una red neuronal 9
Figura 2.2: Matriz de confusión 21
Figura 4.3: Dendograma de clustering jerárquico 52

ÍNDICE DE TABLAS

Tabla 4.1: Distribución porcentual de tópicos por documento 61

CAPÍTULO 1: INTRODUCCIÓN

1.1. Contexto y relevancia del problema de clasificación de documentos

La gestión de la documentación es posiblemente uno de los procesos más importantes para cualquier organización, puesto que un documento no es solo una herramienta práctica para la comunicación y gestión, sino también un medio esencial para conservar el conocimiento humano. Como señala Rodríguez Pérez (2023), los documentos constituyen un pilar esencial en la gestión del conocimiento, asegurando que ideas, investigaciones y hallazgos permanezcan accesibles para futuras generaciones. Bilski (2011, p. 263) considera que – dada la relevancia de los documentos electrónicos –, una correcta clasificación es crucial en el proceso de descubrimiento y asimilación del conocimiento. Sin embargo, a pesar de su relevancia, la documentación suele ser subestimada en la vida cotidiana, lo que conlleva a descuidar su correcta gestión y conservación.

La clasificación de documentos se puede definir como el proceso de agrupar documentos en categorías específicas en función de características compartidas. Estas categorías pueden representar distintos aspectos del documento, como el idioma en el que están escritos, el tema principal que abordan, o cualquier otro criterio relevante para su organización. Según Bilski (2011, p. 263), la clasificación de texto es una parte fundamental del proceso de gestión de la información. Este proceso es particularmente relevante en contextos como bibliotecas digitales, sistemas de archivo empresariales, y la gestión documental en instituciones académicas y gubernamentales. Sin embargo, debido a la enorme cantidad de datos disponibles, realizar esta tarea de manera manual resulta inviable, ya que requiere un esfuerzo significativo en términos de tiempo y recursos.

Durante los últimos años, la cantidad de documentos en formato electrónico ha crecido de manera exponencial, lo que ha generado la necesidad de desarrollar métodos y procesos que permitan su organización, gestión y recuperación de manera eficiente, y es en este contexto, donde nace la clasificación automática de documentos.

El uso de herramientas informáticas hace posible solventar este problema; no solo optimizan los procesos internos de las organizaciones, sino que también mejoran la experiencia de usuario (“UX”) al proporcionar un acceso más rápido y estructurado a la información que se precisa en cada momento. Por ejemplo, en una biblioteca digital universitaria, los artículos pueden clasificarse según distintos criterios como, por ejemplo, las distintas facultades que componen dicha universidad: “Ciencias Sociales”, “Ingeniería”, “Medicina”, “Humanidades”, etc. Este tipo de segmentaciones facilitan la recuperación de información y evitan la dispersión de documentos en múltiples áreas temáticas sin una estructura clara. Como señalan Baharudin et al. (2010, p. 4), las técnicas de minería de texto, aprendizaje automático y procesamiento del lenguaje natural permiten clasificar documentos electrónicos y bibliotecas digitales de forma

más precisa, reduciendo significativamente la necesidad de intervención humana. Estas metodologías optimizan tanto la representación como la extracción de información relevante, y enfrentan desafíos como la reducción de dimensionalidad, la selección del clasificador adecuado y la anotación precisa de los documentos.

La clasificación automática de documentos parece fácil en la teoría, pero resulta una tarea muy complicada de llevar a cabo en la práctica. Uno de los principales desafíos es la dificultad para determinar la categoría que mejor los describe. Dependiendo del enfoque utilizado y de los criterios de clasificación, un mismo documento puede encajar en diferentes categorías, lo que añade un nivel adicional de complejidad. Imaginemos un documento titulado "Impacto del cambio climático en la economía agrícola". El documento contiene información acerca de patrones climáticos, políticas medioambientales y efectos económicos en la producción agrícola. Dependiendo del contexto, el documento podría clasificarse en 'Medio Ambiente', 'Economía' o 'Agricultura', lo que genera ambigüedad para los algoritmos de clasificación automática, que deben decidir qué categoría representa con mayor precisión al documento o si se requiere una clasificación en múltiples categorías.

Siguiendo a Bilski (2011, p. 263), cuando hablamos de clasificación de textos, no se trata solo de asignar las clases correctas a los documentos. Un reto mayor para el clasificador son las funciones semánticas, que se refieren a cómo se relacionan las expresiones con la realidad. El significado de una palabra depende mucho del contexto en el que se use (problema de los homónimos). Además, los conceptos más abstractos también tienen que ser considerados dentro del problema de la clasificación. Otro aspecto importante en la clasificación de textos es el problema de la alta dimensionalidad, donde múltiples características describen un solo texto. Por eso, el clasificador tiene que ser muy preciso al asignar los documentos a las categorías correspondientes.

Para mejorar la precisión en la clasificación, es fundamental evaluar distintos algoritmos de aprendizaje automático y determinar cuáles ofrecen mejores resultados en distintos escenarios. En este Trabajo de Fin de Grado se analizan, a través de experimentos y pruebas en conjuntos de datos acotados, las condiciones que permitan optimizar el rendimiento de los clasificadores y mejorar la clasificación automática de documentos.

1.2. Objetivos

El objetivo general de este Trabajo de Fin de Grado es analizar y demostrar la aplicabilidad de la Inteligencia Artificial en la clasificación automática de documentos textuales, mediante una revisión de la literatura existente y una implementación práctica en entorno MATLAB. Para alcanzar dicho objetivo del trabajo, se han definido los siguientes objetivos específicos, que detallan el proceso metodológico de investigación y aplicación práctica del proyecto:

- Estudiar los fundamentos teóricos de la Inteligencia Artificial, el aprendizaje automático y el procesamiento de lenguaje natural aplicados a la clasificación de documentos textuales.
- Analizar comparativamente los principales algoritmos de clasificación de texto que, incluyen Naïve Bayes, SVM, árboles de decisión, k-NN y redes neuronales, haciendo hincapié en sus ventajas, limitaciones y escenarios de aplicación.
- Explorar técnicas modernas de representación de texto (TF-IDF, *Word Embeddings*, *Transformers*) y preprocesamiento (tokenización, lematización, eliminación de *stopwords*), para comprender su impacto en el rendimiento de los modelos.
- Revisar casos de éxito documentados en la literatura científica, donde se ha aplicado la IA a tareas de clasificación documental, con el fin de identificar buenas prácticas, desafíos comunes y resultados alcanzados en contextos reales (bibliotecas, archivos, repositorios, etc.).
- Implementar un modelo básico de clasificación automática en MATLAB con el fin de demostrar la aplicación de la IA sobre este campo, evaluando su funcionamiento sobre un conjunto reducido de documentos.
- Extraer conclusiones sobre la eficacia de la IA en la clasificación de documentos textuales y formular propuestas de mejora y líneas futuras de trabajo enfocadas a su aplicación práctica en entornos reales como repositorios institucionales o bibliotecas digitales.

1.3. Metodología

La metodología seguida en este Trabajo de Fin de Grado combina una revisión teórica con una implementación práctica orientada a evaluar la aplicabilidad de la Inteligencia Artificial en la clasificación automática de documentos textuales.

En una primera fase, se realizó una revisión sistemática del marco teórico, centrada en obras académicas, artículos científicos y documentación técnica relacionada con Inteligencia Artificial, aprendizaje automático, aprendizaje profundo y procesamiento de lenguaje natural. Para ello, se consultaron fuentes académicas procedentes de plataformas como Springer, Google Scholar y ResearchGate, así como información técnica ofrecida por empresas punteras como IBM. Por otro lado, la revisión de la literatura, enfocada en identificar casos de éxito y estudios previos sobre clasificación documental y tecnologías afines, se llevó a cabo mediante la base de datos LISTA (*Library, Information Science & Technology Abstracts*), especializada en biblioteconomía y ciencias de la información, accesible a través de la Biblioteca de la Universidad Carlos III de Madrid.

La elección de MATLAB como entorno de implementación práctica se debió a su amplio soporte para algoritmos de clasificación supervisada, y extendida documentación que te guía a lo largo del proceso de desarrollo de modelos.

Los datos utilizados en la parte práctica del trabajo se obtuvieron del repositorio institucional e-Archivo de la Universidad Carlos III de Madrid, seleccionando una muestra representativa de documentos textuales académicos. Estos textos fueron sometidos a un proceso de pretratamiento que incluyó tareas como tokenización, lematización y eliminación de palabras vacías, con el objetivo de preparar los datos para su análisis automático. Posteriormente, se desarrolló un experimento en MATLAB en el que se aplicaron los distintos algoritmos seleccionados sobre el conjunto de datos procesado. Finalmente, se analizaron e interpretaron los resultados obtenidos, identificando fortalezas y debilidades de cada modelo, y se formularon conclusiones que apuntan a posibles mejoras y aplicaciones futuras en contextos documentales reales, como bibliotecas digitales o repositorios académicos.

1.4. Motivación y justificación

Este Trabajo de Fin de Grado se fundamenta en la necesidad de aplicar tecnologías basadas en Inteligencia Artificial (IA) para optimizar los procesos de clasificación automática de documentos, un reto clave en el contexto actual de sobrecarga informativa y transformación digital. La elección de esta temática responde tanto a su relevancia en la nueva sociedad digitalizada, como a su perfecta sintonía con el perfil formativo del Grado en Gestión de la información y contenidos digitales, un itinerario académico que combina la ciencia de la documentación con competencias en tecnologías emergentes como la IA, el Big Data o el análisis automatizado de información. Esta formación híbrida proporciona una base idónea para abordar el presente proyecto desde una perspectiva multidisciplinar e innovadora.

Además, el potencial de la IA en el ámbito documental se traduce en una serie de beneficios tales como:

- Optimización de tiempo y recursos: La automatización de la clasificación documental reduce el tiempo requerido para procesar grandes volúmenes de información y minimiza la cantidad de recursos necesarios para abordar estos procesos, permitiendo que dichos recursos se destinen a otras tareas.

- Precisión y consistencia: Los modelos de IA pueden ser entrenados para aplicar criterios de clasificación comunes y sin distinciones de sesgo, evitando errores humanos y garantizando una mayor objetividad en la categorización de documentos.

- Aplicabilidad en repositorios institucionales: La clasificación de documentos tiene una gran utilidad en los repositorios institucionales, ya que en ellos se recoge y organiza la mayor parte del conocimiento generado dentro de una institución, como trabajos académicos, investigaciones o artículos. Esto facilita el acceso a la información y mejora su gestión, algo fundamental en entornos educativos y científicos.

- Escalabilidad y adaptabilidad: Los algoritmos de IA pueden manejar volúmenes crecientes de datos sin comprometer el rendimiento, y pueden ser ajustados o reentrenados para adaptarse a nuevos tipos de documentos o cambios en los criterios de clasificación.

Dado el potencial de la IA en la clasificación de documentos, este TFG se enfoca en demostrar la aplicabilidad y adaptabilidad de esta tecnología. La investigación contribuirá al avance en el uso de IA en la gestión de información, promoviendo soluciones innovadoras para una organización y recuperación de documentos de manera automática.

CAPÍTULO 2: MARCO TEÓRICO

2.1. Inteligencia Artificial

La Inteligencia Artificial es una rama de la informática centrada en desarrollar máquinas capaces de razonar e imitar el comportamiento humano, transfiriendo a ellas conocimiento para que actúen de forma similar a las personas. La IA se ha convertido en una de las áreas de mayor crecimiento, con aplicaciones cotidianas como la personalización de recomendaciones, lo que está transformando nuestra forma de vivir, trabajar e interactuar (Rodríguez Pérez, 2023, p. 3).

La Inteligencia Artificial (IA) es una disciplina que busca entender y construir sistemas capaces de comportarse de manera inteligente. A diferencia de otras ciencias que intentan solo explicar fenómenos, la IA también se ocupa de diseñar agentes que puedan actuar con eficacia y seguridad en situaciones nuevas y complejas. Esta área de estudio abarca desde tareas generales como el aprendizaje, la percepción o el razonamiento, hasta aplicaciones más específicas como conducir vehículos, diagnosticar enfermedades o escribir textos creativos. Su impacto actual no solo es tecnológico, sino también económico y social, y se perfila como una de las ramas de la informática con mayor proyección a futuro (Russell & Norvig, 2021, p. 19).

A lo largo del tiempo, los investigadores han definido la inteligencia artificial desde distintos enfoques. Estos pueden agruparse en cuatro perspectivas principales, que surgen de la combinación de dos dimensiones: si el objetivo es imitar la inteligencia humana o actuar de forma racional, y si se evalúa el pensamiento interno o el comportamiento observable. (Russell & Norvig, 2021, p. 20).

1. Actuar como un Humano - La Prueba de Turing: Uno de los primeros intentos por definir la Inteligencia Artificial fue la prueba de Turing, propuesta por Alan Turing en 1950. Esta prueba plantea que una máquina se considera inteligente si sus respuestas en una conversación escrita son indistinguibles de las de un humano. Para ello, un sistema de IA debe contar con capacidades avanzadas de Procesamiento del Lenguaje Natural (NLP), representación del conocimiento, razonamiento y aprendizaje automático (Russell y Norvig, 2021, p. 20). Sin embargo, expertos consideran que esta prueba no es suficiente para demostrar inteligencia, ya que no implica una comprensión real del mundo ni una toma de decisiones óptima.

2. Pensar como un Humano - Modelado Cognitivo: Otra forma de abordar la IA es intentar replicar los procesos de pensamiento humano. Este enfoque, basado en la ciencia cognitiva, busca modelar la forma en que los humanos perciben, razonan y aprenden. Para ello, se emplean técnicas como la introspección, los experimentos psicológicos y la neuroimagen. Investigadores como Newell y Simon (1961) desarrollaron modelos computacionales del pensamiento humano, como el "*General Problem Solver*" (GPS), que intentaban imitar la resolución de problemas humanos (Russell y Norvig, 2021, p.

21). No obstante, este enfoque enfrenta dificultades como las diferencias individuales en el razonamiento: cada individuo razona de manera distinta y no existe un razonamiento más adecuado que otro.

3. Pensar Racionalmente - Las Leyes del Pensamiento: Desde una perspectiva formal, la IA se define como la aplicación de la lógica y las matemáticas para alcanzar conclusiones correctas. Este enfoque se basa en los trabajos filosóficos de autores como Aristóteles, quien formuló reglas de inferencia lógica, y en el desarrollo de sistemas formales en el siglo XIX. A mediados del siglo XX, se desarrollaron programas que podían resolver problemas descritos en notación lógica, dando origen a la tradición "logicista" dentro de la IA (Russell y Norvig, 2021, p. 21). Sin embargo, esta aproximación es limitada, ya que muchos problemas del mundo real involucran incertidumbre, lo que imposibilita su resolución mediante reglas lógicas únicamente.

4. Actuar Racionalmente - Agentes Inteligentes: El enfoque que más predomina en la actualidad, que define la IA como el estudio de sistemas que toman decisiones óptimas en función de su entorno y objetivos. Un agente inteligente es aquel que actúa para maximizar sus posibilidades de éxito, incluso en condiciones de incertidumbre (Russell y Norvig, 2021, p. 22). Este enfoque es más amplio que el de las "leyes del pensamiento", ya que permite la incorporación de herramientas probabilísticas, aprendizaje automático y heurísticas para la toma de decisiones.

En el contexto de la clasificación de documentos, la IA se traduce en la capacidad de los algoritmos para identificar patrones en grandes volúmenes de texto, facilitando la organización y recuperación de la información. Su impacto en la clasificación automática de documentos es particularmente relevante, ya que ofrece soluciones que van desde técnicas tradicionales de aprendizaje automático hasta modelos avanzados de aprendizaje profundo.

A pesar de sus beneficios, la clasificación automática de documentos mediante IA presenta distintos problemas que pueden afectar a su autenticidad. Uno de los principales problemas es la presencia de sesgos en los datos de entrenamiento, lo que puede llevar a errores en la categorización o la exclusión de ciertos documentos. Como explica Ferrara (2024, p. 2), estos sesgos pueden originarse en diferentes etapas del proceso, desde la recolección de datos hasta el diseño de los algoritmos, e incluso en la interacción de los usuarios con el sistema. Además, interpretar los resultados que genera un modelo no siempre es sencillo, especialmente cuando se trata de algoritmos complejos o poco transparentes, lo que complica la detección de posibles errores o discriminaciones (Ferrara, 2024, p. 6).

Para abordar estos desafíos, es fundamental desarrollar sistemas transparentes y auditables, que permitan comprender cómo y por qué se asigna una categoría a un documento en particular. Asimismo, la combinación de tecnologías inteligentes con la supervisión humana puede mejorar la precisión de los sistemas y reducir el impacto de posibles errores.

2.2. *Machine Learning* en la Clasificación de Documentos

El aprendizaje automático o *Machine Learning* (ML) es una subdisciplina de la IA que permite a los sistemas mejorar su rendimiento en tareas específicas a través de la experiencia, sin necesidad de ser programados explícitamente. En tareas de clasificación, ML se ha convertido en una herramienta clave debido a su capacidad para procesar grandes cantidades de datos y extraer características relevantes para la categorización. Los algoritmos de ML pueden dividirse en tres categorías principales:

- Aprendizaje supervisado: El aprendizaje supervisado se define como conjuntos de datos etiquetados para entrenar algoritmos que clasifiquen datos o predigan resultados con precisión. A medida que se introducen datos de entrada en el modelo, éste ajusta sus ponderaciones hasta lograr las más adecuadas para obtener el mejor resultado posible. Esto ocurre como parte del proceso de validación cruzada para garantizar que el modelo evite el llamado "overfitting[1]" o "underfitting[2]" (IBM, 2024a).

- Aprendizaje no supervisado: El aprendizaje no supervisado utiliza algoritmos de *Machine Learning* para analizar y agrupar conjuntos de datos no etiquetados en subconjuntos denominados clúster. Estos algoritmos descubren patrones ocultos o agrupaciones de datos sin necesidad de intervención humana.

- Aprendizaje por refuerzo: El aprendizaje por refuerzo es un modelo similar al aprendizaje supervisado, pero el algoritmo no se entrena con datos de ejemplo. Este modelo aprende sobre la marcha mediante el método de ensayo y error. Una secuencia de resultados exitosos se reforzará para desarrollar la mejor recomendación para un determinado problema. Este tipo de aprendizaje es menos común en problemas de clasificación de documentos.

2.3. *Deep Learning* y Redes Neuronales en Clasificación de Documentos

El aprendizaje profundo *("Deep Learnin*g" o DL) es una técnica dentro del aprendizaje automático que se basa en modelos con múltiples capas capaces de aprender representaciones jerárquicas de los datos. A diferencia de los enfoques tradicionales, que requerían de procesos manuales para identificar patrones importantes, el *Deep Learning* permite descubrir automáticamente estas representaciones a partir de datos sin procesar, lo cual ha supuesto una

[1] *Overfitting*: El sobreajuste ocurre cuando un modelo aprende no solo los patrones, sino también el ruido en los datos de entrenamiento, lo que lleva a un rendimiento deficiente en datos nuevos.

[2] *Underfitting*: El subajuste ocurre cuando un modelo es demasiado simple para captar los patrones subyacentes en los datos de entrenamiento, lo que provoca un rendimiento deficiente tanto en los datos de entrenamiento como en los datos nuevos.

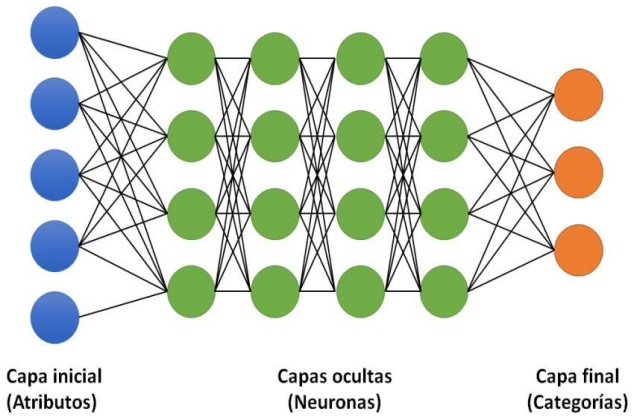
gran mejora en tareas como el reconocimiento de voz o de objetos visuales (LeCun, Bengio & Hinton, 2015, p. 436).

Mientras que los modelos de aprendizaje supervisado requieren datos de entrada estructurados y etiquetados para obtener resultados precisos, los modelos DL utilizan técnicas propias del aprendizaje no supervisado. Con el aprendizaje no supervisado, los modelos pueden extraer las características, los rasgos y las relaciones que necesitan para obtener resultados precisos a partir de datos brutos no estructurados. Además, estos modelos se retroalimentan a sí mismos para aumentar la precisión (IBM, 2024b).

Las redes neuronales - un subconjunto dentro del aprendizaje automático - son la base del aprendizaje profundo. Están formadas por varias capas de nodos, que incluyen una capa de entrada, una o más capas ocultas y una capa de salida. Cada nodo está conectado con otros y tiene un peso y un umbral asignados. Cuando la salida de un nodo supera el umbral establecido, este se activa y transmite la información a la siguiente capa. Si no alcanza ese umbral, la información no se envía a la siguiente capa de la red.

Una red neuronal es un sistema de capas formadas por neuronas y conectadas entre sí, que trata de imitar el funcionamiento del cerebro humano. El proceso comienza con los datos entrando por la capa de entrada y viajan hacia adelante, pasando por las denominadas capas ocultas. En cada capa, las neuronas multiplican los datos por "pesos" que definen la importancia de cada conexión (a mayor peso mayor importancia), suman los resultados y usan una función de activación para decidir si se activan o no. Esto se llama *forward pass*. Este proceso se repite hasta llegar a la capa de salida (capa final) en la que, si la respuesta final es incorrecta, el *backpropagation* entra en acción. Este proceso calcula cuánto contribuyó cada peso al error y ajusta esos pesos retrocediendo capa por capa, como si la red repasara sus pasos para corregir sus fallos. Este ciclo (*forward pass* → cálculo de error → *backpropagation*) se repite ajustando los pesos poco a poco hasta que la red mejora sus predicciones, aprendiendo de forma automática.

Figura 2.1: *Estructura de una red neuronal*



Fuente: Hilmer.vip (2023)

Las arquitecturas de DL más utilizadas incluyen:

- Redes neuronales convolucionales (CNN): Son un tipo de red neuronal especialmente efectiva para reconocimiento de imágenes y procesamiento de datos visuales. Sin embargo, las CNN no pueden son la opción recomendada para procesar secuencias de datos como textos o vídeos, ya que dichos datos están correlacionados y deben estudiarse como bloque. En este sentido, las CNN sufren de "amnesia", es decir, para producir una salida solo tiene en cuenta los datos de la entrada actual y no el de entradas pasadas o futuras.

- Redes neuronales recurrentes (RNN) y Redes Long-Short Term Memory (LSTM): Las RNN son un tipo de red neuronal diseñadas para procesar datos secuenciales como textos, series temporales, secuencias de audio, etc. Tienen cierto grado de memoria. Una RNN tiene dos entradas y dos salidas. Los datos se introducen en la red de forma secuencial y cada neurona no solo tiene en cuenta la entrada actual, sino también un "estado oculto" (*hidden state*) que contiene la información de las entradas anteriores y que le permite preservar y compartir la información entre distintos instantes de tiempo.

El problema de las RNN es que tienen una memoria de corto alcance, ya que cada salida sucesiva depende de la anidación de la función de activación de los estados anteriores. Aquí es donde aparecen las LSTM, que sí implementan mecanismos para "retener" información a largo plazo, manteniendo también la memoria a corto plazo. El truco reside en añadir la denominada "Celda de Estado" que actúa como una memoria a largo plazo que permite aprender, actualizar y olvidar datos a través de compuertas. Cada compuerta tiene una red neuronal, una función sigmoidal y una función multiplicadora.

2.4. Procesamiento de Lenguaje Natural (NLP)

El Procesamiento de Lenguaje Natural (NLP) es una subdisciplina dentro de la Inteligencia Artificial cuyo objetivo es dotar a las máquinas de la capacidad para analizar, comprender y generar lenguaje humano. Esta disciplina se apoya en modelos computacionales capaces de operar sobre grandes volúmenes de texto con el fin de extraer significado, manipular estructuras lingüísticas y llevar a cabo tareas automatizadas relacionadas con la interpretación del lenguaje (Jurafsky & Martin, 2024, p. 3).

Una de las aplicaciones principales del NLP es la clasificación de textos, que consiste en asignar una etiqueta o categoría predefinida a un documento en función de su contenido. Jurafsky y Martin (2024, pp. 56-57) destacan su utilidad en contextos como el análisis de sentimiento, la detección de spam o la categorización temática en bibliotecas digitales y medios sociales. Esta tarea suele abordarse mediante técnicas de aprendizaje automático supervisado, las cuales requieren un conjunto de datos previamente etiquetados y un modelo que pueda

aprender a identificar patrones relevantes en el lenguaje. Este proceso implica distintos pasos a seguir:

- Preprocesamiento del texto: limpieza, tokenización y eliminación de palabras vacías (*stopwords*).
- Extracción de características: con métodos como TF-IDF, *Bag of Words* (BoW) y *Word Embeddings* para convertir el texto en vectores numéricos.
- Algoritmos de clasificación: modelos como K-*means*, *clustering*, Naïve Bayes, SVM y redes neuronales profundas (por ejemplo, BERT) se entrenan para identificar patrones y realizar predicciones.

2.4.1. Técnicas Preliminares de Procesamiento de Texto

2.4.1.1. Tokenización

La tokenización es el proceso de segmentar un texto en unidades más pequeñas, denominadas tokens. Estos pueden ser palabras, caracteres o morfemas, dependiendo del método utilizado (Jurafsky y Martin, 2025, p.18).

Existen diversas técnicas de tokenización, entre las que destacan:

- Tokenización basada en reglas (*top-down*): Utiliza reglas gramaticales y patrones predefinidos para segmentar el texto. Por ejemplo, el estándar "*Penn Treebank*" separa las contracciones (e.g., "*doesn't*" se convierte en "*does*" y "n't") y mantiene los signos de puntuación.

- Byte-Pair Encoding (BPE): Enfoque "*bottom-up*" que combina pares de caracteres o subpalabras más frecuentes en el corpus para formar tokens más grandes. Este método es especialmente útil para manejar palabras desconocidas o raras.

- WordPiece: Utilizado en modelos como BERT, elige las combinaciones que maximizan la probabilidad del modelo de lenguaje.

- Unigram y SentencePiece: Crean una segmentación óptima basada en frecuencias probabilísticas de los subcomponentes del texto.

2.4.1.2. Lematización y *Stemming*

La lematización y el *stemming* son técnicas que permiten la normalización de palabras.

- Lematización: El proceso de reducir las palabras a su forma base o lema. Este utiliza información morfológica y gramatical para identificar la forma canónica de una palabra. Un ejemplo práctico sería la palabra "corriendo" que se lematiza a "correr". La lematización es crucial en tareas de PLN, ya que ayuda a reducir la dimensionalidad del texto sin perder el significado semántico.

- *Stemming*: Método más simple que recorta los sufijos de las palabras sin considerar su estructura gramatical. Utilizando el mismo ejemplo “corriendo” que pasa a ser “corr” (lexema).

2.4.1.3. Eliminación de *Stopwords*

Las *stopwords* son palabras con poca relevancia semántica ("el", "de", "y", etc.), que es conveniente eliminar para reducir la dimensionalidad del texto y mejorar la eficiencia de los modelos.

2.4.2. Métodos de Representación de Texto

Para la clasificación de texto, es fundamental representar de manera efectiva el significado del contenido. Algunas técnicas de modelado que se han implementado en algoritmos de IA son:

2.4.2.1. *Term Frequency - Inverse Document Frequency* (TF-IDF)

El método TF-IDF es una técnica ampliamente utilizada en NLP cuyo objetivo es determinar la importancia de una palabra dentro de un texto en comparación con un corpus de documentos más amplio. Como establece Revuelta San Emeterio (2023, p. 6), esta técnica se basa en dos conceptos clave: la frecuencia de término (TF) y la frecuencia inversa de documento (IDF).

TF mide cuántas veces aparece una palabra dentro de un documento específico. Se calcula dividiendo el número de veces que aparece una palabra entre el total de palabras del documento. Por ejemplo, si una palabra aparece 20 veces en un texto de 1000 palabras, su TF sería 0.02. Sin embargo, esto por sí solo no es suficiente para evaluar su relevancia, ya que palabras comunes como "el" o "de" aparecen con frecuencia en la mayoría de los textos sin aportar información significativa. Para solucionar esto, entra en juego IDF que identifica en

cuántos documentos aparece una palabra dentro del corpus, midiendo así su singularidad (que tan común o rara es la palabra analizada). Su cálculo es igual al logaritmo del cociente entre el número de documentos en el corpus y el número de documentos que contienen la palabra. Cuantas más veces aparezca la palabra en diferentes documentos, menor será su valor IDF, lo que significa que menos representativa es en el documento específico. Por ejemplo, si consideramos un corpus de 500 documentos y la palabra a estudiar aparece en 25 de ellos, su valor IDF es 1.3 (log (20)).

Al combinar estos dos valores mediante su producto (0.02 × 1.3), TF-IDF permite destacar las palabras más representativas de un texto a través de un valor numérico, en este ejemplo 0.026, que representa la importancia relativa de la palabra en el documento.

2.4.2.2. *Bag of Words* (BoW)

El modelo *Bag of Words* representa documentos como vectores en un espacio de alta dimensión, donde cada dimensión corresponde a una palabra única en el corpus. En este modelo, se ignora la gramática y el orden de las palabras, pero se captura su frecuencia. Para la clasificación de documentos, cada texto se convierte en un vector basado en la frecuencia de las palabras o mediante pesos ajustados como TF-IDF, que permiten dar mayor relevancia a términos informativos y reducir el impacto de palabras demasiado comunes. La clasificación se realiza comparando estos vectores mediante medidas como la distancia euclidiana o la similitud del coseno. Además, BoW permite modelar temas dentro de los textos: cada documento se asocia con una distribución de palabras, y ciertos términos tienen mayor probabilidad de aparecer en un tema específico. Esto ayuda a predecir la categoría de un documento basándose en sus palabras más representativas (Aggarwal, 2022).

2.4.2.3. *Word Embeddings*

Los *embeddings* son una forma más avanzada de representar palabras en comparación con los métodos tradicionales como *Bag of Words* o TF-IDF. En lugar de usar vectores dispersos y muy largos, los *embeddings* utilizan vectores densos y de menor dimensión (por ejemplo, entre 50 y 1000 dimensiones en lugar de una dimensión por cada palabra del vocabulario) (Jurafsky y Martin, 2025, p. 117).

Lo que hace a los *word embeddings* especial es que capturan relaciones semánticas entre palabras. Por ejemplo, en una representación dispersa, "coche" y "automóvil" serían completamente distintos, pero en un espacio de *embeddings*, sus vectores estarían más cerca, reflejando su similitud. Esto ayuda a modelos de NLP a entender sinónimos y contextos de manera más efectiva. Uno de los métodos más conocidos es “word2vec”, que se basa en la idea de que el significado de una palabra está determinado por su contexto.

2.4.2.4. *Latent Semantic Analysis* (LSA)

El Análisis Semántico Latente (LSA) es una técnica matemática que busca descubrir relaciones ocultas entre palabras y documentos dentro de grandes colecciones de texto. Este enfoque se basa en la Descomposición en Valores Singulares (SVD), un método algebraico que permite reducir la dimensionalidad de la matriz término-documento. Mediante esta reducción, se proyectan documentos y términos en un espacio semántico de menor dimensión, capturando así patrones subyacentes que permiten detectar similitudes semánticas, como el uso de distintas palabras para expresar ideas similares (sinonimia) o el hecho de que una misma palabra pueda tener varios significados (polisemia). De este modo, LSA facilita una representación más compacta y significativa del contenido textual, útil para tareas como la recuperación de información y la clasificación (Aggarwal, 2022, pp. 41-42). El proceso de LSA se puede resumir en los siguientes pasos:

- Construcción de la matriz término-documento: Se crea una matriz donde las filas representan palabras y las columnas representan documentos. Los valores en la matriz reflejan la frecuencia con la que una palabra aparece en un documento.
- Aplicación de SVD: Esta técnica descompone la matriz original en tres matrices: una matriz de palabras, una matriz diagonal que representa los conceptos latentes, y una matriz de documentos.
- Reducción de la dimensionalidad: Al conservar solo las dimensiones más importantes, se eliminan el ruido y las ambigüedades lingüísticas, permitiendo identificar relaciones semánticas más precisas.

Una de las principales ventajas de LSA es que puede capturar el significado subyacente del texto, incluso cuando las palabras exactas no coinciden. Por ejemplo, LSA puede reconocer que "automóvil" y "coche" están estrechamente relacionados, aunque aparezcan en diferentes contextos.

2.4.2.5. *Latent Dirichlet Allocation* (LDA)

El modelo *Latent Dirichlet Allocation* (LDA) es una técnica probabilística generativa utilizada para extraer temas latentes en grandes colecciones de texto. A diferencia de otros enfoques anteriores, como el Análisis Semántico Latente (LSA), LDA introduce una distribución previa de *Dirichlet*, lo que le permite representar de forma más precisa cómo se combinan los temas dentro de cada documento (Aggarwal, 2022, p. 55). Este modelo asume que cada texto está formado por una mezcla de temas, y que cada tema está compuesto por un conjunto característico de palabras. El proceso generativo que plantea LDA se puede dividir en tres pasos fundamentales:

- En primer lugar, se estima cuántas palabras tendrá un documento, siguiendo una distribución de *Poisson*.
- Después, se asigna a cada documento una distribución de temas, extraída de una distribución de *Dirichlet*.
- Finalmente, para cada palabra del documento, se elige un tema según esa distribución, y la palabra se genera a partir de la distribución de palabras propia de ese tema (Aggarwal, 2022, p. 55).

Una de las fortalezas principales de este modelo es que la distribución de *Dirichlet* funciona como una forma de regularización, lo que contribuye a reducir el riesgo de *overfitting* durante el entrenamiento. Además, LDA permite aplicar el modelo a documentos nuevos sin necesidad de reentrenarlo completamente, lo cual es especialmente útil en contextos donde se incorporan textos de forma continua o en tiempo real (Aggarwal, 2022, p. 57).

2.4.3. Modelos de lenguaje basados en Transformer

Un *transformer* es un modelo de *Deep Learning* que ha transformado por completo el campo del Procesamiento del Lenguaje Natural (NLP). Se trata de una red neuronal diseñada para entender y procesar secuencias de texto, imágenes o cualquier tipo de datos secuenciales, pero de una forma mucho más eficiente que los modelos anteriores como las RNN. Su arquitectura fue introducida por Vaswani et al. en el artículo "*Attention is All You Need*" (2017), y posteriormente ha sido ampliamente estudiada y explicada por expertos como Charu C. Aggarwal en su obra "*Machine Learning for Text*" (2022, p. 375), donde afirma que el modelo *Transformer* ha reemplazado en gran medida a las redes neuronales recurrentes en el procesamiento de lenguaje natural, principalmente porque permite realizar cálculos en paralelo y evita las limitaciones de procesamiento secuencial, lo que le otorga una ventaja computacional significativa.

El *transformer* se basa en un mecanismo llamado *Self-Attention*, que permite al modelo identificar qué palabras o elementos son más relevantes dentro de una secuencia, sin necesidad de procesarlas en orden, como lo hacían los modelos tradicionales como las RNN.

Por ejemplo, si la frase es "El banco está cerca del río", el *transformer* puede analizar todo el contexto al mismo tiempo y comprender que la palabra "banco" hace referencia a un asiento y no a una institución financiera, gracias a la relación semántica con "río".

Modelos como GPT, BERT y T5 están basados en esta arquitectura, permitiendo tareas como la generación de texto, la traducción automática y la respuesta a preguntas.

2.4.3.1. GPT (Generative Pretrained Transformer)

El modelo GPT pertenece a una familia de modelos de lenguaje basados en la arquitectura *transformer*, pero utiliza exclusivamente la parte del decodificador ("*decoder-only*") del *transformer* original. Esto implica que GPT procesa el texto de manera unidireccional, es decir, predice cada palabra basándose únicamente en el contexto de las palabras anteriores, lo que se conoce como modelado de lenguaje causal ("*Causal Language Modeling*") (Aggarwal, 2022, p. 381-382).

2.4.3.2. BERT (Bidirectional Encoder Representations from Transformers)

En contraste con GPT, BERT utiliza únicamente la parte del codificador ("*encoder-only*") de la arquitectura *transformer*, lo que le permite procesar el texto de manera bidireccional, es decir, analizando tanto el contexto a la izquierda como a la derecha de una palabra. Aggarwal (2022, p. 382-383) explica que este enfoque permite a BERT capturar relaciones semánticas más profundas entre las palabras, lo que lo convierte en una herramienta poderosa para tareas como el análisis de sentimientos, la respuesta a preguntas y la clasificación de texto. BERT se entrena a través de dos tareas clave:

- Masked Language Model (MLM): Se ocultan aleatoriamente el 15% de las palabras en una oración y el modelo debe predecirlas basándose en el contexto circundante.
- Next Sentence Prediction (NSP): El modelo recibe pares de oraciones y debe predecir si la segunda oración sigue lógicamente a la primera.

BERT utiliza la técnica de tokenización WordPiece, optimizada para maximizar la probabilidad de los datos de entrenamiento. Además, a diferencia de GPT, BERT requiere un ajuste fino ("*fine-tuning*") para cada tarea específica, lo que implica añadir una capa adicional a la red para tareas como la clasificación de texto.

2.4.3.3. T5 (Text-To-Text Transfer Transformer)

El modelo T5 adopta el método "texto a texto", en el que todas las tareas de procesamiento de lenguaje natural (NLP) se reformulan como tareas de conversión de texto. Según Aggarwal (2022, p.384), esta arquitectura utiliza tanto el codificador como el decodificador del *transformer* original, lo que le permite abordar una amplia variedad de tareas simplemente cambiando el prefijo del texto de entrada.

T5 se entrena con una técnica llamada "*Masked Span Prediction*", similar a BERT, pero en lugar de predecir palabras individuales, T5 reemplaza fragmentos completos de texto con tokens especiales (por ejemplo, "X" y "Y") y luego predice el texto que falta.

2.5. Algoritmos de Clasificación de documentos

2.5.1. Árboles de decisión y *Random Forest*

Los árboles de decisión son un método de aprendizaje supervisado no paramétrico utilizado en problemas tanto de clasificación como de regresión. Su estructura en forma de árbol permite dividir los datos en subconjuntos más pequeños en función de ciertos atributos hasta alcanzar una categoría final.

Un árbol de decisión se compone de dos elementos fundamentales: los nodos de decisión, que representan los puntos donde los datos se dividen en función de un criterio específico, y las hojas, que contienen las categorías finales asignadas a los datos analizados. Cuando un documento es sometido al modelo, el árbol lo clasifica recorriendo sus ramas desde la raíz hasta una hoja, aplicando pruebas sucesivas en cada nodo. Cada nodo interno está etiquetado con una característica específica del documento y determina a qué subárbol debe dirigirse. Finalmente, el documento se clasifica según la categoría de la hoja en la que termina (Dhingra et al., 2022, pp. 1820-1821).

Li et al. (2022, p. 230) explican el proceso de construcción de un árbol de decisión, que se divide en tres etapas principales: selección de características, generación del árbol y *pruning*.

- En primer lugar, la selección de características consiste en elegir los atributos más relevantes de los datos, los cuales servirán como base para la toma de decisiones en los nodos.

- Después de seleccionar las características más importantes, se procede a la generación del árbol, donde se construye la estructura jerárquica a partir de las relaciones entre los datos. El nodo raíz representa la característica con mayor capacidad discriminativa y, a partir de ahí, se crean ramas que dividen los datos en función de sus valores. Cada división se realiza con base en la probabilidad de que una característica específica pertenezca a una determinada categoría. Si no hay más subdivisiones posibles o si un nodo no tiene variabilidad significativa en los datos, se alcanza una hoja y la clasificación se detiene.

- Uno de los problemas comunes en los árboles de decisión es el *overfitting*. Para evitar este problema, se aplica una técnica llamada *pruning*, que reduce el tamaño del árbol eliminando ramas irrelevantes. Esta técnica puede realizarse ajustando manualmente la estructura del árbol, controlando el número de muestras por nodo o estableciendo un peso mínimo para las hojas.

"*Random Forest* es un algoritmo de aprendizaje automático de uso común, registrado por Leo Breiman y Adele Cutler, que combina el resultado de múltiples árboles de decisión para llegar a un resultado único" (IBM, 2024c).

2.5.2. K-Nearest Neighbors (k-NN)

Los clasificadores K-Nearest Neighbors son algoritmos que se basan en la memoria, también conocidos como "aprendices perezosos". Su funcionamiento es el siguiente: cuando se presenta una nueva instancia de prueba, el algoritmo busca los "k" vecinos más cercanos en los datos de entrenamiento y asigna a la nueva instancia la etiqueta que predomine entre esos vecinos (k representa el número de vecinos que queremos utilizar en el modelo). A diferencia de otros algoritmos, los clasificadores k-NN no requieren un proceso de entrenamiento previo, sino que "memorizan" los datos y realizan todo el trabajo de clasificación cuando se realiza la predicción. Es por ello, que los clasificadores k-NN tienden a ser superados por otros métodos de aprendizaje llamados "ansiosos", que construyen un modelo durante la fase de entrenamiento. A pesar de esto, los fundamentos teóricos de los k-NN son importantes, ya que son la base de algunos de los clasificadores más efectivos en la actualidad, como *Random Forests* y las Máquinas de Vectores de Soporte (SVM) (Aggarwal, 2022, p.12).

2.5.3. Naïve Bayes

El clasificador Naïve Bayes es un modelo de aprendizaje automático basado en el Teorema de Bayes, ampliamente utilizado en tareas de clasificación de textos. Su funcionamiento se basa en calcular la probabilidad de que un documento pertenezca a una categoría determinada según las palabras que contiene (Dhingra et al., 2022, p. 1821). Lo que lo hace particular es que asume que cada palabra en un texto es independiente de las demás, es decir, que la presencia de una palabra no influye en la aparición de otra. Aunque esta suposición no es del todo realista, ya que las palabras suelen estar relacionadas, esta simplificación permite que el modelo sea eficiente, rápido y fácil de implementar.

A pesar de la asunción de independencia entre palabras, Naïve Bayes sigue teniendo un funcionamiento óptimo. Esto se debe a que, más que comprender el significado de un texto, el modelo se enfoca en patrones generales de frecuencia. Por ejemplo, si un documento contiene términos como "gol", "set", "penalti" y "canasta", es muy probable que pertenezca a la categoría de "Deportes". El modelo no analiza el significado de cada palabra ni sus relaciones exactas, pero puede reconocer qué términos suelen aparecer juntos en distintos tipos de documentos (Dhingra et al., 2022, p. 1821).

2.5.4. Máquinas de Vectores de Soporte (SVM)

Las Máquinas de Vectores de Soporte (SVM) son algoritmos de clasificación que han demostrado ser altamente efectivos en espacios de alta dimensión. Según Baharudin et al. (2010, pp. 12-13), las SVM se basan en el principio de Minimización del Riesgo Estructural,

cuyo objetivo es reducir el error real al elegir la hipótesis con la menor complejidad posible. El núcleo del algoritmo radica en encontrar el hiperplano óptimo que mejor separa dos clases, maximizando la distancia entre los puntos más cercanos de cada clase y dicho hiperplano. Estos puntos cercanos se denominan vectores de soporte y son los que determinan la posición del hiperplano, haciendo que el modelo descarte la mayoría de las características irrelevantes.

Aggarwal (2022, pp. 178-181) destaca que, desde una perspectiva geométrica, el algoritmo crea dos hiperplanos paralelos a cada lado de la frontera de decisión, con el fin de maximizar el margen entre ellos. Esto contribuye a reducir el error de generalización, ya que un mayor margen suele implicar una mejor capacidad de clasificación para nuevos datos. Además, en términos de optimización, las SVM emplean la función *hinge loss*, la cual penaliza aquellos puntos que se encuentran en el lado incorrecto de la frontera de decisión o que están demasiado cerca de ella, ayudando a prevenir el *overfitting*.

2.5.5. Otras técnicas de *clustering*

2.5.5.1. *Clustering* jerárquico

El *clustering* jerárquico organiza los datos en una estructura de tipo árbol (dendograma), permitiendo observar relaciones en distintos niveles de agrupación. Existen dos enfoques principales: el aglomerativo, que comienza tratando cada objeto como un clúster independiente y los va fusionando progresivamente; y el divisivo, que parte de un único clúster con todos los objetos y lo va dividiendo hasta obtener grupos más pequeños. Uno de los beneficios clave de esta técnica es su utilidad en tareas de visualización o segmentación progresiva, aunque presenta limitaciones como la imposibilidad de deshacer fusiones o divisiones una vez realizadas (Han et al., 2012, pp. 457–459).

2.5.5.2. DBSCAN (*Density-Based Clustering Based on Connected Regions with High Density*)

El algoritmo DBSCAN es una técnica de agrupamiento basada en densidad que permite descubrir clústeres de forma arbitraria dentro de un conjunto de datos, incluso en presencia de ruido. A diferencia de métodos como k-*means*, DBSCAN no requiere que se especifique el número de clústeres. En su lugar, utiliza dos parámetros: ε (epsilon), que determina el radio de vecindad de un punto, y MinPts, que representa el número mínimo de puntos necesarios para que una región se considere densa. Un punto que tiene al menos MinPts en su vecindad ε es un punto central, y puede expandirse para formar un clúster. Los puntos no densamente conectados y que no pertenecen a ningún clúster se tratan como ruido u *outliers* (Han et al., 2012, pp. 471-472).

2.5.5.3. GMM (*Gaussian Mixture Model*)

Una técnica muy empleada para realizar *clustering* desde una perspectiva probabilística son los Modelos de Mezcla Gaussianos (GMM). Este enfoque asume que los datos se generan a partir de una combinación de varias distribuciones gaussianas, cada una de las cuales representa un subgrupo o clúster dentro del conjunto general. En lugar de asignar cada observación a un único grupo, como ocurre en métodos como k-*means*, GMM permite establecer una pertenencia probabilística a cada componente del modelo. Esto significa que una observación puede tener, por ejemplo, un 70% de pertenencia al clúster A y un 30% al B, lo que otorga una mayor flexibilidad al modelado de datos complejos y superpuestos. Para ajustar los parámetros del modelo (medias, covarianzas y pesos de mezcla), se utiliza comúnmente el algoritmo de *Expectation-Maximization* (EM), el cual alterna entre calcular las probabilidades de pertenencia de cada punto a los clústeres (E) y actualizar los parámetros del modelo en base a esas asignaciones (M) (Bishop, 2006, p. 430).

2.6. Evaluación de Modelos en Clasificación de Documentos

La evaluación de modelos de clasificación de documentos textuales se basa en la comparación entre las categorías asignadas por el modelo y las categorías reales establecidas en un conjunto de prueba. Según Sebastiani (2002, p. 37), este procedimiento permite medir empíricamente la efectividad de un clasificador, utilizando métricas derivadas tanto del aprendizaje automático como de la recuperación de información. Entre las métricas más habituales se encuentran:

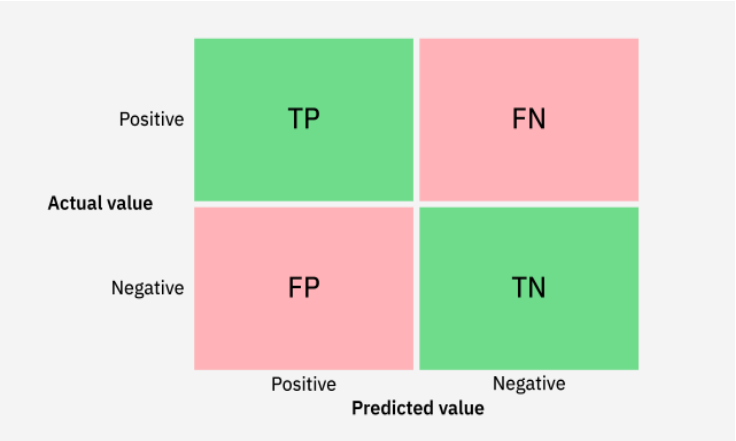
2.6.1. Matriz de confusión

Una métrica basada en una tabla que muestra la distribución de predicciones correctas e incorrectas en cada categoría. Esta matriz permite analizar en detalle los errores cometidos por el clasificador y ajustar su rendimiento, ya que muestra dónde el modelo confunde una clase con otra. Aporta más información que la precisión, exhaustividad y exactitud. Según Dhingra et al. (2022, p. 1820) las cuatro partes que componen una matriz de confusión se definen en de la siguiente manera para problemas de clasificación de documentos textuales:

- Verdaderos Positivos (TP): Un documento identificado correctamente como perteneciente a una categoría.
- Verdaderos Negativos (TN): Documentos que, de manera correcta, no se etiquetan como pertenecientes a una categoría específica.
- Falsos Positivos (FP): Un documento que por error se cree que es relevante para la categoría.

- Falsos Negativos (FN): Un documento que no se ha etiquetado como relacionado con una categoría, pero debería etiquetarse como tal.

Figura 2.2: *Matriz de confusión*



Fuente: IBM Developer (2024)

Una vez ya se conoce y entiende el significado de TP, TN, FP y FN, pasamos a conocer las métricas de evaluación más comunes y efectivas que utilizan los valores de la matriz de confusión.

2.6.2. Exactitud ("*Accuracy*")

La exactitud mide el porcentaje de documentos que han sido correctamente clasificados, considerando tanto los casos positivos como los negativos, es decir, mide con qué frecuencia el modelo acierta. Una exactitud alta significa que el modelo funciona bien.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{2.1}$$

A partir de esta fórmula, también podemos obtener la tasa de error que, al contrario que el *accuracy,* mide el porcentaje de predicciones incorrectas de un modelo con respecto al total de predicciones. Su valor se obtiene restando el *accuracy* a la unidad, es decir:

$$Tasa\ de\ error = 1 - Accuracy \tag{2.2}$$

Sin embargo, esta métrica no es adecuada cuando las categorías están desbalanceadas. Esto se debe al llamado "Problema de la mayoría de clase", que ocurre cuando una de las clases tiene muchos más ejemplos que las demás. En estos casos, el modelo puede simplemente clasificar la mayoría de los documentos en la clase más común para obtener una alta exactitud, sin aprender realmente a identificar la clase minoritaria. Como indican Dhingra et al. (2022, p. 1820), un ejemplo común en problemas de clasificación es que la clase negativa suele estar sobrerrepresentada, mientras que la clase positiva (a menudo la categoría de mayor interés) aparece con menos frecuencia.

Por esta razón, cuando los datos están desbalanceados, el *accuracy* no refleja bien el rendimiento del modelo. En su lugar, se recomiendan métricas como la *precision* y el *recall*, que permiten evaluar mejor qué tan bien se detecta la clase positiva.

2.6.3. Precisión ("*Precision*")

La precisión mide cuántos de los documentos clasificados como pertenecientes a una categoría realmente pertenecen a ella. Es decir, indica el porcentaje de aciertos dentro de los documentos que el modelo ha identificado como positivos en las distintas categorías.

$$\text{Pr}\,e\,cision = \frac{TP}{TP + FP} \tag{2.3}$$

2.6.4. Exhaustividad ("*Recall*")

La exhaustividad o "*recall*" mide cuántos de los documentos que realmente pertenecen a una categoría han sido correctamente identificados por el clasificador. Una alta exhaustividad significa que el modelo no deja fuera muchos documentos relevantes, aunque esto puede venir a costa de una menor precisión.

$$Recall = \frac{TP}{TP + FN} \tag{2.4}$$

2.6.5. *F1-score*

La medida F1 es una métrica que combina precisión y exhaustividad para aportar un equilibrio entre ambas, dado que estas, en ciertas ocasiones, pueden contradecirse (aumentar una puede reducir la otra). Esta métrica es especialmente útil cuando se requiere un balance entre ambos factores, que suele ser el caso en tareas de clasificación de texto.

$$F1score = 2 \times \frac{\mathrm{Pr}\, ecision \times Recall}{\mathrm{Pr}\, ecision + Recall} \tag{2.5}$$

CAPÍTULO 3: REVISIÓN DE LA LITERATURA

Este apartado centra la atención en el análisis de la literatura, con el fin de identificar casos de éxito, experiencias y avances relevantes en el área de la clasificación automática de documentos. Todos los documentos han sido extraídos de la biblioteca de la Universidad Carlos III de Madrid, específicamente de la base de datos *LISTA: Library, Information Science & Technology Abstracts*.

3.1. Relación entre clasificadores textuales y técnicas de clasificación basados en indicadores bibliométricos.

Explorando en profundidad este tema, encontramos el artículo "Classifying documents with link-based bibliometric measures". Couto et al. (2010) abordan el problema de la clasificación automática de documentos como un proceso clave en la organización de bibliotecas digitales, directorios en línea y motores de búsqueda. Los autores exploran el uso de información de enlaces y métricas bibliométricas para mejorar los algoritmos de clasificación. A través de experimentos en diferentes colecciones documentales, el estudio demuestra que el buen uso de hipervínculos y citas puede mejorar la precisión de clasificadores basados en el algoritmo k-NN en comparación con enfoques basados solo en contenido textual.

El trabajo se centra en la premisa de que la estructura de enlaces en los documentos puede proporcionar información relevante para la clasificación automática. Los autores analizan tres métricas bibliométricas:

- Co-citación: Evalúa la similitud entre documentos en función de cuántas veces son citados conjuntamente por otros documentos. Esta métrica resulta útil para identificar documentos que, aunque no tengan referencias directas entre sí, son considerados relacionados por la comunidad académica.

- Acoplamiento bibliográfico: Determina la similitud en base a la cantidad de referencias compartidas por dos documentos. A diferencia de la co-citación, esta métrica se basa en los documentos referenciados por los propios autores, lo que puede indicar un enfoque similar en sus investigaciones.

- Medida de Amsler: Combina las métricas de Co-citación y Acoplamiento bibliográfico en una sola.

Los experimentos fueron realizados en tres conjuntos de datos: una biblioteca digital de artículos científicos, un directorio web y una enciclopedia en línea. En la colección de la ACM Digital Library (biblioteca digital), la medida de Amsler logró un rendimiento superior al de la clasificación basada solo en texto, con un incremento del 9.7% en la métrica de micro-average F1. Esto se debe a que los documentos científicos suelen contener citas bien estructuradas, lo que permite que la información de enlaces contribuya a una mejor agrupación.

En el caso del directorio web “Cade12”, la clasificación basada en texto demostró ser menos eficaz debido a la heterogeneidad y ruido en el contenido de las páginas. Sin embargo, el uso de la co-citación y la medida de Amsler mejoró significativamente el rendimiento del clasificador, alcanzando un 69.8% de mejora respecto a los métodos únicamente textuales. Esto sugiere que, en entornos donde la calidad del texto es variable, las estructuras de enlaces pueden proporcionar un medio más confiable para inferir relaciones temáticas.

En la colección de Wikipedia, “Wiki8”, los clasificadores bibliométricos también demostraron su eficacia, aunque en menor medida en comparación con los otros conjuntos de datos debido a la riqueza del contenido textual. La medida de acoplamiento bibliográfico alcanzó un micro-average F1 del 86.95%, lo que representa un incremento del 7.51% respecto al clasificador basado solo en texto. Esto se debe a que, en Wikipedia, los artículos están interconectados mediante referencias y enlaces internos, lo que facilita la aplicación de estas métricas bibliométricas.

Los autores también exploran la combinación de clasificadores basados en texto y en enlaces. Se observa que la combinación guiada por la fiabilidad de los clasificadores mejora el rendimiento global. En particular, en el directorio web, la combinación de ambos métodos permitió obtener un incremento del 13.82% en micro-average F1, demostrando que cuando los datos son ruidosos, una estrategia combinada puede mejorar significativamente la clasificación.

Este artículo es particularmente relevante para el TFG, ya que proporciona una base teórica y empírica sobre el uso de diferentes métodos de clasificación automática de documentos. Además, la incorporación de estrategias de combinación de clasificadores (en este caso fuentes textuales y bibliométricas) sugiere posibles áreas de mejora para futuras investigaciones, incluyendo la optimización de hiperparámetros en modelos de IA y la adaptación de arquitecturas de *Deep Learning* para el tratamiento de enlaces y referencias.

3.2. Casos prácticos de clasificación automática

Para este tema se revisarán dos documentos que muestran casos prácticos reales de la clasificación automática que se produjeron en el Museo Nacional de Literatura Afrikaans y Centro de Investigación (“*Machine learning for document classification in an archive of the National Afrikaans Literary Museum and Research Centre*”) y en la Biblioteca del Congreso Nacional de Chile (“*Machine learning for libraries with Python libraries: practical case in the Library of Congress of Chile*”).

En el primer caso mencionado, los autores Brokensha, Kotzé y Senekal analizan la clasificación automática de documentos en el contexto de archivos digitales, enfocándose en el *National Afrikaans Literary Museum and Research Centre* (NALN) en Sudáfrica. Los autores plantean que la creciente digitalización de documentos y la falta de personal en los archivos presentan un desafío significativo para la organización y preservación de la

información. En la práctica, se desarrollaron modelos de clasificación automática y sus resultados se compararon con la clasificación manual realizada por expertos.

El estudio se centra en el NALN, un archivo especializado en literatura afrikáans que maneja una gran cantidad de documentos, incluidos libros, artículos académicos, tesis, etc. Debido a restricciones presupuestarias, el personal a cargo de la clasificación de documentos se ha reducido, lo que hace inviable la clasificación manual a gran escala. En este contexto, los autores argumentan que el aprendizaje automático ofrece una solución viable para la generación automática de metadatos y la organización documental.

Para realizar el experimento, los autores recopilaron documentos en afrikáans y los clasificaron en seis categorías establecidas por Galloway y Roux (2019): artículos, reportajes, libros, entrevistas, reseñas y tesis/disertaciones. Estos documentos se utilizaron como conjunto de entrenamiento para modelos de aprendizaje automático con una partición de los datos 80/20, es decir, el 80% para entrenamiento y el 20% para prueba. El procesamiento de texto incluyó la conversión de documentos PDF a texto mediante la biblioteca "pdfplumber", la limpieza de datos con eliminación de caracteres especiales (*stopwords*) y el uso de la vectorización TF-IDF y *Bag-of-Words* para representar los textos numéricamente.

Se entrenaron dos modelos de clasificación: *Support Vector Machines* (SVM) y *Multi-Layer Perceptron* (MLP). Los resultados mostraron que el SVM con vectorización TF-IDF y sin limpieza de datos alcanzó la mejor precisión con un F1-score del 92,9%, superando la clasificación manual en términos de rapidez y consistencia. Se encontró que la limpieza de datos no mejoró significativamente los resultados en el caso de TF-IDF, mientras que en *Bag-of-Words* sí hubo una ligera mejora tras la limpieza de datos.

Otro hallazgo clave fue que el modelo MLP, a pesar de tratarse de una red neuronal con mayor capacidad de aprendizaje, no superó el rendimiento del SVM, lo que indica que, para conjuntos de datos de tamaño moderado, los modelos de aprendizaje clásico pueden ser más eficientes. Además, los autores destacan que el SVM fue más sensible al ruido en los datos, mientras que MLP mantuvo resultados similares en distintos niveles de limpieza del texto.

Los autores también discuten la importancia de la generación automática de metadatos en archivos digitales. Explican que, en el archivo *Freedom of Information Archive* (FOIArchive), se han empleado técnicas similares con éxito para indexar y clasificar documentos de manera automática. En este sentido, se plantea que la aplicación de inteligencia artificial podría aliviar la carga de trabajo del personal de archivo y mejorar la accesibilidad de la información.

Este artículo es altamente relevante para el TFG pues refuerza la idea de que el aprendizaje automático es una herramienta eficaz para la organización documental en entornos de archivo, lo que justifica su aplicación en proyectos similares. Por último, el artículo enfatiza la necesidad de desarrollar herramientas de generación de metadatos automáticas para hacer frente a la creciente cantidad de información digital. Esto coincide con el enfoque del TFG,

que busca explorar cómo la IA puede facilitar la clasificación documental en distintos contextos.

En el caso de la Biblioteca del Congreso Nacional de Chile, Marcelo Lorca González, a través del documento previamente mencionado, explora la aplicación del aprendizaje automático en el análisis de datos bibliográficos en bibliotecas. Específicamente, el estudio se centra en el uso del algoritmo *K-means* para la segmentación de colecciones en la Biblioteca del Congreso Nacional de Chile. Marcelo destaca la importancia de la inteligencia artificial en la optimización de procesos internos de gestión documental y en la mejora de la toma de decisiones basada en datos.

El documento describe en detalle el proceso de implementación. Primero, se seleccionaron datos de una colección de 30.000 títulos pertenecientes a la sede de Valparaíso de la Biblioteca del Congreso. La clasificación de los libros se realizó a partir de los dos primeros dígitos de la Clasificación Decimal Universal (CDU), con el objetivo de identificar las principales áreas temáticas. Posteriormente, se llevó a cabo un proceso de limpieza de datos en Python, eliminando valores nulos y normalizando los datos para tener una información homogénea.

A continuación, se aplicó el método del codo ("*Elbow Method*") para determinar el número óptimo de clústeres. Los resultados indicaron que la segmentación más efectiva se lograba con cuatro agrupaciones principales. La implementación del algoritmo K-means se realizó en Google Colab, utilizando librerías destacadas de Python para ejercicios de clasificación como Pandas para el manejo de datos, Matplotlib para la visualización gráfica y Scikit-learn para la aplicación del modelo de *clustering*.

El documento también presenta resultados visuales que muestran la distribución de las temáticas dentro de la colección. Se observó que el área de derecho (340 CDU) era la más representativa, con más de 8.000 títulos, seguida por economía (330 CDU) y ciencias políticas (320 CDU). Estas categorías formaron clústeres bien definidos, mientras que otras áreas temáticas presentaban mayor dispersión.

Finalmente, los autores destacan que esta metodología podría aplicarse a otros procesos dentro de la biblioteca, como la identificación de tendencias o la predicción de la demanda de determinados títulos. Además, sugieren que futuras investigaciones podrían combinar *K-means* con métodos de aprendizaje supervisado para mejorar la precisión de la clasificación.

Los fundamentos que Marcelo Lorca González expone en el documento guardan una relación directa con el TFG, ya que ambos exploran la aplicación del aprendizaje automático en la clasificación de documentos. En particular, el uso del algoritmo *K-means* en la segmentación de colecciones bibliográficas constituye un enfoque práctico que podría aplicarse en el contexto de investigaciones similares para mejorar la organización y recuperación de información en bases de datos documentales.

3.3. *Clustering* y UX mediante recomendaciones de libros.

Esta sección de la literatura explora "*Exploring Clustering-Based Reinforcement Learning for Personalized Book Recommendation in Digital Library*" Wang et al. (2021), que trata la aplicación del aprendizaje por refuerzo basado en *clustering* como una herramienta que mejora los sistemas de recomendación de libros en bibliotecas digitales. Los autores plantean que, aunque las bibliotecas digitales han facilitado el acceso a grandes volúmenes de información, los usuarios a menudo se enfrentan a dificultades para encontrar recursos relevantes debido a la extensión de los catálogos y a problemas como la escasez de datos y la presencia de ruido. Para abordar estas limitaciones, proponen un sistema de recomendación que integra aprendizaje por refuerzo jerárquico (HRL) con técnicas de *clustering*, con el objetivo de eliminar datos irrelevantes en los historiales de préstamo de los usuarios, y así mejorar la experiencia de usuario (UX).

La investigación gira en torno a dos problemas clave en la recomendación de libros: la presencia de ruido en los datos y la escasez de datos:

- Presencia de ruido en los datos: Se refiere a situaciones en las que los usuarios toman prestados libros que no reflejan realmente sus intereses, por ejemplo, un estudiante de ingeniería toma prestado un libro de historia para un curso obligatorio o para ayudar a un amigo. Este ruido en los datos afecta negativamente la capacidad del sistema de recomendación para identificar patrones de interés reales.

- Escasez de datos: En una biblioteca digital, la cantidad de libros es tan extensa que un usuario solo interactúa con una pequeña fracción de ellos. Esto genera un problema de dispersión en los datos, lo que dificulta la generación de un perfil preciso de interés del usuario.

Para resolver estas problemáticas, los autores proponen un modelo basado en aprendizaje por refuerzo jerárquico (HRL) que opera en dos niveles:

- Nivel alto: Determina si una secuencia de préstamo necesita ser modificada.

- Nivel bajo: Decide qué elementos deben eliminarse dentro de la secuencia de préstamo.

Además, incorporan una estrategia de *clustering* que agrupa libros con características similares antes de introducir los datos al modelo de aprendizaje por refuerzo. Esta técnica permite reducir la dispersión de los datos y mejorar la estabilidad del sistema de recomendación.

El modelo propuesto, denominado *Clustering-based Hierarchical Reinforcement Learning* (CHRL), se compone de tres módulos fundamentales:

- Recomendador básico: Implementado con el modelo NAIS (*Neural Attentive Item Similarity*), el cual emplea un mecanismo de atención para asignar pesos a las interacciones previas de un usuario.

- Revisor de perfiles: Utiliza el aprendizaje por refuerzo jerárquico (HRL) para filtrar los datos "ruidosos".

- Mecanismo de *clustering*: Clasifica los libros en diferentes grupos según sus *embeddings*, generados previamente en la fase de preentrenamiento. El *clustering* se realiza con base en métricas de similitud de contenido y popularidad de los libros.

Los autores realizaron los experimentos sobre dos conjuntos de datos:

- Registros de préstamos de una biblioteca universitaria: 518605 registros de préstamos de 22572 estudiantes, con un total de 124468 libros diferentes.

- Goodbooks-10k: Un conjunto de 898195 registros de 41314 usuarios, con 10000 libros disponibles.

Los resultados mostraron que CHRL superó a varios métodos de referencia: en la biblioteca universitaria, el modelo alcanzó un HR@10 del 92.12%, mientras que otros métodos como HRL-NAIS y Light-GCN obtuvieron 78.34% y 58.90%, respectivamente. En Goodbooks-10k, CHRL logró un NDCG@10 del 43.88%, mejorando significativamente los resultados de los sistemas tradicionales.

Además, los experimentos revelaron que la incorporación de *clustering* redujo la variabilidad de los resultados y estabilizó el rendimiento del sistema. Se demostró que los modelos de aprendizaje por refuerzo sin *clustering* eliminaban datos de más, afectando negativamente la calidad de las recomendaciones.

Este estudio guarda relación con el TFG puesto que, en primer lugar, el modelo CHRL ofrece una metodología que, aunque en el experimento se aplica a la recomendación de libros, también se puede considerar para la clasificación de documentos en bibliotecas digitales, donde los problemas de ruido y escasez de datos están presentes. Además, la combinación de aprendizaje automático con *clustering* sugiere una posible vía de mejora para los sistemas de clasificación de documentos. Por ejemplo, podría emplearse un enfoque similar para agrupar documentos antes de su clasificación.

3.4. *Clustering* y otras técnicas relacionadas (LDA, *embeddings*, etc.).

Este apartado se centra en abordar técnicas de *clustering*, un tipo de modelo muy presente en este TFG, y otras técnicas relacionadas con esta que le sirven de complemento, e incluso la mejoran en muchas ocasiones. Para ello, se discuten tres documentos relevantes:

"*Document Representation Methods for Clustering Bilingual Documents*", "*A Comparison of Term Clusters for Tokenized Words Collected from Controlled Vocabularies, User Keyword Searches, and Online Documents*" y "*Classification, Clustering, and Data Analysis. Recent Advances and Applications*".

El primer documento mencionado en este apartado trata el contexto actual de globalización y digitalización, haciendo énfasis en la gestión de documentos en varios idiomas, que se ha convertido en uno de los principales desafíos para bibliotecas digitales e instituciones académicas. Los autores Shutian Ma, Chengzhi Zhang y Daqing buscan abordar el problema de la clasificación de documentos bilingües, destacando que la clasificación automática es uno de los dos métodos más utilizados en la gestión documental, pero que requiere datos previamente etiquetados, lo que implica un alto costo en tiempo y recursos. Como alternativa, el *clustering* de documentos ofrece una solución más escalable al agrupar textos sin necesidad de entrenamiento previo.

El artículo se centra en la comparación de cuatro métodos distintos de representación documental: *Vector Space Model* (VSM), *Latent Semantic Indexing* (LSI), *Latent Dirichlet Allocation* (LDA) y Doc2Vec. Para evaluar su rendimiento, los autores emplean dos tipos de conjuntos de documentos bilingües: paralelos (documentos en distintos idiomas con traducción exacta) y comparables (documentos en distintos idiomas sobre el mismo tema). Para ello, los experimentos se realizaron sobre dos conjuntos de datos reales:

- Corpus paralelo: Conjunto de 4,904 pares de blogs en inglés y chino extraídos de la plataforma Engadget, distribuidos en 20 categorías.
- Corpus comparable: Documentos sobre tecnología móvil y energía eólica en inglés y chino, obtenidos del proyecto TTC.

Los resultados fueron evaluados mediante dos métricas:

- V-*measure*: Para corpus paralelos, basada en la comparación entre etiquetas reales y clústeres generados.
- *Silhouette Coefficient*: Para corpus comparables, evaluando la cohesión interna de los clústeres sin necesidad de etiquetas previas.

Los autores explican que la calidad del *clustering* depende en gran medida de cómo se representen los documentos antes de ser procesados por los algoritmos de agrupamiento. Cada método de representación tiene sus ventajas y desventajas:

- *Vector Space Model* (VSM): Representa documentos como vectores en un espacio multidimensional basado en la frecuencia de palabras. Se calcula utilizando TF-IDF para asignar pesos a los términos, pero al tratar cada término de manera independiente, no captura relaciones semánticas como sinonimia o polisemia. En los experimentos

realizados, VSM mostró un rendimiento inferior al de otros métodos en todas las tareas de *clustering*.

- *Latent Semantic Indexing* (LSI): Utiliza la técnica de descomposición en valores singulares (SVD) para reducir la dimensionalidad y capturar relaciones semánticas entre términos. Sin embargo, su rendimiento varió dependiendo del corpus utilizado. En los corpus paralelos, LSI mostró estabilidad en dimensiones superiores a 80, pero en corpus comparables su desempeño variaba considerablemente.

- *Latent Dirichlet Allocation* (LDA): Es un modelo probabilístico que representa documentos como distribuciones de temas latentes. Se observó que LDA obtuvo mejores resultados en la tarea de *clustering* de corpus comparables de menor tamaño, donde la identificación de temas subyacentes fue más efectiva (alcanzando valores de *Silhouette Coefficient* de 0.88). En corpus paralelos grandes, su desempeño fue menos competitivo en comparación con Doc2Vec.

- Doc2Vec: Utiliza redes neuronales para generar representaciones vectoriales continuas de los documentos. En los experimentos, Doc2Vec fue el método más eficiente para corpus paralelos grandes, alcanzando los mejores valores de V-*measure* (hasta 0.46) cuando se emplearon dimensiones superiores a 50. Su capacidad para capturar relaciones semánticas y estructurales lo hizo un método fiable para el *clustering* de documentos con gran cantidad de datos.

La relevancia del estudio para el TFG radica en que explora algunos de los métodos de representación documental que se mencionan a lo largo de este TFG. Del estudio se obtiene en clave que, la elección del método de representación es crucial para mejorar la precisión de los algoritmos de clasificación automática basados en IA, y esta se debe adaptar al tipo de datos disponibles como, en este caso, Doc2Vec para conjuntos de datos amplios y bien estructurados, y LDA para datos limitados o dispersos.

El segundo documento para comentar se relaciona con el ámbito de la recuperación de información, uno de los desafíos más relevantes para las bibliotecas digitales es garantizar que los usuarios accedan de manera eficiente a los documentos que realmente responden a sus necesidades. Los autores Nowick, Travnicek, Eskridge y Stein analizan diferentes métodos de agrupación de términos empleados en la organización documental. El estudio se estructura en torno a la comparación de tres fuentes de términos:

- Vocabularios controlados: Son términos asignados por expertos en bibliotecología y organización documental. Esto garantiza consistencia y coherencia en la clasificación de documentos, ya que eliminan ambigüedades y establecen jerarquías temáticas bien definidas. Sin embargo, presentan limitaciones en términos de actualización, ya que la incorporación de nuevos términos es un proceso lento que requiere consenso entre bibliotecólogos. Además, algunos términos pueden no coincidir con el lenguaje natural utilizado por los usuarios, lo que dificulta su adopción en búsquedas en línea.

- Términos de búsqueda de los usuarios: Son las palabras clave que los usuarios ingresan en motores de búsqueda o catálogos bibliográficos. Reflejan de manera más natural la forma en que las personas buscan información, pero pueden presentar problemas como ambigüedad, errores ortográficos o el uso de términos coloquiales.

- Términos extraídos de documentos: Se obtienen mediante análisis de texto automatizado, lo que permite identificar conceptos relevantes dentro de grandes volúmenes de documentos. Los autores emplearon el software *Text Analysis Portal for Research* (TAPoR) para tokenizar los términos de documentos en formato HTML. Tras eliminar las *stopwords* y procesar los términos más frecuentes, encontraron que este método proporciona una visión integral del contenido documental, pero que los términos extraídos tienden a ser más generales y carecen de la especificidad que tienen los términos controlados o los términos de usuario.

Para llevar a cabo la comparación, los autores calcularon distancias entre términos utilizando el índice de Jaccard, que mide la similitud entre pares de palabras basándose en su co-ocurrencia en cada uno de los tres conjuntos de términos. Posteriormente, aplicaron análisis de correlación de Spearman para evaluar la relación entre las distancias de términos en cada fuente de datos. Los principales hallazgos del estudio fueron:

- Los términos de los vocabularios controlados tenían mayor relación con los términos de búsqueda de los usuarios que con los términos extraídos de los documentos.

- Los términos extraídos de documentos mostraban mayor dispersión y menor correlación con los otros dos métodos, lo que sugiere que los documentos en línea contienen información más heterogénea y difícil de estructurar en categorías fijas.

- El análisis de clústeres reveló que los términos más cercanos conceptualmente eran similares en los tres métodos, pero a medida que aumentaba la distancia dentro de los clústeres, las diferencias entre las fuentes se hacían más evidentes.

- Se identificaron diferencias significativas en la distribución de los términos únicos de cada fuente. Los términos de usuario contenían un 39% de errores ortográficos o variaciones en la escritura, mientras que los documentos presentaban una mayor cantidad de términos genéricos sin valor clasificatorio.

Los autores proponen que una combinación de enfoques podría mejorar la organización de bibliotecas digitales, integrando la estructura de los vocabularios controlados, la flexibilidad de las búsquedas de los usuarios y la riqueza de información contenida en los documentos digitales.

El estudio va en línea con los objetivos del TFG puesto que la comparación entre vocabularios controlados, términos de búsqueda y términos extraídos de documentos es fundamental para el desarrollo de sistemas de clasificación automática basados en IA, dado

que estos modelos pueden aprovechar técnicas avanzadas de procesamiento del lenguaje natural (PLN) y aprendizaje automático para mejorar la organización de documentos. En este sentido, se pueden abrir líneas de investigación para explorar cómo modelos de IA, como BERT o Word2Vec, pueden ser utilizados para generar representaciones más precisas de los términos y mejorar la clasificación automática de documentos.

La última fuente que se consulta en este apartado se trata de una reseña de Mirkin sobre un libro que enfatiza la importancia de los métodos de clasificación y *clustering* en la gestión de grandes conjuntos de datos. Los autores Krzysztof Jajuga, Andrzej Sokołowski y Hans-Hermann Bock analizan con detalle la evolución de los algoritmos de clasificación supervisada, destacando la eficacia de métodos como las máquinas de soporte vectorial (SVM) y *Random Forest* (versión mejorada del *Decision Tree*) en la categorización precisa de datos estructurados y no estructurados. También se presentan enfoques de aprendizaje no-supervisados como el *clustering* basado en densidad (DBSCAN) y el *clustering* jerárquico, resaltando sus ventajas en la segmentación de conjuntos de datos sin etiquetar.

Uno de los aspectos más relevantes del texto es la atención que se presta a la interpretabilidad de los modelos de clasificación automática. Los autores discuten cómo la creciente complejidad de los modelos basados en *Deep Learning* ha generado un dilema entre rendimiento y explicabilidad, señalando estrategias para solventarlo, como el uso de técnicas de explicabilidad como LIME y SHAP. Además, el texto examina los retos en la reducción del ruido en los datos y la optimización de hiperparámetros para mejorar la precisión de los modelos.

En cuanto a las aplicaciones prácticas se refiere, el libro describe estudios de caso en ámbitos como la medicina, donde los modelos de clasificación se utilizan para la detección temprana de enfermedades, y el análisis de redes sociales, donde los algoritmos de *clustering* permiten la identificación de comunidades y tendencias. Se destaca cómo la combinación de técnicas tradicionales con nuevos enfoques de *Deep Learning* está revolucionando la clasificación automática, especialmente en entornos de *Big Data*.

El contenido del libro resulta de interés para el desarrollo del Trabajo de Fin de Grado por múltiples aspectos. En primer lugar, proporciona una base teórica sólida sobre las metodologías utilizadas en el procesamiento y categorización de datos, lo que permite comprender las ventajas y limitaciones de los distintos enfoques. Además, la problemática de la interpretabilidad mencionada en el texto es un aspecto clave en el contexto del TFG, ya que en la clasificación de documentos es fundamental garantizar que los resultados sean comprensibles y justificados.

3.5. *Clustering*, clasificación y técnicas bibliométricas.

Narges Neshat y Anahita Kermani, a través del artículo "*Using the Citation-Content-Based Approach to Patent Clustering*", examinan métodos de agrupamiento de patentes con el objetivo de mejorar su recuperación y clasificación. Los autores exploran la utilización de citas dentro del contenido de las patentes como criterio de agrupamiento, comparando este enfoque con otros métodos tradicionales como el acoplamiento bibliográfico y el uso de títulos de citas. Para ello, desarrollan un sistema basado en *clustering Fuzzy C-Means* (FCM) y evalúan su desempeño mediante métricas de precisión y exhaustividad. El estudio se centra en un conjunto de patentes estadounidenses y analiza cómo la selección de atributos (citas completas vs. palabras en los títulos de las citas) influye en la efectividad del agrupamiento.

Los autores expresan que la correcta agrupación de patentes es crucial para la organización y recuperación de información tecnológica. En este sentido, analizan dos enfoques principales:

- Acoplamiento bibliográfico: Vincula patentes a través de las referencias que comparten.
- Palabras de los títulos de las citas: Extrae atributos específicos de los títulos de patentes citadas.

El estudio se desarrolla en tres fases:

1) La primera fase corresponde con la selección de datos, en este caso, una base de datos de 717 patentes citadas en 75 patentes pertenecientes a la clase 977/774 de la US Patent Classification (USPC). Se extraen atributos tanto de las citas completas como de las palabras clave de los títulos de citas, y se realiza un preprocesamiento eliminando términos irrelevantes mediante el "*Porter Stemmer*" y la eliminación de *stopwords*.

2) En la segunda fase es donde se desarrolla el experimento. Se implementa el algoritmo *Fuzzy C-Means* (FCM) en un entorno de programación en Perl. FCM se elige porque permite agrupamientos superpuestos, lo que es adecuado para las patentes, ya que pueden pertenecer a múltiples dominios tecnológicos.

3) La última parte concluye con la evaluación del experimento, utilizando métricas de *precision* y *recall*. Se observó que el acoplamiento bibliográfico generó agrupaciones más coherentes y estructuradas, con un EBR promedio de 0.615, mientras que el método basado en palabras de citas tuvo un EBR ligeramente inferior de 0.568. Sin embargo, el segundo enfoque redujo la cantidad de atributos en un 40%, sugiriendo que puede ser una opción más eficiente en términos de almacenamiento y procesamiento de datos. Los resultados también muestran que en umbrales de alta exhaustividad (0.009-0.014), el *recall* obtenido con palabras de títulos de citas fue comparable al del

acoplamiento bibliográfico, lo que indica que, en contextos donde la precisión no es prioritaria, este método podría ser una alternativa válida.

El estudio analizado tiene similitud con el TFG pese a que, en lugar de clasificar documentos, clasifique patentes, ya que ambos exploran estrategias de clasificación automática basadas en ML. En particular, el uso de *clustering Fuzzy C-Means* (FCM) y las métricas de evaluación pueden ser aplicables al análisis y organización de documentos en bases de datos. Además, la problemática de la selección de atributos en el *clustering* de patentes es extrapolable a la clasificación de documentos textuales. En futuras investigaciones, se podría explorar si una estrategia similar de selección de atributos (por ejemplo, uso de palabras clave vs. metadatos completos) mejora la precisión de la clasificación automática.

CAPÍTULO 4: CASO PRÁCTICO DE ALGORITMO DE CLASIFICACIÓN DE DOCUMENTOS TEXTUALES

4.1. Obtención de los datos

El primer paso en cualquier proyecto de *Machine Learning* o *Deep Learning* es la recopilación de datos. Aunque a simple vista pueda parecer una etapa secundaria, en realidad es fundamental, ya que el modelo posteriormente a desarrollar aprende directamente a partir de dichos datos. Por ello, es crucial que la información recopilada sea lo más precisa posible y esté bien normalizada, es decir, que todos los datos sigan una misma estructura y no contengan errores.

Para realizar el experimento, se han recopilado datos relacionados con capítulos de libros disponibles acceso abierto en el repositorio institucional e-Archivo de la Universidad Carlos III de Madrid. Para la descarga de dichos capítulos, se ha utilizado la herramienta "DSpace", una plataforma de repositorio digital muy utilizada por instituciones académicas para almacenar, gestionar y difundir documentos académicos. DSpace soporta el protocolo OAI-PMH (*Open Archives Initiative Protocol for Metadata Harvesting*), promovido por la OAI *Initiative*, que establece estándares técnicos para la interoperabilidad entre diferentes repositorios digitales. Gracias a esta compatibilidad, ha sido posible recolectar de forma automática y estructurada los metadatos de los capítulos, sin necesidad de acceder uno a uno a cada documento.

Finalmente, se han seleccionado cincuenta capítulos, todos ellos en castellano, como muestra para este experimento:

[1] 02_investi_univers.pdf
[2] 13mendia.pdf
[3] CAPITULO1-OAI-Introduccion.pdf
[4] CAPITULO7-OAI-Espana.pdf
[5] CAPITULO8-OAI-Software.pdf
[6] Estrategias_EIES_2022.pdf
[7] Percepcion_PSCTE_2005_109_134.pdf
[8] actividad_rodriguez_1993.pdf
[9] algoritmo_CASEIB_2011.pdf
[10] algoritmos_JOR_2003.pdf
[11] analisis_AOISFA_2007.pdf
[12] analisis_CIBEM_2017.pdf
[13] analisis_CNIM_2018_ps.pdf
[14] analisis_IPSCEI_2004.pdf
[15] aplicabilidad_fernandez_2010.pdf
[16] aplicacion_IIICSG_2016_ps.pdf
[17] aplicación_arrabales_2006.pdf

[18] aprendizaje_nieto_2016.pdf
[19] archivos_eiroa_2019.pdf
[20] bibliotecas_hernandez_2014_ps.pdf
[21] condicionalidad_molina_2018.pdf
[22] diseno_JNR_2017.pdf
[23] efectividad_ros_FECYT_2010.pdf
[24] estudio_CNIM_2018_ps.pdf
[25] evaluacion_XLJA_2019_ps.pdf
[26] globalizacion_mendez_SI_1999.pdf
[27] implementacion_CASEIB_2011.pdf
[28] influencia_AMF_2009.pdf
[29] influencia_MATCOMP_2015.pdf
[30] influencia_RM_1997.pdf
[31] informacion_nunez-nickel_1996.pdf
[32] innovacion_nieto_ONC_2010.pdf
[33] interfaz_CASEIB_2011.pdf
[34] introduccion_ciller_palacio_2016.pdf
[35] justicia_jullien_2021.pdf
[36] mecanismos_corres_2024.pdf
[37] mendez_maredata_ECOSISTEMAS_2018.pdf
[38] metadatos_mendez_FESABID2000_2000.pdf
[39] modelado_fernandez_2011_pp.pdf
[40] modelo_mendez_JCD_1999_ps.pdf
[41] nuevas_mendez_FESABID_1998_ps.pdf
[42] obtencion_AJA_2016_ps.pdf
[43] organizacion_rodriguez_1993_ps.pdf
[44] participacion_nieto_RSE_2009_pre.pdf
[45] reduccion_RLIN_2011_ps.pdf
[46] reparacion_Garcia_Laborum_2024.pdf
[47] seguridad_Peces_1992.pdf
[48] sincronizacion_URSI_2014.pdf
[49] sistema_mallo_1996.pdf
[50] transparencia_CAM_2021.pdf

4.2. Clasificación de Documentos con MATLAB

Para el desarrollo de un algoritmo de clasificación automática se ha utilizado la herramienta MATLAB.

MATLAB nació en los años 70 gracias a Cleve Moler, un matemático y científico computacional que trabajaba en la Universidad de Nuevo México. Su principal motivación fue

crear una herramienta sencilla que ayudara a sus estudiantes a acceder a las poderosas bibliotecas de álgebra lineal LINPACK y EISPACK, utilizadas para resolver sistemas de ecuaciones lineales. En ese entonces, estas bibliotecas requerían programación en Fortran, un lenguaje que no era fácil de manejar para los estudiantes de matemáticas. Así, Moler desarrolló una versión inicial de MATLAB que simplificaba mucho este proceso, permitiendo a los estudiantes usar las bibliotecas sin necesidad de programar en Fortran (Moler y Little, 2020, p. 5).

En los años 80, Moler se asoció con Jack Little y Steve Bangert para dar un gran paso: comercializar MATLAB. Juntos fundaron MathWorks en 1984, lo que permitió a MATLAB expandirse mucho más allá del ámbito académico. A partir de ahí, la herramienta comenzó a incorporarse en diversas áreas. A lo largo de los años, MATLAB ha añadido nuevas funcionalidades, como gráficos interactivos, simulación de sistemas y toolboxes especializados para distintos campos. Todo esto ha contribuido a convertirlo en una de las plataformas más utilizadas por ingenieros, científicos y empresas (Moler y Little, 2020, p. 5).

Hoy MATLAB es mucho más que una herramienta académica. Se utiliza en una variedad de campos como ingeniería, finanzas, ciencias aplicadas, inteligencia artificial y educación. Su evolución ha sido continua, siempre adaptándose a las necesidades de los usuarios y añadiendo nuevas funcionalidades que mantienen su relevancia (Moler y Little, 2020, p. 5).

4.2.1. Funcionamiento y Aplicaciones de MATLAB

El contenido de las siguientes dos secciones se basa en la documentación oficial proporcionada por MathWorks (2021) y en la experiencia propia utilizando dicha plataforma de programación.

MATLAB es una plataforma de programación y cálculo numérico diseñada para facilitar el análisis de datos, el desarrollo de algoritmos y la creación de modelos matemáticos. Su principal característica es el manejo eficiente de matrices y arrays, permitiendo realizar operaciones complejas con una sintaxis sencilla. Esto es especialmente útil en campos donde las operaciones matriciales son fundamentales, como la ingeniería y las ciencias aplicadas, entre otros.

El entorno de MATLAB está compuesto por diversas herramientas que permiten al usuario interactuar de manera intuitiva con los datos y el código. Por ejemplo, la ventana de comandos permite ejecutar instrucciones y ver los resultados de forma inmediata, lo que facilita la experimentación y el aprendizaje. Además, MATLAB incluye una amplia gama de funciones predefinidas para tareas computacionales, y ofrece la posibilidad de crear funciones personalizadas para reutilizar secuencias de comandos.

Una de las fortalezas de MATLAB es su capacidad para crear aplicaciones interactivas mediante App Designer, una herramienta que permite diseñar interfaces gráficas de usuario de manera visual y programar su comportamiento. Esto es especialmente útil para desarrollar aplicaciones personalizadas que automatizan tareas específicas.

MATLAB se usa en muchos campos diferentes gracias a sus potentes capacidades de cálculo y su amplia biblioteca de funciones:

- Ingeniería: MATLAB es muy común en el mundo de la ingeniería. Los ingenieros lo utilizan para diseñar sistemas, simular procesos y analizar señales. Gracias a su capacidad para resolver ecuaciones diferenciales y hacer simulaciones, MATLAB es una herramienta clave para los ingenieros que necesitan modelar sistemas dinámicos y predecir su comportamiento.

- Ciencias Aplicadas: En campos como la física, la biología o la química, MATLAB facilita la resolución de problemas complejos. Por ejemplo, se usa para analizar grandes cantidades de datos experimentales o simular fenómenos físicos. La capacidad de trabajar con matrices grandes y realizar operaciones de álgebra lineal hace que MATLAB sea indispensable para investigadores en estas áreas.

- Finanzas: En el mundo de las finanzas, MATLAB se utiliza para desarrollar modelos matemáticos que permiten optimizar carteras de inversión, analizar riesgos y realizar simulaciones de Montecarlo. Los analistas financieros lo usan para estudiar el comportamiento de los mercados y crear modelos predictivos basados en datos históricos. Esto lo convierte en una herramienta clave en el análisis cuantitativo.

- Inteligencia Artificial y Machine Learning: MATLAB ofrece herramientas específicas para trabajar con ML e IA. Utilizando el "*Statistics and Machine Learning Toolbox*", es posible desarrollar modelos de predicción, como redes neuronales o Máquinas de Soporte Vectorial (SVM), para tareas como el análisis de patrones o la clasificación de datos.

- Educación: MATLAB es una herramienta muy popular en las universidades, ya que su interfaz interactiva y su sintaxis simple hacen que los estudiantes puedan aprender y experimentar de manera efectiva, lo que facilita mucho el proceso de aprendizaje.

4.2.2. Ventajas y Limitaciones de MATLAB

MATLAB tiene muchas ventajas que lo hacen una de las herramientas más utilizadas en la investigación y la industria. Algunas de las más importantes son:

- Facilidad de uso: Una de las grandes ventajas de MATLAB es su sintaxis sencilla. Gracias a que se centra en el trabajo con matrices, puedes realizar operaciones complejas con solo unas líneas de código.

- *Toolboxes* especializados: MATLAB tiene una enorme cantidad de *toolboxes* que cubren áreas muy específicas como procesamiento de señales, análisis de imágenes, *Machine Learning*, etc. Estos *toolboxes* contienen funciones y herramientas predefinidas, lo que te ahorra mucho tiempo porque no tienes que programar todo desde cero. Para la clasificación de documentos textuales es necesario instalar la herramienta de MATLAB "*Text Analytics Toolbox*".

- Visualización de datos: Otra de las características destacadas de MATLAB es su capacidad de generar gráficos 2D y 3D de manera sencilla. Esto es muy útil, ya que poder visualizar los resultados de tus cálculos de forma clara te ayuda a entender mejor los datos y hacer presentaciones de calidad.

- Compatibilidad con otros lenguajes: MATLAB se puede integrar fácilmente con otros lenguajes como Python, C++, Java o incluso R.

- Comunidad y soporte: MATLAB tiene una gran comunidad global de usuarios, lo que significa que es fácil encontrar respuestas a problemas comunes, tutoriales y ejemplos de código. MathWorks también ofrece soporte técnico, lo que hace que el uso de MATLAB sea aún más accesible para estudiantes y profesionales.

A pesar de sus muchas ventajas, MATLAB también tiene algunas limitaciones que se deben considerar:

- Costo de las licencias: MATLAB es un software propietario, lo que significa que necesitas pagar por una licencia para usarlo. Esto puede ser un obstáculo para muchos estudiantes o pequeñas empresas, ya que las licencias pueden ser bastante caras. Además, los *toolboxes* adicionales también tienen un costo extra.

- Requisitos hardware: Aunque MATLAB es eficiente, puede ser muy intensivo en recursos, especialmente cuando se manejan grandes volúmenes de datos o se realizan cálculos complejos. En estos casos, necesitarás un equipo con suficiente capacidad de procesamiento y memoria para evitar que el rendimiento se vea afectado.

- En la clasificación de documentos textuales, MATLAB presenta una limitación relacionada con el idioma. Una de las etapas clave del procesamiento, como la lematización "normalizeWords" para normalizar las palabras, solo está disponible de forma nativa en inglés y alemán. Otros idiomas, como el español, que es el idioma de los documentos utilizados en el experimento, no cuentan con esta funcionalidad integrada.

- Uno de los principales algoritmos para la clasificación de documentos son los modelos de tipo *word embedding*, como Word2Vec, con el fin de capturar mejor las relaciones semánticas entre palabras. Inicialmente se intentó entrenar un modelo directamente desde MATLAB utilizando la función "trainWord2Vec", que aparece en algunos foros y ejemplos, pero rápidamente surgió un problema: dicha función no forma parte del conjunto oficial de funciones incluidas en el *Text Analytics Toolbox*, al menos en su versión pública. A pesar de tener instalada la versión R2024b y el *toolbox* correspondiente, MATLAB no reconoce esta función, lo que impide su uso directo. Esto supuso una limitación importante, ya que MATLAB no permite entrenar modelos Word2Vec propios de forma nativa. Además, aunque existen funciones como "word2vec" o "readWordEmbedding", estas están pensadas para usar modelos preentrenados a nivel de palabra, y no están optimizadas para representar documentos completos, como se necesita en este caso para aplicar *clustering*.

4.2.3. Algoritmos en MATLAB para la Clasificación Automática de Documentos

Este apartado recoge la parte práctica (código MATLAB) del experimento, que se encuentra en la sección de anexos de este proyecto. Para ello, primero se procederá a explicar la parte de código común, es decir, aquella parte que podría reutilizarse para cualquier experimento relacionado con la clasificación de documentos textuales. Posteriormente, se mostrarán distintas técnicas de *clustering* que se pueden utilizar en clasificación de documentos, y las partes del código que identifican a cada una de estas.

Durante la parte práctica del trabajo se ha utilizado inteligencia artificial, concretamente asistentes como ChatGPT, como apoyo para generar y adaptar código en MATLAB. Esta herramienta ha permitido resolver errores más rápido, probar distintas técnicas y agilizar el proceso de desarrollo.

En todo momento se ha supervisado personalmente el contenido generado, y se ha revisado y ajustado el código según las necesidades reales del proyecto, asegurándome de que cumpliera los objetivos y funcionara correctamente. Las decisiones metodológicas y la interpretación de los resultados son completamente propios, y la IA ha actuado únicamente como una ayuda puntual dentro de un proceso guiado por criterio personal.

La primera tarea para desarrollar el algoritmo consiste en procesar los documentos PDF seleccionados por el usuario. Para ello, el código comienza solicitando al usuario que elija una carpeta donde se encuentren los archivos PDF. En este trabajo, se ha seleccionado una carpeta que contiene una muestra de 50 documentos. Una vez realizada esta selección, se procede a extraer automáticamente el texto de cada archivo mediante una función incorporada en MATLAB. El contenido textual extraído se almacena en un arreglo de cadenas, donde cada

elemento corresponde al texto de un documento distinto. El código completo que implementa este procedimiento puede consultarse en el Anexo A.

Con el fin de mantener los archivos organizados y evitar reprocesamientos innecesarios, todos los PDFs que se han procesado correctamente se mueven a una subcarpeta denominada "processed". Esto permite tener separados los archivos ya tratados de los que aún no han sido procesados o han generado errores.

Posteriormente, se realiza una fase de preprocesamiento sobre el texto obtenido. En primer lugar, todo el contenido se convierte a minúsculas para unificar el formato y evitar duplicidades en el análisis. Luego, se tokeniza el texto, lo que significa que se divide en palabras individuales. Para mejorar la calidad del análisis y reducir el ruido en los datos, se eliminan las palabras vacías (*stopwords*), como artículos, preposiciones o conjunciones, que son comunes, pero no aportan valor semántico. También se suprimen los signos de puntuación y todas aquellas palabras con menos de tres caracteres, ya que en general no son representativas.

Además, se eliminan manualmente algunas preposiciones y adverbios frecuentes en español que no son útiles para los objetivos de este análisis. En un contexto distinto, como en el caso de documentos en inglés, aquí se podría utilizar la función “normalizeWords” para reducir las palabras a sus formas raíz (lematización), aunque en este caso no ha sido necesario. Al final de este proceso, se obtiene un conjunto de documentos en formato limpio y estructurado, listo para ser analizado en las siguientes etapas del trabajo.

Durante la parte práctica del trabajo se han desarrollado múltiples algoritmos de clasificación automática en MATLAB. La elección de estos algoritmos se basa en el sólido soporte que ofrece MATLAB para su implementación, así como en la amplia documentación y ejemplos prácticos que proporciona la plataforma. El objetivo general ha sido encontrar formas eficientes y semánticamente coherentes de agrupar un conjunto heterogéneo de documentos en PDF, previamente procesados y convertidos a representaciones textuales. A continuación, se describen los modelos utilizados, el propósito de cada uno y su funcionamiento básico.

- K-means con TF-IDF:

Este primer modelo aplica el algoritmo K-*means* directamente sobre los documentos representados mediante una matriz TF-IDF, sin reducción de dimensionalidad previa. La TF-IDF permite valorar la importancia relativa de cada palabra en cada documento, eliminando el sesgo de términos frecuentes, pero poco informativos. La novedad de este enfoque radica en que se utiliza la distancia coseno en lugar de la euclidiana, lo cual es más adecuado para datos textuales, ya que mide similitud angular entre vectores. Además, el número óptimo de clústeres

se selecciona automáticamente mediante el índice de *Silhouette*[3], lo que permite una agrupación más ajustada. El código completo está disponible en el Anexo B.

- K-means con LSA (Latent Semantic Analysis):

Este modelo introduce una mejora importante respecto al anterior mediante la aplicación de Latent Semantic Analysis (LSA) antes del *clustering*. A través de una descomposición SVD, se proyecta la matriz TF-IDF en un espacio de menor dimensión, donde los vectores reflejan temas latentes en lugar de simples frecuencias de palabras. Esto permite capturar relaciones entre documentos que no comparten exactamente las mismas palabras, pero sí contenidos semánticamente cercanos. Posteriormente se aplica K-*means* sobre estos vectores reducidos. El modelo mantiene las visualizaciones y la extracción de palabras clave, pero su diferencia clave es el enfoque semántico, que mejora la calidad del agrupamiento en presencia de sinónimos o redacción diversa. Ver código en el Anexo C.

- *Clustering* Jerárquico:

Este modelo emplea *clustering* jerárquico, un enfoque que permite agrupar documentos en una estructura en forma de árbol (dendrograma), sin necesidad de establecer de antemano el número de clústeres. A partir de la matriz TF-IDF, se calcula una matriz de distancias coseno entre los documentos. Además, el código permite opcionalmente aplicar una reducción semántica mediante LSA, activando o desactivando una variable de control ("applyLSA"). Si se activa, los documentos se proyectan en un espacio semántico de menor dimensión antes de calcular las distancias, lo que puede mejorar la calidad del agrupamiento. El dendrograma generado permite visualizar la estructura de similitud entre los documentos y seleccionar manualmente el número de clústeres deseado. A diferencia de métodos como K-*means*, este enfoque es determinista y proporciona una visión jerárquica que resulta útil para explorar el corpus a diferentes niveles de detalle. El código correspondiente se encuentra en el Anexo D.

- DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*):

Este modelo emplea DBSCAN, un algoritmo que identifica agrupaciones basadas en la densidad local de puntos. La principal diferencia frente a K-*means* o *clustering* jerárquico es que DBSCAN no requiere especificar el número de clústeres, y además puede detectar documentos "ruidosos" que no pertenecen a ningún grupo. Para aplicar este algoritmo correctamente a texto, se realiza una reducción semántica con LSA y se calcula una matriz de distancias coseno precomputadas, ya que DBSCAN no trabaja directamente con esta métrica.

[3] Índice de silhoutte: es una métrica que mide qué tan bien se agrupa un punto con los de su mismo clúster frente a otros clústers. Su valor va de -1 a 1: cuanto más cercano a 1, mejor definido está el clúster. Para encontrar el número óptimo de clases, se calcula este índice para diferentes valores de k y se elige el que tenga la silueta media más alta.

Este modelo es especialmente útil para detectar temas minoritarios o excepcionales en el corpus. El código está disponible en el Anexo E.

- GMM (*Gaussian Mixture Models*):

Aquí se utiliza *Gaussian Mixture Models* (GMM), un enfoque probabilístico que permite asignar a cada documento una probabilidad de pertenecer a varios clústeres. Este modelo es más flexible que K-*means*, ya que no asume formas esféricas ni tamaños iguales en los clústeres. La diferencia clave es que GMM realiza un ajuste estadístico y permite modelar solapamientos entre grupos. Se reduce la dimensionalidad con LSA para evitar *overfitting*, y se usa el criterio BIC (*Bayesian Information Criterion*) para estimar automáticamente el número óptimo de componentes. Esto lo hace especialmente valioso cuando se sospecha que los datos tienen una estructura compleja o continua. Código completo en el Anexo F.

- K-means con PCA (alternativa a *word embeddings*):

Durante el desarrollo del trabajo se exploró la posibilidad de representar los documentos mediante *word embeddings* como "Word2Vec" o "GloVe", pero debido a las limitaciones del entorno MATLAB, no fue posible utilizar estos modelos preentrenados. Como alternativa, se implementó un enfoque basado en PCA (*Principal Component Analysis*) aplicado directamente sobre la matriz TF-IDF. Esta técnica de reducción de dimensionalidad permite obtener una representación vectorial más compacta de los documentos, conservando aquellas componentes que explican mayor varianza del conjunto de datos.

En concreto, se redujo el espacio a 50 dimensiones, lo que hace posible trabajar con vectores densos más manejables y adecuados para técnicas de agrupamiento. A continuación, se aplicó el algoritmo K-*means* sobre estos vectores para clasificar automáticamente los documentos en ocho clústeres.

Este modelo es útil como alternativa a LSA y sirve para comparar el rendimiento de una reducción numérica como PCA frente a técnicas de reducción semántica. Aunque no capta relaciones de significado como LSA, sí permite una organización eficiente de los documentos basada en la estructura estadística del contenido. El código correspondiente se encuentra en el Anexo G.

- LDA (*Latent Dirichlet Allocation*):

A diferencia de los modelos anteriores, este enfoque no realiza un *clustering* de documentos, sino que emplea LDA para detectar automáticamente los tópicos latentes presentes en el corpus mediante el uso de la función "fitlda". LDA genera una serie de temas representados por conjuntos de palabras clave, y asigna a cada documento una distribución probabilística sobre dichos temas. Esto permite no solo identificar cuál es el tema dominante en cada texto, sino también reconocer temas secundarios que están presentes con cierto peso.

En este caso, se define un umbral del 10 % para mostrar los tópicos adicionales que también contribuyen al contenido del documento.

Este modelo es especialmente útil cuando el objetivo principal es interpretar el contenido de los documentos y comprender cómo se reparten las temáticas dentro del conjunto, más que clasificarlos en grupos cerrados. Los resultados incluyen una lista de palabras clave para cada tópico y la distribución temática individual por documento. El código correspondiente se encuentra en el Anexo H.

4.3. Evaluación de resultados de clasificación

En este apartado se analizan de forma cualitativa los resultados del experimento, ya que, al tratarse de una muestra pequeña, no tendría mucho sentido dividir los datos en conjunto de entrenamiento y prueba. El propósito principal de esta parte del trabajo es mostrar que la Inteligencia Artificial puede ser una herramienta eficaz para la clasificación de textos, más allá de obtener métricas cuantitativas precisas.

KMEANS

- Clúster 1: Trabajo, relaciones laborales y estructura organizativa.

Palabras clave: jornada, trabajadores, horas, trabajador, reducción, desempleo, solidaridad, formación, derecho, bronce.

Documentos recogidos: [11], [18], [21], [30], [32], [43], [44], [45], [46], [47], [48]

Este clúster agrupa correctamente la mayoría de los documentos. Estos encajan perfectamente, ya que abordan temas como reducción de jornada, estructura del trabajo y participación de los trabajadores. reparacion_Garcia_Laborum_2024.pdf también puede considerarse adecuado por su enfoque en justicia social. seguridad_Peces_1992.pdf se ajusta de forma más general, desde el punto de vista del derecho. Sin embargo, hay tres documentos que desentonan: aprendizaje_nieto_2016.pdf trata sobre modelos de aprendizaje automático en salud; influencia_RM_1997.pdf analiza propiedades de materiales metálicos; y sincronizacion_URSI_2014.pdf se centra en telecomunicaciones. Estos tres últimos no están relacionados directamente con la temática central del clúster y deberían haber sido agrupados en clústeres con temática más técnica.

- Clúster 2: Gestión de información, metadatos y documentación digital

Palabras clave: metadatos, información, rmación, imágenes, consultado, tesauros, semántica, intranet, gestión, sintaxis.

Documentos recogidos: [14], [26], [31], [38], [40], [41], [49]

Este clúster se caracteriza por un enfoque claro hacia la organización de información, los sistemas de metadatos y los lenguajes documentales. Documentos como metadatos_mendez_FESABID2000_2000.pdf, modelo_mendez_JCD_1999_ps.pdf, nuevas_mendez_FESABID_1998_ps.pdf y analisis_IPSCEI_2004.pdf encajan perfectamente. informacion_nunez-nickel_1996.pdf y sistema_mallo_1996.pdf también son apropiados, ya que estudian sistemas de información aplicados a la gestión empresarial. El caso más dudoso es globalizacion_mendez_SI_1999.pdf que, aunque no trata directamente sobre metadatos, sí menciona flujos de información y estructuras globales, lo que podría justificar su inclusión, aunque encajaría también en un clúster más político.

- Clúster 3: Robótica, automatización y control

Palabras clave: robot, robots, eslabones, bandeja, monje, blandos, cuello, robótica, blando, controlador.

Documentos recogidos: [22], [25], [42]

Este es un clúster excelente en cuanto a cohesión temática. Los tres documentos están centrados en robótica y sistemas automatizados. diseno_JNR_2017.pdf y obtencion_AJA_2016_ps.pdf abordan diseño de estructuras robóticas, y evaluacion_XLJA_2019_ps.pdf se enfoca en pruebas de evaluación técnica. Las palabras clave también son muy específicas y precisas.

- Clúster 4: Modelado técnico y análisis de materiales

Palabras clave: malla, texturizado, casco, modelado, ajuste, combate, citar, segmentos, adhesivo, wolframio.

Documentos recogidos: [13], [28], [29], [39]

Este grupo también está bien formado. modelado_fernandez_2011_pp.pdf y influencia_MATCOMP_2015.pdf se centran en simulaciones técnicas y refuerzos estructurales, mientras que influencia_AMF_2009.pdf trata sobre adhesivos en odontología, con base técnica similar. analisis_CNIM_2018_ps.pdf aborda evaluación biomecánica. Todas las palabras clave apuntan a un entorno de modelado 3D y simulación física. Es un clúster coherente y bien agrupado.

- Clúster 5: Automatización en subestaciones eléctricas

Palabras clave: 61850, subestaciones, demostración, eléctricas, automatización, amarís, hormigo, salazar, grids, fenosa.

Documentos recogidos: [16]

Este clúster está compuesto por un único documento, pero tiene sentido debido a la especificidad del tema. El contenido gira en torno a automatización de redes eléctricas

siguiendo el estándar IEC 61850. No hay otro documento con este nivel de especialización, por lo que su aislamiento en un clúster propio está justificado.

- Clúster 6: Interfaz médica y adquisición de imágenes

Palabras clave: v444, 50501, a2ah8, xxix, 5292p3, biomédica, interfaz, 5373, 28ch, pinole.

Documentos recogidos: [27], [33]

Ambos documentos están claramente vinculados a la imagen médica y la creación de interfaces gráficas para uso clínico. Las palabras clave son códigos y términos técnicos propios del área biomédica. El clúster es pequeño, pero temáticamente sólido.

- Clúster 7: Repositorios institucionales, OAI-PMH y ciencia abierta

Palabras clave: investigación, oaipmh, universidades, repositorios, creo, 10072007, archivos, repositorio, metadatos, software.

Documentos recogidos: [1], [3], [4], [5], [6], [19], [20], [37], [50]

Este es uno de los clústeres más coherentes del experimento. Todos los documentos tratan sobre repositorios académicos, visibilidad científica, interoperabilidad o acceso abierto. Las palabras clave apuntan a tecnologías OAI-PMH y gestión de archivos. Todos los documentos están bien agrupados.

- Clúster 8: Incentivos, percepción pública e innovación

Palabras clave: encuestados, incentivos, fiscales, científi, freno, empresas, ayudas, preferencias, esfuerzo, tecnológica.

Documentos recogidos: [7], [8], [12], [23]

Este grupo tiene un enfoque mixto entre percepción pública de la ciencia, evaluación de políticas e incentivos a la innovación. Todos los documentos están relacionados con análisis cuantitativos de política científica. actividad_rodriguez_1993.pdf es el más cercano al ámbito económico, pero puede considerarse adecuado por su análisis sobre incentivos. Es un clúster coherente, aunque algo híbrido.

- Clúster 9: IA cognitiva y conciencia artificial

Palabras clave: consciencia, hilos, consciousness, klor, tejidos, procesadores, baars, hilo, parallel, inconscientes.

Documentos recogidos: [10], [17], [24]

Este clúster es muy específico y se centra en modelos de conciencia artificial, sistemas cognitivos y procesamiento paralelo. Todos los documentos están alineados con esta temática y las palabras clave son muy representativas.

- Clúster 10: Derecho, política y elecciones

Palabras clave: partidos, penal, candidaturas, electorales, políticos, derecho, maltrato, animal, elecciones, candidatos.

Documentos recogidos: [2], [15], [34], [35], [36]

Este clúster reúne textos con un fuerte componente jurídico y político. 13mendia.pdf, justicia_jullien_2021.pdf, mecanismos_corres_2024.pdf e introduccion_ciller_palacio_2016.pdf encajan muy bien. aplicabilidad_fernandez_2010.pdf podría parecer técnico, pero si se entiende desde una perspectiva ética o regulatoria, su inclusión es razonable.

La implementación directa de K-Means sobre la matriz TF-IDF generó agrupaciones razonables, aunque con ciertas limitaciones en cuanto a cohesión semántica. Al no reducir la dimensionalidad ni suavizar el espacio vectorial, los documentos se agruparon principalmente por coincidencias léxicas más que por afinidades temáticas profundas. Esto provocó que algunos clústeres fueran poco interpretables y que textos con vocabulario similar, pero distinto enfoque, quedaran agrupados. En general, fue útil como punto de partida para establecer la segmentación del corpus, pero con resultados algo rígidos.

KMEANS CON LSA

- Clúster 1: Análisis de materiales, simulaciones y biomecánica

Palabras clave: casco, freno, combate, adhesivo, wolframio, ensayos, grano, balístico.

Documentos recogidos: [12], [13], [27], [28], [29]

Este grupo agrupa principalmente estudios sobre propiedades mecánicas y biomateriales. influencia_MATCOMP_2015.pdf y influencia_AMF_2009.pdf encajan perfectamente, ya que se centran en resistencia de materiales. analisis_CNIM_2018_ps.pdf y CIBEM_2017.pdf abordan análisis clínico y biomecánico, que también tiene relación. implementacion_CASEIB_2011.pdf, aunque centrado en una interfaz médica, puede considerarse relacionado si se analiza su componente de hardware o simulación. En general, este clúster está bien definido en torno a materiales y su aplicación médica o estructural.

- Clúster 2: Documentación, archivos, gestión de información

Palabras clave: metadatos, texturizado, malla, archivos, información, modelado, derecho, socialismo.

Documentos recogidos: [14], [16], [19], [20], [26], [37], [38], [39], [40], [41], [46], [47], [50]

Este grupo tiene una temática más amplia, pero gira alrededor del tratamiento de la información, archivos institucionales y modelos de gestión. Destacan textos como metadatos_mendez_FESABID2000_2000.pdf y modelo_mendez_JCD_1999_ps.pdf, que se centran en estructuras de información. archivos_eiroa_2019.pdf y transparencia_CAM_2021.pdf también encajan bien. modelado_fernandez_2011_pp.pdf parece no encajar por completo, ya que trata de simulación 3D, pero puede estar aquí por los términos "modelo" y "estructura". reparacion_Garcia_Laborum_2024.pdf y seguridad_Peces_1992.pdf se relacionan más desde el plano legal o institucional. En conjunto, es un clúster coherente, aunque algo heterogéneo.

- Clúster 3: Repositorios OAI-PMH y acceso abierto

Palabras clave: oaipmh, repositorios, protocolo, metadatos, archivos, consulta.

Documentos recogidos: [3], [4], [5], [33]

Este clúster está claramente centrado en tecnologías de acceso abierto y protocolos de interoperabilidad como OAI-PMH. Los capítulos 1, 7 y 8 pertenecen a un mismo marco teórico y están bien agrupados. interfaz_CASEIB_2011.pdf puede parecer fuera de lugar, pero si se considera su parte técnica sobre visualización y estructura de datos, tiene sentido incluirlo. Buena coherencia global.

- Clúster 4: Relaciones laborales y condiciones de trabajo

Palabras clave: jornada, trabajadores, reducción, empresa, formación, turnos. Documentos

recogidos: [11], [32], [44], [45]

Todos los documentos tratan sobre empleo, condiciones laborales o participación del trabajador. Es un clúster muy bien agrupado y con sentido social-económico.

- Clúster 5: Procesamiento técnico, paralelismo y algoritmos

Palabras clave: hilos, procesadores, tejidos, parallel, fricción.

Documentos recogidos: [10], [24]

Ambos textos exploran estructuras algorítmicas, modelos de ejecución o estructuras técnicas. Aunque breves, los documentos son coherentes.

- Clúster 6: Igualdad de género y conciliación

Palabras clave: igualdad, conciliación, mujeres, hombres, colaboración.

Documentos recogidos: [2]

El documento está centrado en igualdad institucional, con referencias a figuras políticas. Al ser un tema tan específico, está bien que tenga su propio clúster. No hay otros textos parecidos en el conjunto.

- Clúster 7: Derecho penal y política electoral

Palabras clave: partidos, penal, elecciones, electoral, crímenes.

Documentos recogidos: [15], [36]

Ambos documentos están relacionados con estructuras legales o aplicación jurídica. mecanismos_corres_2024.pdf trata de reparación en contextos políticos y aplicabilidad_fernandez_2010.pdf puede enfocarse desde la ética legal si se interpreta así. Clúster jurídico/político acertado.

- Clúster 8: Cláusulas sociales y comercio internacional

Palabras clave: condicionalidad, comercio, cláusulas, gobernanza.

Documentos recogidos: [21]

Este documento trata sobre gobernanza económica y cláusulas sociolaborales. Es temáticamente único y está bien que se agrupe solo. No hay otro documento en la colección con este enfoque.

- Clúster 9: Producción audiovisual

Palabras clave: cine, producción, audiovisual, rodaje, televisión. Documentos

recogidos: [34]

El contenido es muy específico del mundo del cine, por lo que es lógico que no comparta grupo con ningún otro documento. Su aislamiento es correcto.

- Clúster 10: Robótica y sistemas blandos

Palabras clave: robot, robots, blandos, controlador, cuello.

Documentos recogidos: [22], [25], [42]

Clúster técnico y muy coherente. Todos los textos tratan de estructuras robóticas, mecanismos flexibles o controladores. Buena agrupación.

- Clúster 11: Percepción social, IA, incentivos y estructuras organizativas

Palabras clave: consciencia, incentivos, encuestados, maltrato, fiscales, empresas.

Documentos recogidos: [7], [8], [17], [23], [30], [31], [35], [43], [48], [49]

Este es uno de los clústeres más amplios y también más heterogéneos. Hay documentos sobre percepción pública (Percepcion_PSCTE), sobre conciencia artificial (aplicación_arrabales), y sobre incentivos laborales o fiscales (actividad_rodriguez, ros_FECYT). También hay textos técnicos como sincronizacion_URSI_2014.pdf y materiales como influencia_RM_1997.pdf. Aunque se pueden justificar por compartir un enfoque institucional o estructural, quizá sería conveniente dividirlo entre subtemas más concretos.

- Clúster 12: Universidad, docencia e investigación científica

Palabras clave: investigación, universidades, docencia, sexenios, estudiantes.

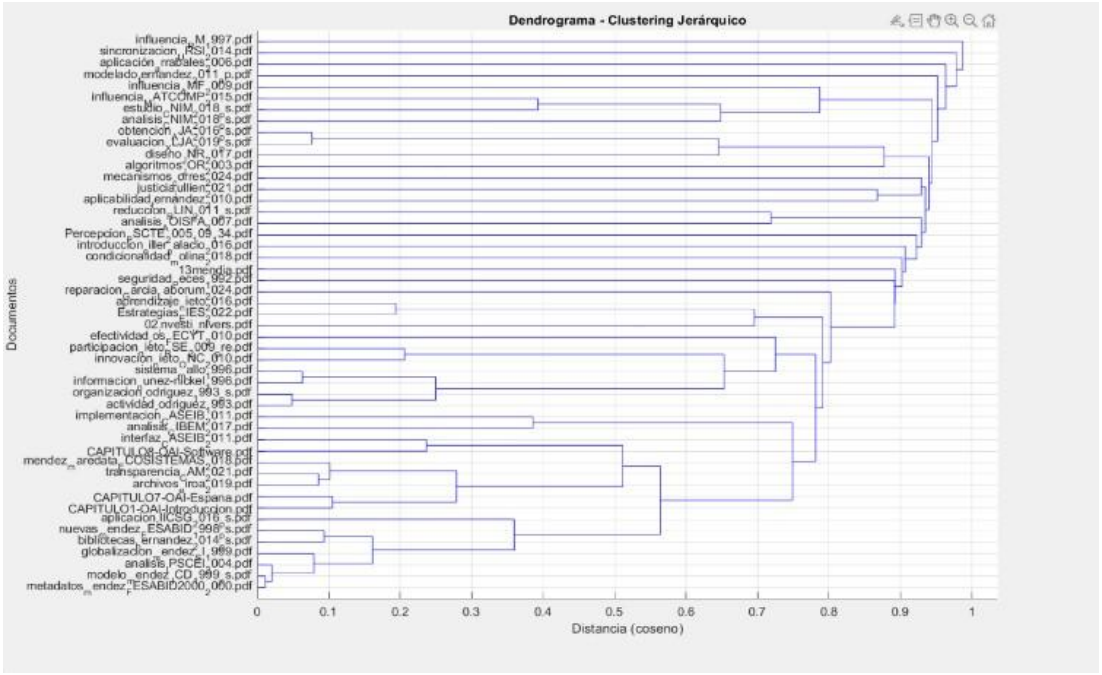
Documentos recogidos: [1], [6], [18]

Este clúster agrupa textos sobre investigación universitaria y políticas científicas. Todos los documentos están bien agrupados, ya que tratan directamente sobre el sistema académico y su funcionamiento.

Al aplicar una reducción de dimensionalidad mediante LSA (Latent Semantic Analysis) antes del K-Means, se mejoró notablemente la calidad de los clústeres. LSA permitió capturar relaciones semánticas latentes entre términos y documentos, facilitando que el algoritmo agrupase textos no solo por palabras compartidas, sino por significados similares. Las agrupaciones obtenidas fueron más coherentes temáticamente, y se observaron clústeres bien definidos (como uno claramente centrado en temas de metadatos o tecnologías de la información). Esta combinación demostró ser una de las más eficaces para este tipo de corpus académico.

HIERARCHICAL CLUSTERING

Figura 4.3: *Dendograma de clustering jerárquico*



Fuente: Elaboración propia a través de MATLAB

- Clúster 1: Política electoral y partidos

Palabras clave: partidos, candidaturas, electorales, políticos, elecciones, candidatos, electoral, colombia, venecia, derecho.

Documentos recogidos: [36]

Este clúster contiene un único documento, pero su asignación es coherente. El contenido de mecanismos_corres_2024.pdf está claramente relacionado con procesos políticos, reparaciones y estructuras institucionales, lo cual se refleja bien en las palabras clave extraídas. No hay otros documentos en el conjunto con temática tan específicamente electoral, por lo que está correctamente aislado.

- Clúster 2: Derecho penal y justicia restaurativa

Palabras clave: penal, maltrato, animal, crimenes, animales, derecho, restaurativa, internacional, delitos.

Documentos recogidos: [15], [35]

Este clúster agrupa correctamente textos con orientación jurídica y penal. justicia_jullien_2021.pdf trata sobre justicia restaurativa, y aplicabilidad_fernandez_2010.pdf, aunque también podría vincularse a IA, tiene una dimensión legal que encaja.

- Clúster 3: Jornada laboral y condiciones de trabajo

Palabras clave: jornada, horas, reducción, desempleo, trabajadores, turnos, días, trabajador, fabricantes, automóviles.

Documentos recogidos: [11], [45]

Ambos documentos tratan aspectos laborales y reducción de jornada. Las palabras clave coinciden con temas laborales, como desempleo y organización del tiempo. Es un clúster pequeño, pero correctamente agrupado.

- Clúster 4: Gestión del conocimiento, universidades y sistemas de información Palabras

clave: investigación, metadatos, oaipmh, universidades, repositorios, empresa, creo, archivos, 10072007, empresas.

Documentos recogidos: [1], [2], [3], [4], [5], [6], [7], [8], [12], [14], [16], [18], [19], [20], [21], [23], [26], [27], [31], [32], [33], [34], [37], [38], [40], [41], [43], [44], [46], [47], [49], [50]

Este clúster contiene una gran variedad de documentos relacionados con gestión institucional, universidad, políticas científicas y metadatos. Las palabras clave reflejan bien este enfoque mixto entre investigación, documentación y estructuras académicas. Aunque es un grupo numeroso y algo heterogéneo, los textos tienen en común su orientación hacia la gestión del conocimiento, digitalización o gobernanza educativa. La coherencia general es buena, aunque podría dividirse en subclústeres para mayor precisión.

- Clúster 5: Robótica y estructuras blandas

Palabras clave: robot, robots, eslabones, bandeja, monje, blandos, cuello, robótica, blando, parallel.

Documentos recogidos: [10], [22], [25], [42]

Un ejemplo de muy buen agrupamiento, ya que todos los textos están relacionados con robótica blanda y estructuras mecánicas. Las palabras clave indican claramente el tema y los documentos comparten el enfoque técnico.

- Clúster 6: Materiales y simulación computacional

Palabras clave: casco, numérico, combate, klor, tejidos, adhesivo, wolframio, grano, hilos, itria.

Documentos recogidos: [13], [24], [28], [29]

Los cuatro documentos están enfocados en materiales, pruebas numéricas y propiedades mecánicas. Las palabras clave reflejan conceptos técnicos y físicos. El clúster es muy coherente. Podría considerarse una continuación técnica del clúster 5, pero con foco más material y de simulación.

- Clúster 7: Modelado 3D y representación gráfica

Palabras clave: texturizado, malla, modelado, citar, segmentos, ajuste, personaje, primitivas, vértices, objetos.

Documentos recogidos: [39]

Aunque es solo un documento, su contenido técnico justifica que esté aislado. Las palabras clave apuntan a modelado gráfico y representación de geometría 3D.

- Clúster 8: Conciencia artificial y teorías cognitivas

Palabras clave: consciencia, consciousness, baars, inconscientes, consciente, conscientes, artificial, teorías, multiagente, crick.

Documentos recogidos: [17]

Este texto aborda el estudio de la consciencia artificial. Dada su temática específica y la terminología, está bien que forme su propio clúster. Las palabras clave son muy precisas y el aislamiento del documento está justificado.

- Clúster 9: Sincronización y señales

Palabras clave: estimacion, sincronizacion, canal, estaciones, ofdm, metodo, simultanea, senal.

Documentos recogidos: [48]

Se trata de un documento técnico centrado en sistemas de transmisión y sincronización.
No hay otros textos similares, así que es lógico que forme un clúster individual.

- Clúster 10: Aleaciones metálicas y tratamientos

Palabras clave: bronce, cobre, aceros, sinterización, acero, 304l, inoxidables, 316l, aleante, 1150.

Documentos recogidos: [30]

Este documento trata sobre la composición y propiedades de materiales metálicos. Las palabras clave son muy específicas del campo metalúrgico. Está bien separado del resto.

Este modelo destacó principalmente por su capacidad visual de representar las distancias entre documentos. El dendrograma permitió identificar relaciones progresivas y observar cómo se van formando clústeres a diferentes niveles. La principal dificultad fue determinar objetivamente dónde realizar el corte para establecer un número óptimo de grupos. A pesar de ello, los resultados fueron satisfactorios: se identificaron agrupaciones bien fundamentadas, como textos centrados en políticas laborales o en arquitectura de sistemas de información. Este enfoque fue especialmente útil para explorar la estructura general del corpus más allá de una segmentación fija.

DBSCAN

- Documentos clasificados como "ruido" (no asignados en ningún clúster): [2], [7], [10], [11], [15], [17], [21], [22], [23], [25], [30], [34], [35], [36], [39], [42], [45], [46], [47], [48]

- Documentos recogidos en clúster 1: [1], [3], [4], [5], [6], [8], [12], [13], [14], [16], [18], [19], [20], [24], [26], [27], [28], [29], [31], [32], [33], [37], [38], [40], [41], [43], [44], [49], [50]

Tras aplicar el modelo de agrupamiento DBSCAN a la muestra de documentos, los resultados no han sido satisfactorios. Aunque se generó un único clúster principal, 20 documentos se clasificaron como "ruido", es decir, no se asignaron a ningún clúster, lo que indica que el modelo no logró organizar bien gran parte de la muestra.

Además, el hecho de que se haya formado solo un clúster es un problema, ya que los documentos incluidos en él no comparten temáticas similares entre todos ellos. Se mezclan textos sobre educación, tecnología, políticas públicas, bibliotecas, entre otros temas muy distintos, lo que demuestra que el modelo no ha sido capaz de diferenciar contenidos relevantes.

Cabe destacar que se intentaron ajustar los hiperparámetros "épsilon" y "minPts" del modelo con el objetivo de mejorar los resultados, pero no se lograron los resultados esperados. Esto sugiere que la distribución de densidades entre los documentos no era lo suficientemente homogénea para que DBSCAN pudiera identificar clústeres bien definidos. Este algoritmo funciona mejor cuando hay una separación clara entre grupos de alta y baja densidad, lo que no parece haber ocurrido en este caso.

GMM

- Clúster 1: Derecho filosófico / justicia restaurativa

  Documentos recogidos: [35]

Este documento queda aislado, lo cual tiene sentido dado su contenido filosófico y reflexivo sobre la justicia. Es un tema específico y poco compartido con el resto, por lo que el modelo ha hecho bien en asignarlo a un clúster único.

- Clúster 2: Condicionalidad social y política internacional

  Documentos recogidos: [21]

  También aparece solo, lo cual puede justificarse por su enfoque muy concreto en cláusulas de condicionalidad sociolaboral, que no comparte directamente núcleo temático con otros documentos.

- Clúster 3: Diseño de robots

  Documentos recogidos: [22]

  Se trata de un documento centrado en aspectos técnicos del diseño de sistemas robóticos, y no encuentra similitudes suficientes con otros textos. La separación individual aquí es aceptable.

- Clúster 4: Repositorios y software académico

  Documentos recogidos: [5]

  El capítulo sobre software y protocolos del entorno OAI también ha sido ubicado en solitario, probablemente por el uso de terminología específica que no se repite con frecuencia en otros textos. Sin embargo, podría haber sido incluido en el clúster donde se encuentran ubicados otros documentos relacionados con OAI.

- Clúster 5: Macrogrupo institucional-universitario

Documentos recogidos: [2], [3], [4], [8], [16], [18], [19], [20], [26], [27], [29], [31], [33], [34], [37], [38], [39], [40], [41], [42], [43], [44], [46], [49]

Este es el clúster más grande y variado, que agrupa documentos relacionados con universidades, sistemas de información, innovación académica, gestión institucional y repositorios. Aunque temáticamente es amplio, los textos comparten un enfoque institucional o documental, lo que justifica su agrupación general. Podría beneficiarse de una subdivisión interna.

- Clúster 6: Evaluación académica

  Documentos recogidos: [12]

  Este documento está centrado en aspectos concretos de análisis cuantitativo y percepción en educación. Aunque se podría haber agrupado con textos similares del clúster 11, el modelo lo ha separado como un caso específico.

- Clúster 7: Algoritmos e inteligencia artificial

  Documentos recogidos: [10]

  Texto técnico sobre algoritmos. Probablemente su vocabulario técnico no fue compartido con otros textos, pero podría haber tenido cierta relación con aplicación_arrabales_2006.pdf o aplicabilidad_fernandez_2010.pdf por su temática respecto a la IA. Sin emabrgo, su aislación es correcta ya que no se trata de agrupar por agrupar si el contenido no es demasiado parecido como en estos casos, hay que tener en cuenta que el tamaño de la muestra no es muy grande, en muetsras más grandes seguramente hubiese sido agrupado con otros docuemntos con los que comparta mayor relación.

- Clúster 8: Gestión universitaria e institucional

  Documentos recogidos: [14]

  Este análisis institucional queda aislado. Sin embargo, por contenido parece más cercano al clúster 5 o 11, por lo que esta separación podría considerarse discutible.

- Clúster 9: Aplicaciones éticas o filosóficas de la IA

  Documentos recogidos: [15]

  Al igual que en otros modelos, queda apartado debido a su temática híbrida entre derecho, filosofía e inteligencia artificial. Su asignación a un clúster independiente es coherente, pese a que como se ha mencionado anteriormente, se podría haber agrupado con algoritmos_JOR_2003.pdf aunque la compatibilidad no sea tan alta.

- Clúster 10: Materiales y resistencia

  Documentos recogidos: [28]

  Centrado en propiedades de materiales, pero queda solo. Dada la existencia de otros documentos técnicos (influencia_MATCOMP_2015.pdf, etc.), su separación podría ser un punto débil del modelo.

- Clúster 11: Ciencia, sociedad, y políticas técnicas

Documentos recogidos: [1], [6], [7], [11], [13], [23], [24], [25], [30], [32], [36], [45], [47], [48]

Este clúster reúne textos centrados en estrategias científicas, políticas laborales, análisis técnico y social. Aunque no todos los documentos tienen una temática idéntica, comparten cierto enfoque sociotécnico, con peso en la política pública, tecnología, e impacto institucional. La agrupación es bastante razonable.

- Clúster 12: Conciencia artificial y neurociencia

Documentos recogidos: [17]

Este texto trata sobre conciencia artificial, modelos cognitivos y agentes inteligentes.
Su aislamiento es coherente, dada la especificidad del tema.

El modelo GMM permitió modelar la distribución de los documentos de forma probabilística, captando la superposición entre grupos. Los resultados fueron mixtos: algunos clústeres tenían sentido temático, pero en otros casos la asignación parecía arbitraria. El modelo fue más flexible que K-Means, al no imponer fronteras rígidas entre clústeres, aunque esto también complicó la interpretación de los resultados. La selección del número óptimo de clústeres mediante BIC fue útil, pero la convergencia del modelo fue más compleja que en otros métodos.

WORD EMBEDDINGS (PCA + KMEANS)

- Clúster 1: Documentos técnicos sobre robótica y evaluación

  Documentos recogidos: [22], [25]

Ambos tienen un enfoque técnico y cuantitativo, el primero centrado en diseño robótico y el segundo en evaluación de sistemas. Aunque no son idénticos temáticamente, comparten una orientación metodológica y terminología técnica, por lo que su agrupación es coherente.

- Clúster 2: Justicia y reparación

  Documentos recogidos: [35], [46]

Abordan temáticas relacionadas con justicia, derechos y reparación, desde un enfoque ético, legal y social. Su agrupación es muy acertada, ya que comparten tanto vocabulario como enfoque reflexivo sobre el sistema judicial y la reparación del daño.

- Clúster 3: Infraestructura documental y bibliotecas

Documentos recogidos: [3], [4], [5], [14], [20], [38], [40], [41]

Aquí se agrupan documentos relacionados con repositorios, metadatos y documentación académica. Este clúster representa de forma clara el eje temático de sistemas de información, bibliotecas y gestión de recursos digitales, por lo que la agrupación resulta muy adecuada.

- Clúster 4: Empresa, sociedad y transparencia

Documentos recogidos: [8], [23], [31], [43], [44], [49], [50]

Este clúster contiene textos vinculados a temáticas de gestión empresarial, responsabilidad social, transparencia y organización institucional. Aunque no todos tienen un enfoque idéntico, sí comparten un lenguaje relacionado con administración, gestión y políticas organizativas. La agrupación es aceptable y muestra cohesión temática.

- Clúster 5: Producción científica, investigación y condiciones laborales

Documentos recogidos: [1], [7], [11], [32], [45]

Este grupo incluye documentos vinculados al ámbito de la producción científica, la percepción académica, y las condiciones laborales o investigadoras. Aunque tienen distintos enfoques (sociológico, político, económico), comparten términos relacionados con ciencia, empleo, innovación y análisis de contextos.

- Clúster 6: Documento aislado

Documentos recogidos: [2]

El documento 13mendia.pdf ha quedado agrupado solo, probablemente por tener un lenguaje institucional, político o de igualdad poco presente en otros textos. Su aislamiento puede estar justificado, pero también podría haberse aproximado a otros documentos como los del clúster 4. Su separación no es un error, pero sí una posible limitación del método.

- Clúster 7: Clúster técnico y académico amplio

Documentos recogidos: [6], [10], [12], [13], [16], [17], [18], [19], [24], [26], [27], [28], [29], [30], [33], [34], [37], [39], [42], [48]

Este es el grupo más grande y diverso. Se observa que estos textos, aunque variados, tienen en común un enfoque técnico-académico aplicado, con muchas referencias a implementación, tecnología, ciencia, y aplicaciones reales. Es lógico que hayan sido agrupados juntos, aunque también sería posible subdividirlos en subconjuntos más específicos.

- Clúster 8: Temas legales, condiciones laborales y seguridad

Documentos recogidos: [15], [21], [36], [47]

Este grupo reúne documentos que, aunque aparentemente parezcan dispares, todos ellos abordan temas legales, laborales o de protección institucional. Puede entenderse como un grupo de textos con orientación normativa o regulatoria, y aunque no es el clúster más compacto, tiene cierto sentido.

Dado que en MATLAB no fue posible aplicar directamente modelos Word2Vec entrenables, se optó por una solución alternativa: representar los documentos mediante *embeddings* preprocesados y aplicar posteriormente una reducción de dimensionalidad con PCA, seguido de K-*Means*. Esta combinación ofreció resultados aceptables, aunque los

clústeres eran menos finos semánticamente que los obtenidos con LSA. La riqueza del análisis se vio limitada por el uso indirecto de *embeddings*, pero el experimento sirvió para probar el potencial de métodos más modernos de representación textual dentro de las limitaciones técnicas del entorno.

LDA

- Tópico 1: Empresa, producción, gestión y formación

Palabras clave destacadas: empresas, empresa, formación, producción, innovación, sector, actividades.

Documentos con este tema como principal: [2], [8], [10], [22], [31], [33], [42], [43], [48], [49]

Este tema agrupa documentos centrados en aspectos organizacionales y técnicos del entorno productivo, con una fuerte presencia en estudios sobre formación, implementación tecnológica o estructuras empresariales. Refleja bien contenidos relacionados con gestión y desarrollo profesional.

- Tópico 2: Investigación, educación superior y universidades

Palabras clave destacadas: investigación, universidad, también, este, social, sino. Documentos con

este tema como principal: [1], [18], [19], [20], [3], [4], [5], [6], [36], [46]

Este tópico recoge documentos del ámbito universitario y científico, donde se discuten políticas educativas, gestión del conocimiento, innovación académica y estructuras de investigación. Es un tema con alta coherencia y presencia transversal en textos académicos.

- Tópico 3: Trabajo, derecho laboral y condiciones

Palabras clave destacadas: trabajo, derecho, jornada, internacional, tiempo, trabajador, horas.

Documentos con este tema como principal: [14], [15], [16], [21], [24], [29], [44], [45], [47]

Agrupa textos que tratan sobre derechos laborales, condiciones de empleo, políticas de jornada y relaciones sociales en el trabajo. Muchos de estos documentos analizan el impacto de medidas laborales o normativas sobre los trabajadores.

- Tópico 4: Política, tecnología y análisis institucional

Palabras clave destacadas: resultados, tecnología, partidos, españa, política, selección, estudios.

Documentos con este tema como principal: [7], [9], [27], [28], [35], [41]

Este tema mezcla el análisis de sistemas políticos con herramientas tecnológicas, incorporando textos que usan metodología cuantitativa o tratan sobre procesos institucionales. Algunos documentos tienen un enfoque dual (sociopolítico y técnico).

- Tópico 5: Estilo académico y enfoque metodológico

Palabras clave destacadas: modelo, capítulo, proceso, figura, forma, manera.

Documentos con este tema como principal: [11], [12], [23], [26], [32], [38]

Este tópico es más abstracto y refleja el estilo de redacción típico de textos académicos o científicos. Se caracteriza por el uso de terminología metodológica y estructuras comunes en la literatura investigadora. Es menos temático y más estructural.

- Tópico 6: Información, datos y sistemas documentales

Palabras clave destacadas: información, datos, metadatos, sistemas, archivos, acceso, gestión.

Documentos con este tema como principal: [13], [19], [5], [25], [40], [39], [37], [47]

Es uno de los tópicos más coherentes, agrupando documentos que tratan sobre gestión de datos, digitalización, interoperabilidad, sistemas de archivo y bibliotecas digitales. Refleja un enfoque claro sobre la tecnología documental.

A continuación, se muestra una tabla que contiene cada documento de manera individual y los respectivos porcentajes de similitud que estos tienen con cada uno de los tópicos:

Tabla 4.1: *Distribución porcentual de tópicos por documento*

Documento	T1	T2	T3	T4	T5	T6
02_investi_univers.pdf	23.37	39.56	-	10.52	-	-
13mendia.pdf	27.73	13.48	13.45	10.70	25.76	-
CAPITULO1-OAI-Introduccion.pdf	12.63	16.49	-	-	12.17	44.98

CAPITULO7-OAI-Espana.pdf	13.87	14.65	-	-	12.58	48.49
CAPITULO8-OAI-Software.pdf	-	13.01	-	-	14.97	54.50
Estrategias_EIES_2022.pdf	13.61	23.75	12.91	-	16.64	25.47
Percepcion_PSCTE_2005_109_134.pdf	14.13	18.07	15.87	45.51	-	-
actividad_rodriguez_1993.pdf	33.14	10.57	11.00	20.84	14.16	10.29
algoritmo_CASEIB_2011.pdf	-	14.69	20.76	24.23	17.76	13.55
algoritmos_JOR_2003.pdf	29.48	11.13	35.33	-	-	-
analisis_AOISFA_2007.pdf	12.18	13.30	11.16	15.70	28.26	19.39
analisis_CIBEM_2017.pdf	-	-	-	15.51	46.53	12.65
analisis_CNIM_2018_ps.pdf	19.68	16.76	-	-	16.45	32.75
analisis_IPSCEI_2004.pdf	16.38	12.33	37.77	10.54	12.67	10.31
aplicabilidad_fernandez_2010.pdf	15.11	20.87	25.69	-	12.82	19.46
aplicacion_IIICSG_2016_ps.pdf	10.93	20.76	20.21	15.28	15.46	17.36
aplicación_arrabales_2006.pdf	26.20	23.83	15.35	-	12.28	12.56
aprendizaje_nieto_2016.pdf	20.01	25.53	-	11.82	14.69	18.04
archivos_eiroa_2019.pdf	14.47	27.78	-	-	11.52	32.46
bibliotecas_hernandez_2014_ps.pdf	18.01	37.43	19.12	-	-	-

condicionalidad_molina_2018.pdf	-	22.60	23.24	12.25	21.45	11.87
diseno_JNR_2017.pdf	40.98	13.00	11.18	14.64	-	10.74
efectividad_ros_FECYT_2010.pdf	-	28.67	10.30	15.72	28.81	11.31
estudio_CNIM_2018_ps.pdf	-	15.70	32.03	17.04	18.62	-
evaluacion_XLJA_2019_ps.pdf	19.34	18.06	10.54	10.95	17.71	23.39
globalizacion_mendez_SI_1999.pdf	23.63	20.64	-	-	52.59	-
implementacion_CASEIB_2011.pdf	-	-	15.75	38.05	22.53	-
influencia_AMF_2009.pdf	-	21.61	-	24.89	25.92	16.58
influencia_MATCOMP_2015.pdf	-	-	42.70	12.16	14.39	17.29
influencia_RM_1997.pdf	17.54	17.58	17.61	15.71	17.86	13.70
informacion_nunez-nickel_1996.pdf	45.06	13.11	14.92	-	10.53	-
innovacion_nieto_ONC_2010.pdf	-	12.29	14.88	-	36.54	22.55
interfaz_CASEIB_2011.pdf	34.68	18.77	-	14.22	13.48	10.49
introduccion_ciller_palacio_2016.pdf	16.90	19.83	18.93	10.17	24.21	-
justicia_jullien_2021.pdf	13.11	14.38	12.74	36.83	16.20	-
mecanismos_corres_2024.pdf	18.19	20.87	-	11.58	17.16	27.72
mendez_maredata_ECOSISTEMAS_2018.pdf	14.35	20.75	-	-	17.95	33.45

metadatos_mendez_FESABID2000_2000.pdf	-	16.95	18.11	-	33.56	13.35
modelado_fernandez_2011_pp.pdf	12.28	16.58	-	-	21.07	39.51
modelo_mendez_JCD_1999_ps.pdf	-	16.24	-	-	24.91	31.09
nuevas_mendez_FESABID_1998_ps.pdf	-	15.21	19.39	24.91	16.58	14.78
obtencion_AJA_2016_ps.pdf	34.82	-	-	28.53	12.59	-
organizacion_rodriguez_1993_ps.pdf	31.21	16.33	20.62	11.05	12.31	-
participacion_nieto_RSE_2009_pre.pdf	23.16	10.21	41.97	-	-	-
reduccion_RLIN_2011_ps.pdf	19.21	16.24	25.08	12.46	14.59	12.43
reparacion_Garcia_Laborum_2024.pdf	16.88	38.49	14.49	12.87	11.73	-
seguridad_Peces_1992.pdf	-	14.50	22.50	-	13.61	34.50
sincronizacion_URSI_2014.pdf	32.96	15.10	12.92	16.57	-	14.25
sistema_mallo_1996.pdf	22.19	21.58	-	14.34	11.86	20.85

Fuente: Elaboración propia

LDA ha sido el modelo más resolutivo a nivel interpretativo. Aunque no realiza una agrupación como tal, ha permitido identificar con claridad los temas dominantes en cada documento, así como los porcentajes de pertenencia a cada tópico. Esto facilitó una comprensión transversal del corpus, permitiendo detectar no solo qué documentos trataban ciertos temas, sino en qué medida lo hacían. La calidad y coherencia de los temas extraídos (como universidad e investigación, condiciones laborales, tecnologías documentales, etc.) demostraron la capacidad del modelo para representar de forma significativa el contenido.

4.4. Sistema de recomendación de documentos para el usuario

Una vez agrupados y analizados los documentos, el siguiente paso natural consiste en poner esos resultados al servicio de un usuario final. Para ello, se ha desarrollado un sistema de recomendación que, a partir de los intereses del usuario, sugiere qué documentos del corpus pueden resultarle más relevantes. De este modo, el trabajo no se limita a clasificar los textos, sino que aprovecha esa representación para facilitar su descubrimiento, que es uno de los usos más habituales de estas técnicas en repositorios institucionales y bibliotecas digitales.

Este planteamiento enlaza con el estudio de Wang et al. (2021) analizado en el apartado 3.3, donde el *clustering* se emplea para mejorar la experiencia de usuario en un sistema de recomendación de libros. A diferencia de aquel trabajo, que parte del historial de préstamos de los usuarios y utiliza aprendizaje por refuerzo, el sistema que se presenta aquí es mucho más sencillo y no necesita datos de uso: se basa únicamente en el contenido de los documentos. El código completo puede consultarse en el Anexo I.

4.4.1. Planteamiento: recomendación basada en contenido

Los sistemas de recomendación suelen dividirse en dos grandes familias. Los sistemas de filtrado colaborativo recomiendan elementos en función del comportamiento de otros usuarios con gustos similares, por lo que requieren un histórico de interacciones. Los sistemas basados en contenido, en cambio, recomiendan elementos parecidos a aquellos por los que el usuario ya ha mostrado interés, comparando directamente las características de los propios elementos. Dado que en este experimento se dispone únicamente de cincuenta documentos y de ningún dato de uso, se ha optado por un sistema basado en contenido, que además evita el problema del arranque en frío, es decir, la imposibilidad de recomendar cuando todavía no existe historial del usuario.

La idea central es aprovechar la misma representación utilizada en los modelos de *clustering*. Cada documento se describe mediante su vector TF-IDF, que se proyecta después en un espacio semántico de 30 dimensiones mediante LSA (Anexo C). En ese espacio, dos documentos con una temática parecida quedan próximos entre sí, aunque no compartan exactamente las mismas palabras. Recomendar consiste, por tanto, en situar el perfil del usuario en ese mismo espacio y devolver los documentos más cercanos, utilizando como medida de proximidad la similitud del coseno, la misma que se empleó en la agrupación de los documentos.

4.4.2. Funcionamiento del sistema

El sistema reutiliza los primeros pasos del código común (carga de los documentos preprocesados, alineación con los nombres de los PDF, cálculo de la matriz TF-IDF y

eliminación de documentos vacíos) y añade las siguientes etapas:

- Agrupación temática: sobre la representación LSA se aplica K-means con el número de clústeres seleccionado automáticamente mediante el índice de Silhouette, igual que en el Anexo C. El clúster no interviene en el cálculo de la similitud, pero se muestra junto a cada recomendación para dar contexto al usuario.

- Perfil del usuario: el usuario puede expresar sus intereses de dos formas, mediante una ventana de diálogo. La primera es escribir unas palabras clave o una frase; el texto recibe el mismo preprocesado que el corpus (minúsculas, eliminación de signos de puntuación, *stopwords* y palabras cortas), se convierte en un vector TF-IDF con los pesos del corpus y se proyecta en el espacio LSA. La segunda es seleccionar de una lista uno o varios documentos que ya le han interesado; en ese caso, el perfil es el promedio de sus vectores LSA.

- Cálculo de la similitud y ranking: se calcula la similitud del coseno entre el perfil y todos los documentos, se ordenan de mayor a menor y se seleccionan los cinco primeros. Si el perfil se ha construido a partir de documentos, estos se excluyen del ranking para no recomendar lo que el usuario ya conoce.

- Presentación de resultados: para cada documento recomendado se muestra su nombre, el porcentaje de similitud, el clúster al que pertenece y las cinco palabras con mayor peso TF-IDF, que ayudan a entender por qué se recomienda. Además, se genera un gráfico en dos dimensiones con los clústeres, el perfil del usuario y los documentos recomendados, y se ofrece la opción de abrir directamente el PDF más recomendado.

- Comprobación de coherencia: como no se dispone de valoraciones de usuarios con las que medir la calidad de las recomendaciones, el código calcula una medida interna sencilla: para cada documento, el porcentaje de sus cinco vecinos más próximos que pertenece a su mismo clúster. Un valor alto indica que las recomendaciones son coherentes con la agrupación temática obtenida.

Por ejemplo, un usuario interesado en los repositorios institucionales puede escribir esas palabras y obtener los documentos más próximos a ese tema, o bien seleccionar uno de los capítulos sobre OAI-PMH y pedir documentos similares al que ya conoce.

4.4.3. Limitaciones y posibles mejoras

El sistema debe entenderse como un prototipo. Al basarse solo en el contenido, no aprende de las preferencias del usuario con el paso del tiempo, y su calidad depende de la del preprocesado: al no disponerse de lematización en español (apartado 4.2.2), las variantes de una misma palabra, como el singular y el plural, se tratan como términos distintos. Además, una consulta formada únicamente por palabras que no aparecen en el vocabulario del corpus no

puede proyectarse, por lo que el código avisa al usuario en ese caso. Por último, la muestra de cincuenta documentos es reducida, de modo que la comprobación de coherencia realizada es solo orientativa.

Como mejoras futuras, podrían incorporarse valoraciones explícitas del usuario para ajustar su perfil con el tiempo, combinar el enfoque basado en contenido con filtrado colaborativo cuando existan datos de uso, sustituir LSA por *word embeddings* o integrar el sistema en una interfaz gráfica creada con App Designer (apartado 4.2.1), de manera que un usuario sin conocimientos de programación pueda utilizarlo.

4.5. Conclusiones y líneas futuras de investigación

Este Trabajo de Fin de Grado muestra cómo la Inteligencia Artificial puede aplicarse de forma efectiva en la clasificación automática de documentos, ofreciendo una solución viable a uno de los principales retos en la gestión de información en entornos digitales. A través del estudio de distintos enfoques teóricos y la implementación práctica de un modelo de clasificación, se ha evidenciado que las tecnologías basadas en IA no solo optimizan el tiempo y los recursos, sino que también mejoran la precisión y la estructura en la organización documental.

El trabajo representa un claro ejemplo del potencial que tiene la IA en el ámbito de la Documentación, especialmente en un contexto donde el volumen de datos crece de forma exponencial. Además, pone de relieve la necesidad de seguir investigando e incorporando estas herramientas en los sistemas de gestión de información para adaptarse a las nuevas exigencias del entorno digital.

En términos comparativos, el modelo que ha ofrecido mejores resultados en cuanto a coherencia temática y segmentación significativa ha sido la combinación de K-*Means* con reducción de dimensionalidad mediante LSA (*Latent Semantic Analysis*). Esta técnica permitió capturar relaciones semánticas profundas, generando clústeres temáticos bien definidos que reflejaban con claridad el contenido de los documentos. También destacó el modelo LDA (*Latent Dirichlet Allocation*) que, aunque no agrupa documentos en clústeres tradicionales, proporcionó una visión transversal muy rica sobre los tópicos dominantes y su distribución porcentual en cada texto.

En contraste, el modelo DBSCAN fue el menos eficaz, ya que no logró identificar clústeres con sentido temático claro y clasificó una parte significativa de los documentos como "ruido". Este resultado sugiere que la distribución de densidades en el corpus no era adecuada para este tipo de algoritmo. Por su parte, GMM (*Gaussian Mixture Models*) ofreció una agrupación más flexible, pero menos interpretativa, y en algunos casos generó clústeres temáticamente débiles o poco justificados. Finalmente, el uso de *Word Embeddings* con PCA + K-*Means* proporcionó una segmentación aceptable, aunque menos fina semánticamente que LSA, debido a las limitaciones en la representación contextual del significado.

Esta comparación permite concluir que, para tareas de clasificación documental con corpus reducido y temáticas variadas, las técnicas que integran reducción semántica (como LSA o LDA) resultan más adecuadas que los métodos puramente estadísticos o basados en densidad.

A partir del trabajo realizado, una línea futura de investigación interesante sería explorar cómo estos sistemas de análisis automático de documentos (como los basados en LDA o técnicas de *clustering*) pueden ser integrados dentro de plataformas universitarias de gestión del conocimiento. Por ejemplo, podrían aplicarse en repositorios institucionales para organizar de forma temática los Trabajos de Fin de Grado (TFG), TFM, artículos académicos o proyectos

de investigación, facilitando tanto la recuperación como el análisis comparativo entre documentos.

El primer paso consiste en aumentar la muestra de documentos para poder analizar datos de manera masiva. Una muestra de mayor tamaño permitiría dividir el conjunto en documentos de entrenamiento y de prueba (por ejemplo, en una proporción del 80 % para entrenamiento y 20 % para prueba). Con la muestra de 50 documentos que hemos utilizado para introducir y comprobar la eficacia de estos modelos de IA, no tiene sentido realizar esta división, ya que un número tan reducido de documentos de entrenamiento no permitiría al modelo aprender correctamente los patrones ni aplicarlos de forma efectiva a los documentos de prueba. Una vez dispongamos de una muestra masiva de documentos, podremos obtener métricas relevantes que indiquen, de manera objetiva y automática, qué modelos ofrecen mejores resultados y, en consecuencia, aplicarlos en los distintos repositorios.

Otro paso sería desarrollar interfaces interactivas que permitan a estudiantes, docentes e investigadores explorar visualmente los temas dominantes de un corpus, detectar tendencias emergentes o incluso sugerir líneas de trabajo basadas en vacíos temáticos. Esto también podría servir como apoyo a la dirección de trabajos académicos, proponiendo temas relevantes en función del contenido existente en el repositorio.

Además, sería interesante comparar distintos algoritmos de *topic modeling* (como LDA, NMF o BERTopic con *embeddings* más avanzados) y evaluar su impacto real en contextos educativos, analizando si efectivamente mejoran la comprensión del conocimiento almacenado. En esta línea, también resulta pertinente contrastar el rendimiento de los modelos desarrollados en entornos como MATLAB con aquellos implementados en Python, un lenguaje ampliamente utilizado en el ámbito del procesamiento de lenguaje natural y la inteligencia artificial. Python ofrece un ecosistema muy completo que incluye bibliotecas como NLTK para el preprocesamiento lingüístico, Gensim y Scikit-learn para el modelado de temas, así como BERTopic o *frameworks* como *Hugging Face Transformers* para enfoques más recientes basados en modelos de lenguaje y *embeddings* contextuales.

Por último, una integración con sistemas de recomendación personalizados podría ayudar a los estudiantes a descubrir documentos o temas afines a sus intereses o línea de investigación, fomentando así un mejor acceso al conocimiento académico.

BIBLIOGRAFÍA

Aggarwal, C. C. (2022). *Machine learning for text* (2nd ed.). Springer. https://doi.org/10.1007/978-3-030-96623-2

Baharudin, B., Lee, L. H., Khan, K., & Khan, A. (2010). A review of machine learning algorithms for text-documents classification. *Journal of Advances in Information Technology*, 1, 4–20. https://doi.org/10.4304/jait.1.1.4-20

Bilski, A. (2011). A Review of Artificial Intelligence Algorithms in Document Classification. *International Journal of Electronics and Telecommunications, 57*, 263–270. https://doi.org/10.2478/v10177-011-0035-6

Bishop, C. M. (2006). Pattern recognition and machine learning. Springer.

Brokensha, S., Kotzé, E., & Senekal, B. (2023). Machine learning for document classification in an archive of the National Afrikaans Literary Museum and Research Centre. *Journal of the Society of South African Archivists, 56,* 134–147. https://doi.org/10.4314/jsasa.v56i1.9

Couto, T., Ziviani, N., Calado, P., Cristo, M., Gonçalves, M., Moura, E., & Brandão, W. (2010). Classifying documents with link-based bibliometric measures. *Information Retrieval Journal, 13*(4), 315–345. https://doi.org/10.1007/s10791-009-9119-7

Ferrara, E. (2023). Fairness and Bias in Artificial Intelligence: A Brief Survey of Sources, Impacts, and Mitigation Strategies. *Sci, 6*, 3. https://doi.org/10.3390/sci6010003

Han, J., Kamber, M., & Pei, J. (2012). In Han J., Kamber M. and Pei J.(Eds.), *10 - Cluster Analysis: Basic Concepts and Methods.* Morgan Kaufmann. https://doi.org/10.1016/B978-0-12-381479-1.00010-1

Holdsworth, J., & Scapicchio, M. (2024, Jun 17). *¿Qué es el aprendizaje profundo?* IBM. https://www.ibm.com/mx-es/think/topics/deep-learning

IBM. *¿Qué es el aprendizaje supervisado?* IBM. https://www.ibm.com/es-es/topics/supervised-learning

IBM. *¿Qué es el bosque aleatorio?* IBM. https://www.ibm.com/mx-es/think/topics/random-forest

Jurafsky, D., & Martin, J. H. (2024). *Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition with language models* (3rd ed., draft). Stanford University and University of Colorado at Boulder.

Li, R., Liu, M., Xu, D., Gao, J., Wu, F., & Zhu, L. (2022). A Review of Machine Learning Algorithms for Text Classification. Paper presented at the *Cyber*, 226–234.

LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature, 521*(7553), 436–444. https://doi.org/10.1038/nature14539

Lorca González, M. (2024). Machine learning for libraries with Python libraries: practical case in the Library of Congress of Chile. *South African Journal of Libraries & Information Science, 90*(2), 1–7. https://doi.org/10.7553/90-2-2406

Ma, S., Zhang, C., & He, D. (2016). Document representation methods for clustering bilingual documents. *Proceedings of the Association for Information Science & Technology, 53*(1), 1–10. https://doi.org/10.1002/pra2.2016.14505301065

MathWorks. *MATLAB documentation*. MathWorks. https://www.mathworks.com/help/matlab/

Mirkin, B. (2003). Classification, Clustering, and Data Analysis. Recent Advances and Applications (Book). *Journal of Classification, 20*(2), 287–293.

Moler, C., & Little, J. (2020). A history of MATLAB. *Proc.ACM Program.Lang., 4*(HOPL)https://doi.org/10.1145/3386331

M. Dhingra, D. Dhabliya, M. K. Dubey, A. Gupta, & D. H. Reddy. (2022). A Review on Comparison of Machine Learning Algorithms for Text Classification. Paper presented at the *2022 5th International Conference on Contemporary Computing and Informatics (IC3I)*, 1818–1823. https://doi.org/10.1109/IC3I56241.2022.10072502

Neshat, N., & Kermani, A. (2024). Using the Citation-Content-Based Approach to Patent Clustering. *International Journal of Information Science & Management, 22*(2), 139–149. https://doi.org/10.22034/ijism.2024.1977932.0

Nowick, E. A., Travnicek, D., & Eskridge, K. (2010). A comparison of term clusters for tokenized words collected from controlled vocabularies, user keyword searches, and online documents. *Library Philosophy and Practice*, *2010*(NOVEMBER).

Revuelta San Emeterio, F. (2023). *Clasificación sobre Texto, Exploración de técnicas de Procesamiento de Lenguaje Natural y su impacto en el rendimiento sobre los Transformers.*

Rodríguez Pérez, D. (2023). *Clasificación de documentos con Inteligencia Artificial.*

Russell, S. J., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed., Global ed.). Pearson.

Sebastiani, F. (2002). Machine learning in automated text categorization. *ACM Comput.Surv., 34*(1), 1–47. https://doi.org/10.1145/505282.505283

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. Paper presented at the *Proceedings of the 31st International Conference on Neural Information Processing Systems*, Long Beach, California, USA. 6000–6010.

Wang, X., Wang, Y., Guo, L., Xu, L., Gao, B., Liu, F., Li, W., & Arias, J. J. P. (2021*).* Exploring Clustering-Based Reinforcement Learning for Personalized Book Recommendation in Digital Library. *Information (2078-2489), 12*(5), 198. https://doi.org/10.3390/info12050198

ANEXOS

ANEXO A. BLOQUE DE CÓDIGO

Extracción y Preprocesamiento de Texto desde PDFs en MATLAB.

```
%% === BLOQUE 1: Procesamiento de PDFs === clc;
clear; close all;

% Selección de carpeta con los documentos PDF
folderPath = uigetdir('', 'Seleccione la carpeta con los PDFs');
filePattern = fullfile(folderPath, '*.pdf');
pdfFiles = dir(filePattern);

if isempty(pdfFiles)
    error('No se encontraron archivos PDF en la carpeta seleccionada.');
end

processedFolder = fullfile(pdfFiles(1).folder, 'processed'); if
~exist(processedFolder, 'dir')
    mkdir(processedFolder);
end

documentsText = strings(length(pdfFiles), 1); for k
= 1:length(pdfFiles)
    filePath = fullfile(pdfFiles(k).folder, pdfFiles(k).name); fprintf('Procesando: %d
    %s\n', k, pdfFiles(k).name);
    try
        extractedText = extractFileText(filePath);
```

```
            if isempty(strtrim(extractedText))
                fprintf('Saltando PDF (sin texto extraíble): %s\n',
pdfFiles(k).name);
            else
                documentsText(k) = extractedText;
                movefile(filePath, fullfile(processedFolder, pdfFiles(k).name));
            end catch ME
            fprintf('Error al procesar %s: %s\n', pdfFiles(k).name, ME.message);
        end
    end

    % Preprocesamiento inicial documentsText =
    lower(documentsText);
    documents = tokenizedDocument(documentsText); documents
    = lower(documents);
    documents = erasePunctuation(documents); documents
    = removeStopWords(documents); documents =
    removeShortWords(documents, 3);

    % Eliminar palabras adicionales
    preps = ["carlos" "ante" "bajo" "cabe" "contra" "desde" "entre" "hacia" ...
             "hasta" "para" "según" "sobre" "tras" "como" "cuando" "cuándo" ...
             "cómo" "cuanto" "cuánto" "estos" "estas" "esos" "esas" ... "aquel"
             "aquella" "aquellos" "aquellas"];

    documents = removeWords(documents, preps);
```

```
% Guardar resultado save('C:\processedDocuments.mat',

'documents'); pdfNames = {pdfFiles.name}';
```

ANEXO B. BLOQUE DE CÓDIGO

Modelo K-*means*

```
clc; clear; close all;


% === Paso 1: Cargar los documentos procesados ===

% Se espera que el archivo .mat contenga un arreglo de objetos tokenizedDocument

documentsStruct = load('C:\processedDocuments.mat');

fieldNames = fieldnames(documentsStruct);

documents = documentsStruct.(fieldNames{1});

disp("Documentos cargados:");

disp(class(documents));


% === Paso 2: Cargar los nombres de los archivos PDF ===

% Se asume que están en la carpeta 'processed', ordenados alfabéticamente para
emparejar con los documentos

pdfFiles = dir('C:\docstfg\processed\*.pdf'); pdfFiles

= sortrows(struct2table(pdfFiles), 'name'); pdfFiles =

table2struct(pdfFiles);

pdfNames = {pdfFiles.name}';


% === Paso 3: Alinear el número de documentos con los nombres de archivos PDF ===

% En caso de discrepancia, se trunca al menor número de elementos

numDocs = numel(documents);

numPDFs = numel(pdfNames);


if numDocs ~= numPDFs

    warning('Número de documentos y PDFs no coincide. Se ajustarán: %d docs vs %d
PDFs.', numDocs, numPDFs);
```

```
minLength = min(numDocs, numPDFs); documents =

documents(1:minLength); pdfNames =

pdfNames(1:minLength);

end


% === Paso 4: Construir la bolsa de palabras y calcular la matriz TF-IDF ===

% La TF-IDF refleja la importancia relativa de cada palabra en cada documento

bag = bagOfWords(documents);

tfidfMatrix = tfidf(bag);  % salida en formato disperso (sparse)


% === Paso 5: Eliminar documentos vacíos o irrelevantes (con TF-IDF casi cero) ===

% Esto evita problemas numéricos y mejora la calidad del clustering

rowNorms = sqrt(sum(tfidfMatrix.^2, 2));  % norma L2 de cada documento

nonZeroRows = rowNorms > 1e-6;  % umbral mínimo

documentsClean = documents(nonZeroRows); tfidfMatrixClean

= tfidfMatrix(nonZeroRows, :); pdfNamesClean =

pdfNames(nonZeroRows);


% === Paso 6: Convertir a matriz densa para métodos que no aceptan sparse (como
evalclusters) ===

% evalclusters requiere una matriz densa cuando se usa distancia no euclídea

tfidfMatrixCleanDense = full(tfidfMatrixClean);


% === Paso 7: Determinar automáticamente el número óptimo de clústeres ===

% Se utiliza el método de silhouette con distancia coseno, evaluando entre 2 y 10
clusters

fprintf('Calculando número óptimo de clústeres...\n');

eva = evalclusters(tfidfMatrixCleanDense, 'kmeans', 'silhouette', ... 'KList',

                   2:10, 'Distance', 'cosine');
```

```
optimalK = eva.OptimalK;

fprintf('Número óptimo de clústeres encontrado: %d\n\n', optimalK);


% === Paso 8: Aplicar K-means con el número óptimo de clústeres ===

% Se utiliza distancia coseno y múltiples replicaciones para mayor estabilidad [idx,

C] = kmeans(tfidfMatrixCleanDense, optimalK, ...

               'Distance', 'cosine', 'Replicates', 5);


% === Paso 9: Visualización en 2D del clustering con reducción SVD ===

% Solo para representar gráficamente (visualización no afecta al clustering)

[U2, S2, ~] = svds(tfidfMatrixClean, 2);

score2D = U2 * S2;


figure;

gscatter(score2D(:,1), score2D(:,2), idx);

title(sprintf('Clustering de documentos con K-means y TF-IDF (K = %d)', optimalK));

xlabel('Componente 1 (SVD)');

ylabel('Componente 2 (SVD)'); grid

on;

legend(arrayfun(@(x)  sprintf('Cluster  %d',  x),  unique(idx),  'UniformOutput',
false));


% === Paso 10: Imprimir lista de documentos agrupados por clúster === fprintf('\n===

Documentos agrupados por clúster ===\n');

for i = 1:optimalK fprintf('\nCluster

    %d:\n', i);

clusterNames = pdfNamesClean(idx == i); for j

= 1:length(clusterNames)

    fprintf('- %s\n', clusterNames{j});
```

```
end

end


% === Paso 11: Mostrar las palabras clave más representativas de cada clúster ===

% Se calcula el promedio de los valores TF-IDF de las palabras por clúster

numTopWords = 10;

vocab = bag.Vocabulary;


fprintf('\n\n=== Palabras clave por clúster ===\n');

for i = 1:optimalK

clusterDocs = tfidfMatrixCleanDense(idx == i, :);

meanTfidf = mean(clusterDocs, 1);  % vector promedio de TF-IDF por palabra [~,

topIndices] = maxk(meanTfidf, numTopWords);  % top palabras con mayor peso
medio

fprintf('\nCluster %d:\n', i); disp(vocab(topIndices));

end
```

ANEXO C. BLOQUE DE CÓDIGO

Modelo K-*means* con LSA

```
clc; clear; close all;


%% === PASO 1: Cargar documentos desde archivo .mat ===
% Se parte de un archivo que contiene una variable con documentos ya tokenizados
% en formato tokenizedDocument (preprocesado: stopwords eliminadas, etc.) documentsStruct
= load('C:\processedDocuments.mat');
fieldNames = fieldnames(documentsStruct);
documents = documentsStruct.(fieldNames{1});
disp("Documentos cargados:");
disp(class(documents));


%% === PASO 2: Cargar nombres de los archivos PDF y ordenarlos alfabéticamente ===
% Se asume que los nombres están en la carpeta 'processed' y que deben coincidir pdfFiles
= dir('C:\docstfg\processed\*.pdf');
pdfFiles = sortrows(struct2table(pdfFiles), 'name');
pdfFiles = table2struct(pdfFiles);
pdfNames = {pdfFiles.name}';  % Celda de nombres


%% === PASO 3: Alinear longitud entre nombres de PDFs y documentos ===
% Si el número no coincide, se recorta al mínimo de ambos para evitar errores
numDocs = numel(documents);
numPDFs = numel(pdfNames); if
numDocs ~= numPDFs
    warning('Número de documentos y PDFs no coincide. Se ajustan a %d.',
min(numDocs, numPDFs));
minLength = min(numDocs, numPDFs);
```

```
documents = documents(1:minLength); pdfNames =

pdfNames(1:minLength);

end


%% === PASO 4: Crear bolsa de palabras y matriz TF-IDF ===

% La bolsa de palabras (bagOfWords) identifica todos los términos únicos del corpus

% La matriz TF-IDF asigna un peso a cada término en cada documento

bag = bagOfWords(documents);

tfidfMatrix = tfidf(bag);  % Resultado en formato disperso (sparse matrix)


%% === PASO 5: Eliminar documentos con muy poca información (norma ≈ 0) ===

% Esto evita problemas numéricos y mejora la calidad del clustering

rowNorms = sqrt(sum(tfidfMatrix.^2, 2));  % Norma L2 de cada fila

nonZeroRows = rowNorms > 1e-6;  % Umbral para considerar contenido suficiente

documentsClean = documents(nonZeroRows);

pdfNamesClean = pdfNames(nonZeroRows);

tfidfMatrixClean = tfidfMatrix(nonZeroRows, :);


%% === PASO 6: Aplicar LSA (SVD) para reducción semántica ===

% LSA = Latent Semantic Analysis. Aquí se usa SVD para extraer componentes semánticos.

% Esto permite identificar temas latentes y relaciones más profundas entre documentos

numComponents = 30;  % Número de dimensiones semánticas (ajustable)

[U, S, ~] = svds(tfidfMatrixClean, numComponents);  % SVD truncado

lsaFeatures = U * S;  % Proyección de los documentos en el espacio semántico


%% === PASO 7: Calcular número óptimo de clústeres con método de silhouette ===

% evalclusters calcula la coherencia interna de cada agrupación para K = 2 a 12
```

```
% Se usa distancia 'cosine' por ser adecuada con texto y TF-IDF

disp('Calculando número óptimo de clústeres con silhouette...');

eva = evalclusters(lsaFeatures, 'kmeans', 'silhouette', ...

                   'KList', 2:12, 'Distance', 'cosine');

optimalK = eva.OptimalK;

fprintf('Número óptimo de clústeres: %d\n\n', optimalK);


%% === PASO 8: Aplicar K-means sobre el espacio semántico reducido ===

% Se ejecuta K-means con replicación para obtener resultados estables

[idx, C] = kmeans(lsaFeatures, optimalK, ...

'Distance', 'cosine', 'Replicates', 5);


%% === PASO 9: Visualización 2D (solo para graficar resultados) ===

% Se usa SVD con 2 componentes para representar los documentos en 2D

% No afecta al clustering, es solo para interpretación visual [U2,

S2, ~] = svds(tfidfMatrixClean, 2);

score2D = U2 * S2;


figure;

gscatter(score2D(:,1), score2D(:,2), idx);

title(sprintf('Clustering de documentos con K-means + LSA (%d dimensiones)',
numComponents));

xlabel('Componente 1 (SVD)');

ylabel('Componente 2 (SVD)'); grid

on;

legend(arrayfun(@(x) sprintf('Cluster %d', x), 1:optimalK, 'UniformOutput',
false));


%% === PASO 10: Mostrar los documentos agrupados por clúster ===
```

```
fprintf('\n=== Documentos agrupados por clúster (con LSA) ===\n');

for i = 1:optimalK

fprintf('\nCluster %d:\n', i); clusterDocs =

pdfNamesClean(idx == i); for j =

1:length(clusterDocs)

    fprintf('- %s\n', clusterDocs{j});

end

end


%% === PASO 11: Palabras clave más representativas por clúster ===

% Aunque el clustering se hizo con LSA, usamos TF-IDF para ver qué palabras dominan
por grupo

vocab = bag.Vocabulary;

tfidfMatrixDense = full(tfidfMatrixClean);

numTopWords = 10;


fprintf('\n\n=== Palabras clave por clúster (con LSA) ===\n'); for

i = 1:optimalK

clusterDocs = tfidfMatrixDense(idx == i, :);

meanTfidf = mean(clusterDocs, 1);  % Media TF-IDF por palabra en el clúster [~,

topIdx]  =  maxk(meanTfidf,  numTopWords);  %  Términos  más  representativos

fprintf('\nCluster %d:\n', i);

disp(vocab(topIdx));

end
```

ANEXO D. BLOQUE DE CÓDIGO

Modelo *Clustering* Jerárquico

```
clc; clear; close all;

%% === PASO 1: Cargar documentos tokenizados desde archivo .mat ===
documentsStruct = load('C:\processedDocuments.mat');
fieldNames = fieldnames(documentsStruct);
documents = documentsStruct.(fieldNames{1});
disp("Documentos cargados:");
disp(class(documents));

%% === PASO 2: Cargar y ordenar nombres de archivos PDF ===
pdfFiles = dir('C:\docstfg\processed\*.pdf');
pdfFiles = sortrows(struct2table(pdfFiles), 'name');
pdfFiles = table2struct(pdfFiles);
pdfNames = {pdfFiles.name}';

%% === PASO 3: Alinear longitud de documentos y nombres de archivo ===
numDocs = numel(documents);
numPDFs = numel(pdfNames); if
numDocs ~= numPDFs
    warning('Número de documentos y PDFs no coincide. Se ajusta a %d.', min(numDocs,
numPDFs));
minLength = min(numDocs, numPDFs); documents =
documents(1:minLength); pdfNames =
pdfNames(1:minLength);
end
```

```
%% === PASO 4: Calcular matriz TF-IDF ===
bag = bagOfWords(documents);
tfidfMatrix = tfidf(bag);  % Matriz dispersa

%% === PASO 5: Filtrar documentos con contenido nulo o casi nulo ===
rowNorms = sqrt(sum(tfidfMatrix.^2, 2));
validDocs = rowNorms > 1e-6; documentsClean
= documents(validDocs); pdfNamesClean =
pdfNames(validDocs);
tfidfMatrixClean = tfidfMatrix(validDocs, :);

%% === PASO 6 (opcional): Aplicar reducción semántica con LSA ===
applyLSA = true;  % Cambia a false si no quieres usar LSA
if applyLSA
numComponents = 30;  % Dimensiones semánticas
[U, S, ~] = svds(tfidfMatrixClean, numComponents);
docMatrix = U * S;  % Representación semántica
else
docMatrix = full(tfidfMatrixClean);  % Sin reducción
end

%% === PASO 7: Calcular matriz de distancias y aplicar linkage jerárquico ===
distMatrix = pdist(docMatrix, 'cosine');          % Distancia coseno
Z = linkage(distMatrix, 'average');                % Método de enlace

%% === PASO 8: Visualización del dendrograma con etiquetas a la izquierda ===
figure('Position', [100, 100, 1200, 800]);        % Ventana grande
maxLabels = 60;                                        % Número máximo de etiquetas
visibles
```

```
if length(pdfNamesClean) > maxLabels

dendrogram(Z, maxLabels, 'Orientation', 'right');

    title(sprintf('Dendrograma (primeros %d documentos)', maxLabels), 'FontSize',
14);

else

dendrogram(Z, 0, 'Labels', pdfNamesClean, 'Orientation', 'right');

title('Dendrograma - Clustering Jerárquico', 'FontSize', 14);

end


xlabel('Distancia (coseno)');

ylabel('Documentos'); set(gca,

'FontSize', 10); grid on;


%% === PASO 9: Crear agrupación jerárquica con K clústeres === K

= 10;  % Puedes ajustar el número de clústeres deseado clusters

= cluster(Z, 'maxclust', K);  % Agrupación automática


%% === PASO 10: Mostrar documentos agrupados por clúster ===

fprintf('\n=== Documentos agrupados por clúster (Jerárquico) ===\n');

for i = 1:K

fprintf('\nCluster %d:\n', i);

clDocs = pdfNamesClean(clusters == i); for j =

1:length(clDocs)

    fprintf('- %s\n', clDocs{j});

end

end
```

```
%% === PASO 11: Mostrar palabras clave representativas por clúster ===

fprintf('\n=== Palabras clave representativas por clúster ===\n'); vocab

= bag.Vocabulary;

tfidfMatrixDense = full(tfidfMatrixClean);     % Convertimos sparse a full para
cálculos

numTopWords = 10;


for i = 1:K

clusterDocs = tfidfMatrixDense(clusters == i, :);

meanTfidf = mean(clusterDocs, 1);  % Media TF-IDF por término

    [~, topIdx] = maxk(meanTfidf, numTopWords);       % Índices de términos más
representativos

fprintf('\nCluster %d:\n', i); disp(vocab(topIdx));

end
```

ANEXO E. BLOQUE DE CÓDIGO

Modelo DBSACN

```
clc; clear; close all;

%% === PASO 1: Cargar documentos tokenizados desde archivo .mat ===
% Se carga el archivo que contiene los documentos ya tokenizados
documentsStruct = load('C:\processedDocuments.mat');
fieldNames = fieldnames(documentsStruct);
documents = documentsStruct.(fieldNames{1});
disp("Documentos cargados:");
disp(class(documents));  % Debería mostrar 'tokenizedDocument'

%% === PASO 2: Cargar nombres de los archivos PDF ===
% Se cargan los nombres de los archivos PDF ya procesados, ordenados alfabéticamente
pdfFiles = dir('C:\docstfg\processed\*.pdf');
pdfFiles = sortrows(struct2table(pdfFiles), 'name');
pdfFiles = table2struct(pdfFiles);
pdfNames = {pdfFiles.name}';

%% === PASO 3: Alinear número de documentos con nombres de PDF ===
% Se ajusta a la cantidad mínima para evitar desajustes
minLength = min(numel(documents), numel(pdfNames));
documents = documents(1:minLength);
pdfNames = pdfNames(1:minLength);

%% === PASO 4: Calcular matriz TF-IDF (Bolsa de palabras + pesos) ===
% Se construye la bolsa de palabras y luego se calcula TF-IDF
```

```
bag = bagOfWords(documents);

tfidfMatrix = tfidf(bag);  % Matriz dispersa que representa el peso de cada palabra


%% === PASO 5: Eliminar documentos vacíos o sin contenido relevante ===

rowNorms = sqrt(sum(tfidfMatrix.^2, 2));  % Calculamos la norma L2 por fila

valid = rowNorms > 1e-6;                          % Umbral mínimo para considerar un
documento válido

documentsClean = documents(valid);

pdfNamesClean = pdfNames(valid);

tfidfMatrixClean = tfidfMatrix(valid, :);


%% === PASO 6: Reducción semántica con LSA (Latent Semantic Analysis) ===

% Aplicamos SVD para obtener una representación semántica compacta

numComponents = 30;  % Número de dimensiones semánticas (ajustable)

[U, S, ~] = svds(tfidfMatrixClean, numComponents);

lsaFeatures = U * S;  % Matriz con los documentos proyectados en espacio semántico
reducido


%% === PASO 7: Aplicar DBSCAN con distancia coseno ===

% DBSCAN permite identificar clústeres de densidad sin definir K

% También identifica puntos como "ruido" (no pertenecientes a ningún clúster)

epsilon = 0.5;       % Radio de vecindad (ajustable: prueba 0.3 - 0.7)

minPts = 3;          % Mínimo de puntos para formar un clúster denso


% Como DBSCAN no acepta directamente distancia coseno en vector, usamos matriz
precomputada

Y = pdist2(lsaFeatures, lsaFeatures, 'cosine');  % Distancia coseno entre documentos

dbIdx = dbscan(Y, epsilon, minPts, 'Distance', 'precomputed');  % -1 indica "ruido"


%% === PASO 8: Visualización 2D usando reducción con SVD ===
```

```
[U2, S2, ~] = svds(tfidfMatrixClean, 2);  % Para visualizar en 2D
coords2D = U2 * S2;

figure('Position', [100, 100, 1000, 600]);
gscatter(coords2D(:,1), coords2D(:,2), dbIdx); title('Clustering
con DBSCAN (LSA + Coseno)', 'FontSize', 14); xlabel('Componente
1 (SVD)');
ylabel('Componente 2 (SVD)'); grid
on;

%% === PASO 9: Mostrar documentos agrupados por clúster ===
fprintf('\n=== Documentos agrupados por DBSCAN ===\n');
uniqueClusters = unique(dbIdx);  % Incluye -1 si hay ruido

for i = uniqueClusters' if
    i == -1
    fprintf('\n Ruido / No asignado:\n'); else
    fprintf('\nCluster %d:\n', i);
end
docsInCluster = pdfNamesClean(dbIdx == i); for j
= 1:length(docsInCluster)
    fprintf('- %s\n', docsInCluster{j});
end
end
```

ANEXO F. BLOQUE DE CÓDIGO

Modelo GMM

```
clc; clear; close all;

%% === PASO 1: Cargar documentos desde archivo .mat ===
documentsStruct = load('C:\processedDocuments.mat');
fieldNames = fieldnames(documentsStruct);
documents = documentsStruct.(fieldNames{1});
disp("Documentos cargados:");
disp(class(documents));

%% === PASO 2: Cargar nombres de PDF ordenados ===
pdfFiles = dir('C:\docstfg\processed\*.pdf'); pdfFiles
= sortrows(struct2table(pdfFiles), 'name'); pdfFiles =
table2struct(pdfFiles);
pdfNames = {pdfFiles.name}';

%% === PASO 3: Alinear documentos y nombres === minLength
= min(numel(documents), numel(pdfNames)); documents =
documents(1:minLength);
pdfNames = pdfNames(1:minLength);

%% === PASO 4: Matriz TF-IDF ===
bag = bagOfWords(documents);
tfidfMatrix = tfidf(bag);

%% === PASO 5: Filtrar documentos vacíos ===
```

```
rowNorms = sqrt(sum(tfidfMatrix.^2, 2)); valid
= rowNorms > 1e-6;
documentsClean = documents(valid);
pdfNamesClean = pdfNames(valid);
tfidfMatrixClean = tfidfMatrix(valid, :);

%% === PASO 6: Reducción semántica con LSA ===
numComponents = 30;
[U, S, ~] = svds(tfidfMatrixClean, numComponents);
lsaFeatures = U * S;

%% === PASO 7: Estimar número óptimo de clústeres con BIC ===
maxClusters = 12;
options = statset('MaxIter', 500);
BIC = zeros(1, maxClusters);

fprintf("Calculando BIC para encontrar número óptimo de clústeres...\n");
for k = 1:maxClusters
try
    gm = fitgmdist(lsaFeatures, k, ...
        'CovarianceType', 'diagonal', ...
        'RegularizationValue', 1e-5, ... 'Options',
        options, ...
        'Replicates', 3); BIC(k) =
    gm.BIC;
catch
    BIC(k) = Inf;  % Si falla, asignamos infinito
    fprintf("Error en k = %d, se descarta por inestabilidad.\n", k);
```

```
end
end

[~, optimalK] = min(BIC);
fprintf("Número óptimo de clústeres (BIC): %d\n", optimalK);

%% === PASO 8: Entrenar GMM final con mejores parámetros === gmm
= fitgmdist(lsaFeatures, optimalK, ...
'CovarianceType','diagonal', ...
'RegularizationValue',1e-5, ...
'Options', options, ...
'Replicates', 5);

clusterIdx = cluster(gmm, lsaFeatures);  % Asignación final de clústeres

%% === PASO 9: Visualización en 2D con SVD ===
[U2, S2, ~] = svds(tfidfMatrixClean, 2); coords2D
= U2 * S2;

figure('Position', [100, 100, 1000, 600]);
gscatter(coords2D(:,1), coords2D(:,2), clusterIdx);
title(sprintf('Clustering GMM (K = %d, LSA + Coseno)', optimalK), 'FontSize', 14);
xlabel('Componente 1'); ylabel('Componente 2');
grid on;

%% === PASO 10: Mostrar PDFs agrupados por clúster ===
fprintf('\n=== Documentos agrupados por GMM ===\n'); for
k = 1:optimalK
```

```
fprintf('\nCluster %d:\n', k);
docs = pdfNamesClean(clusterIdx == k); for i =
1:length(docs)
    fprintf('- %s\n', docs{i});
end
end
```

ANEXO G. BLOQUE DE CÓDIGO

Modelo K-means con PCA (alternativa a word embeddings)

```
clc; clear; close all;

%% === PASO 1: Cargar documentos tokenizados desde archivo .mat ===
documentsStruct = load('C:\processedDocuments.mat');
fieldNames = fieldnames(documentsStruct);
documents = documentsStruct.(fieldNames{1});
disp("Documentos cargados:");
disp(class(documents));  % tokenizedDocument

%% === PASO 2: Cargar nombres de archivos PDF ===
pdfFiles = dir('C:\docstfg\processed\*.pdf'); pdfFiles
= sortrows(struct2table(pdfFiles), 'name'); pdfFiles =
table2struct(pdfFiles);
pdfNames = {pdfFiles.name}';

%% === PASO 3: Alinear longitud documentos - nombres ===
minLength = min(numel(documents), numel(pdfNames));
documents = documents(1:minLength);
pdfNames = pdfNames(1:minLength);

%% === PASO 4: Calcular matriz TF-IDF ===
bag = bagOfWords(documents);
tfidfMatrix = tfidf(bag);  % Matriz dispersa (sparse)

%% === PASO 5: Eliminar documentos vacíos ===
```

```
rowNorms = sqrt(sum(tfidfMatrix.^2, 2)); valid
= rowNorms > 1e-6;
documentsClean = documents(valid);
pdfNamesClean = pdfNames(valid);
tfidfMatrixClean = tfidfMatrix(valid, :);

%% === PASO 6: Reducción dimensional con PCA (pseudo-embeddings) ===
% Convertimos TF-IDF a formato denso
tfidfDense = full(tfidfMatrixClean);

% Aplicamos PCA para reducir a 50 dimensiones
numComponents = 50;
[coeff, score, ~] = pca(tfidfDense, 'NumComponents', numComponents);

% "score" es nuestra representación densa por documento (pseudo-embedding)
docVectors = score;

%% === PASO 7: Clustering con K-means sobre embeddings PCA ===
numClusters = 8;  % Puedes ajustar este valor o usar silhouette [idx,
~] = kmeans(docVectors, numClusters, ...
'Distance', 'cosine', ...
'Replicates', 5);

%% === PASO 8: Visualización 2D (PCA 2 componentes) ===
[coeff2D, score2D] = pca(tfidfDense, 'NumComponents', 2);
figure('Position', [100, 100, 1000, 600]);
gscatter(score2D(:,1), score2D(:,2), idx); title('Clustering
con TF-IDF + PCA (2D)', 'FontSize', 14);
```

```
xlabel('Componente 1'); ylabel('Componente 2');
grid on;

%% === PASO 9: Mostrar PDFs agrupados por clúster ===
fprintf('\n=== Documentos agrupados con PCA + K-means ===\n'); for
i = 1:numClusters
fprintf('\nCluster %d:\n', i); clusterDocs =
pdfNamesClean(idx == i); for j =
1:length(clusterDocs)
    fprintf('- %s\n', clusterDocs{j});
end
end
```

ANEXO H. BLOQUE DE CÓDIGO

Modelo LDA (Análisis de tópicos)

```
clc; clear; close all;

%% === PASO 1: Cargar documentos tokenizados ===
documentsStruct = load('C:\processedDocuments.mat');
fieldNames = fieldnames(documentsStruct);
documents = documentsStruct.(fieldNames{1});
disp("Documentos cargados:");
disp(class(documents));  % Debería ser tokenizedDocument

%% === PASO 2: Crear bolsa de palabras (bag-of-words) ===
bag = bagOfWords(documents);
bag = removeInfrequentWords(bag, 2);     % Elimina palabras poco frecuentes
bag = removeEmptyDocuments(bag);         % Elimina documentos vacíos

%% === PASO 3: Entrenar modelo LDA ===
numTopics = 6;  % Puedes ajustar este número
mdl = fitlda(bag, numTopics, 'Verbose', 1);

%% === PASO 4: Mostrar palabras clave por tópico ===
disp("=== Tópicos detectados ===");
for i = 1:numTopics fprintf("\nTópico
    %d:\n", i); words =
    topkwords(mdl, 10, i);
    disp(words);
end
```

```
%% === PASO 5: Obtener distribución temática por documento ===
docTopicProb = transform(mdl, bag);  % Matriz: documentos × temas

%% === PASO 6: Cargar nombres de PDF y alinear ===
pdfDir = dir('C:\docstfg\processed\*.pdf');

if numel(pdfDir) == 0
error('No se encontraron archivos PDF en la carpeta especificada.');
end

pdfTable = struct2table(pdfDir);
pdfTable = sortrows(pdfTable, 'name');     % Orden alfabético
pdfNames = pdfTable.name;

if ischar(pdfNames)
pdfNames = {pdfNames};  % Asegurar formato de celda
end

% Alinear número de nombres con número de documentos
pdfNames = pdfNames(1:min(numel(pdfNames), size(docTopicProb, 1)));

%% === PASO 7: Mostrar distribución de temas por documento === fprintf("\n===
Distribución de temas por documento (en porcentajes) ===\n");

for i = 1:size(docTopicProb, 1)
    fprintf('\n📄 %s\n', pdfNames{i});
```

```
[sortedProbs, sortedIdx] = sort(docTopicProb(i, :), 'descend');

% Tópico dominante
fprintf('Tema principal: Tópico %d (%.2f%%)\n', ...
    sortedIdx(1), sortedProbs(1)*100);

% Temas secundarios (≥10%) for j =
2:numTopics
    if sortedProbs(j) >= 0.10
        fprintf('Tópico secundario: %d (%.2f%%)\n', ... sortedIdx(j),
            sortedProbs(j)*100);
    end
end
end
```

ANEXO I. BLOQUE DE CÓDIGO

Sistema de recomendación para el usuario

```
clc; clear; close all;

%% === PASO 1: Cargar documentos tokenizados desde archivo .mat ===
% Se parte del archivo generado en el Anexo A (documentos preprocesados)
documentsStruct = load('C:\processedDocuments.mat');
fieldNames = fieldnames(documentsStruct);
documents = documentsStruct.(fieldNames{1});
disp("Documentos cargados:");
disp(class(documents)); % Debería mostrar 'tokenizedDocument'

%% === PASO 2: Cargar nombres de los archivos PDF ordenados ===
pdfFiles = dir('C:\docstfg\processed\*.pdf');
pdfFiles = sortrows(struct2table(pdfFiles), 'name');
pdfFiles = table2struct(pdfFiles);
pdfNames = {pdfFiles.name}';
pdfFolder = pdfFiles(1).folder; % Carpeta de los PDF (para poder abrirlos)

%% === PASO 3: Alinear número de documentos y nombres de archivo ===
minLength = min(numel(documents), numel(pdfNames));
documents = documents(1:minLength);
pdfNames = pdfNames(1:minLength);

%% === PASO 4: Calcular la matriz TF-IDF ===
bag = bagOfWords(documents);
tfidfMatrix = tfidf(bag); % Matriz dispersa (sparse)

%% === PASO 5: Eliminar documentos vacíos o sin contenido relevante ===
rowNorms = sqrt(sum(tfidfMatrix.^2, 2));
valid = rowNorms > 1e-6;
```

```
documentsClean = documents(valid);
pdfNamesClean = pdfNames(valid);
tfidfMatrixClean = tfidfMatrix(valid, :);
numDocsClean = numel(pdfNamesClean);

%% === PASO 6: Reducción semántica con LSA ===
% Se guarda también V para poder proyectar después la consulta del usuario
numComponents = 30; % Número de dimensiones semánticas (ajustable)
[U, S, V] = svds(tfidfMatrixClean, numComponents);
lsaFeatures = U * S; % Documentos en el espacio semántico
% Se normalizan los vectores: producto escalar = similitud coseno
lsaNorm = lsaFeatures ./ max(sqrt(sum(lsaFeatures.^2, 2)), eps);

%% === PASO 7: Agrupación temática con K-means (como en el Anexo C) ===
fprintf('Calculando número óptimo de clústeres...\n');
eva = evalclusters(lsaFeatures, 'kmeans', 'silhouette', ...
    'KList', 2:12, 'Distance', 'cosine');
optimalK = eva.OptimalK;
[idx, ~] = kmeans(lsaFeatures, optimalK, ...
    'Distance', 'cosine', 'Replicates', 5);
fprintf('Número de clústeres: %d\n', optimalK);

%% === PASO 8: Obtener el perfil del usuario ===
% Se puede indicar con palabras clave o con documentos que ya le han interesado
modo = questdlg('¿Cómo desea indicar sus intereses?', ...
    'Sistema de recomendación', ...
    'Palabras clave', 'Documentos de interés', 'Palabras clave');
if isempty(modo)
    error('Operación cancelada por el usuario.');
end
excluded = false(numDocsClean, 1); % Documentos que no se recomendarán
if strcmp(modo, 'Palabras clave')
```

```
    % --- Opción A: consulta de texto libre ---
    answer = inputdlg('Escriba palabras clave o una frase sobre su interés:', ...
        'Perfil del usuario', [1 70]);
    if isempty(answer) || isempty(strtrim(answer{1}))
        error('No se ha introducido ninguna consulta.');
    end
    % Mismo preprocesado que el aplicado al corpus (Anexo A)
    queryDoc = tokenizedDocument(lower(string(answer{1})));
    queryDoc = erasePunctuation(queryDoc);
    queryDoc = removeStopWords(queryDoc);
    queryDoc = removeShortWords(queryDoc, 3);
    % TF-IDF con el vocabulario y los pesos IDF del corpus
    % (las palabras que no están en el vocabulario se ignoran)
    queryTfidf = tfidf(bag, queryDoc);
    if nnz(queryTfidf) == 0
        error('Ninguna palabra de la consulta aparece en el vocabulario.');
    end
    userProfile = full(queryTfidf * V); % Proyección en el espacio LSA
else
    % --- Opción B: documentos que ya le han interesado ---
    [sel, ok] = listdlg('PromptString', 'Seleccione los documentos de interés:', ...
        'ListString', pdfNamesClean, 'SelectionMode', 'multiple', ...
        'ListSize', [450 350], 'Name', 'Perfil del usuario');
    if ~ok || isempty(sel)
        error('No se ha seleccionado ningún documento.');
    end
    userProfile = mean(lsaFeatures(sel, :), 1); % Promedio de los elegidos
    excluded(sel) = true; % No se recomiendan los ya elegidos
end

%% === PASO 9: Calcular la similitud y generar el ranking ===
profileNorm = userProfile / max(norm(userProfile), eps);
```

```
similarity = lsaNorm * profileNorm'; % Similitud coseno con cada documento
similarity(excluded) = -Inf;
topN = min(5, sum(~excluded)); % Número de documentos a recomendar
[sortedSim, order] = sort(similarity, 'descend');
recommended = order(1:topN);

%% === PASO 10: Mostrar los documentos recomendados ===
vocab = bag.Vocabulary;
tfidfDense = full(tfidfMatrixClean);
numKeywords = 5;
fprintf('\n=== Documentos recomendados ===\n');
for r = 1:topN
    docId = recommended(r);
    [~, topIdx] = maxk(tfidfDense(docId, :), numKeywords);
    fprintf('\n%d. %s\n', r, pdfNamesClean{docId});
    fprintf('   Similitud: %.1f%% | Cluster: %d\n', ...
        max(sortedSim(r), 0) * 100, idx(docId));
    fprintf('   Palabras clave: %s\n', strjoin(vocab(topIdx), ', '));
end
fprintf('\nClúster predominante: %d\n', mode(idx(recommended)));

%% === PASO 11: Visualización 2D del perfil y las recomendaciones ===
[U2, S2, V2] = svds(tfidfMatrixClean, 2);
coords2D = U2 * S2;
if strcmp(modo, 'Palabras clave')
    profile2D = full(queryTfidf * V2);
else
    profile2D = mean(coords2D(sel, :), 1);
end
figure('Position', [100, 100, 1000, 600]);
gscatter(coords2D(:,1), coords2D(:,2), idx);
hold on;
```

```
scatter(coords2D(recommended,1), coords2D(recommended,2), ...
    180, 'k', 'p', 'filled');
scatter(profile2D(1), profile2D(2), 180, 'r', 'd', 'filled');
title('Perfil del usuario y documentos recomendados', 'FontSize', 14);
xlabel('Componente 1 (SVD)'); ylabel('Componente 2 (SVD)'); grid on;
clusterLabels = arrayfun(@(x) sprintf('Cluster %d', x), ...
    unique(idx)', 'UniformOutput', false);
legend([clusterLabels, {'Recomendados', 'Perfil del usuario'}], ...
    'Location', 'best');

%% === PASO 12: Comprobación de coherencia con la agrupación ===
% Se comprueba si los vecinos más próximos de cada documento comparten clúster
kNeighbors = min(5, numDocsClean - 1);
simMatrix = lsaNorm * lsaNorm';
simMatrix(1:numDocsClean+1:end) = -Inf; % Se excluye el propio documento
[~, neighbors] = maxk(simMatrix, kNeighbors, 2);
sameCluster = idx(neighbors) == idx;
fprintf('\nCoherencia: %.1f%% de los %d vecinos comparten clúster.\n', ...
    mean(sameCluster(:)) * 100, kNeighbors);

%% === PASO 13 (opcional): Abrir el documento más recomendado ===
best = pdfNamesClean{recommended(1)};
msg = sprintf('¿Desea abrir el documento más recomendado?\n%s', best);
openDoc = questdlg(msg, 'Abrir documento', 'Sí', 'No', 'Sí');
if strcmp(openDoc, 'Sí')
    winopen(fullfile(pdfFolder, best));
end
```

ANEXO J. DECLARACIÓN DE USO DE IA GENERATIVA EN EL TRABAJO DE FIN DE GRADO

He usado IA Generativa en este trabajo

Marca lo que corresponda:

SI	NO

Si has marcado SI, completa las siguientes 3 partes de este documento:

Parte 1: reflexión sobre comportamiento ético y responsable

Ten presente que el uso de IA Generativa conlleva unos riesgos y puede generar una serie de consecuencias que afecten a la integridad moral de tu actuación con ella. Por eso, te pedimos que contestes con honestidad a las siguientes preguntas (*Marca lo que corresponda*):

Pregunta		
1. En mi interacción con herramientas de IA Generativa he remitido **datos de carácter sensible** con la debida autorización de los interesados.		
SÍ, he usado estos datos con autorización	NO, he usado estos datos sin autorización	NO, no he usado datos de carácter sensible
2. En mi interacción con herramientas de IA Generativa he remitido **materiales protegidos por derechos de autor** con la debida autorización de los interesados.		
SÍ, he usado estos materiales con autorización	NO, he usado estos materiales sin autorización	NO, no he usado materiales protegidos
3. En mi interacción con herramientas de IA Generativa he remitido **datos de carácter personal** con la debida autorización de los interesados.		
SÍ, he usado estos datos con autorización	NO, he usado estos datos sin autorización	NO, no he usado datos de carácter personal

4. Mi utilización de la herramienta de IA Generativa ha **respetado sus términos de uso**, así como los principios éticos esenciales, no orientándola de manera maliciosa a obtener un resultado inapropiado para el trabajo presentado, es decir, que produzca una impresión o conocimiento contrario a la realidad de los resultados obtenidos, que suplante mi propio trabajo o que pueda resultar en un perjuicio para las personas.

SI	NO

Si **NO** has contado con la autorización de los interesados en alguna de las preguntas 1, 2 ó 3, explica brevemente el motivo *(por ejemplo, "los materiales estaban protegidos pero permitían su uso para este fin" o "los términos de uso, que se pueden encontrar en esta dirección (...), impiden el uso que he hecho, pero era imprescindible dada la naturaleza del trabajo".*

Parte 2: declaración de uso técnico

Utiliza el siguiente modelo de declaración tantas veces como sea necesario, a fin de reflejar todos los tipos de iteración que has tenido con herramientas de IA Generativa. Incluye un ejemplo por cada tipo de uso realizado donde se indique: *[Añade un ejemplo]*.

Declaro haber hecho uso del sistema de IA Generativa (Nombre del sistema/herramienta IA y versión: ChatGPT, Gemini, Copilot…) **para:**

Documentación y redacción:

- *Soporte a la reflexión en relación con el desarrollo del trabajo: proceso iterativo de análisis de alternativas y enfoques utilizando la IA*

- *Revisión o reescritura de párrafos redactados previamente*

Declaro que he usado ChatGPT o4-mini para explicar ideas de forma más clara y accesible. La IA me permitió reformular lo que yo entendía internamente pero no lograba expresar bien, facilitando que otras personas puedan comprender mejor mis planteamientos.

- *Búsqueda de información o respuesta a preguntas concretas*

- *Búsqueda de bibliografía*
- *Resumen de bibliografía consultada*
- *Traducción de texto consultados*

Desarrollar contenido específico

Se ha hecho uso de IA Generativa como herramienta de soporte para el desarrollo del contenido específico del TFG, incluyendo:

- *Asistencia en el desarrollo de líneas de código (programación)*

He utilizado IA Generativa como soporte para generar código en lenguaje MATLAB como bien se expone en el comienzo del apartado 4.2.3. "Algoritmos en MATLAB para la Clasificación Automática de Documentos" correspondiente a este TFG. Dicho código ha contado siempre con supervisión y razonamiento humano.

- *Generación de esquemas, imágenes, audios o vídeos*

- *Procesos de optimización*

He solicitado ayuda para corregir y explicar errores de un código que no funcionaba correctamente, o para conocer la explicación de alguna función a implementar en el modelo que desconocía.

- *Tratamiento de datos: recogida, análisis, cruce de datos…*

- *Inspiración de ideas en el proceso creativo*

- *Otros usos vinculados a la generación de puntos concretos del desarrollo específico del trabajo*

Si crees necesario incluirlo, añade, en cada uno de los casos:

empleando el *prompt* …
(escribe la petición realizada a la IAG)
teniendo como interacción …
(describe qué interacción realizaste con la IAG tras la respuesta del prompt)

Ejemplo: ***Declaro*** *haber hecho uso del sistema de IA Generativa* ***Chat GPT 3.5*** *para* ***buscar información*** *empleando* ***el prompt: "Dime un ejemplo concreto que ilustre la***

aplicación de la propuesta urbanística de la Ciudad de los 15 minutos en España" *teniendo como interacción* ***la proposición del ejemplo concreto del distrito de Chamartín que se ha reflejado en la memoria****.*

Parte 3: reflexión sobre utilidad

Por favor, aporta una valoración personal (formato libre) sobre las fortalezas y debilidades que has identificado en el uso de herramientas de IA Generativa en el desarrollo de tu trabajo. Menciona si te ha servido en el proceso de aprendizaje, o en el desarrollo o en la extracción de conclusiones de tu trabajo.

El uso de herramientas de IA Generativa ha supuesto una ventaja significativa, especialmente en el ámbito de la programación. Su capacidad para detectar errores, sugerir mejoras en el código y explicar fragmentos complejos ha sido de gran ayuda para agilizar el desarrollo técnico del proyecto. En alguna ocasión, cuando me encontraba atascado en algún error o necesitaba implementar funcionalidades específicas y desconocidas por mi persona, estas herramientas me ofrecieron soluciones rápidas y comprensibles, ahorrándome horas de búsqueda y prueba-error.

Por otro lado, también he encontrado útil la IA generativa en la parte más conceptual del trabajo, particularmente al intentar explicar ideas de una forma más clara y accesible. A veces, lo que yo entendía internamente no era tan fácil de expresar de manera que otras personas pudieran comprenderlo igual de bien. La IA me ayudó a reformular estas ideas, detectando cuándo una idea sonaba confusa, permitiéndome comunicar de forma más efectiva lo que antes me costaba transmitir con claridad.